

RDB Connection Function

KV-XCM02

User's Manual

Read this manual before using the product.
Keep this manual in a safe place for later reference.

Chapter 1	Overview of the Unit
Chapter 2	DB Mapping Configuration
Chapter 3	RDB Connection Settings
Chapter 4	RDB Connection Function
Chapter 5	Function Block

- Error Code
- Restriction
- Quotation/String Identifier for Each RDB
- SQL Command Description
- List of Devices Used
- Troubleshooting

Preface

This manual provides an overview of the KV-XCM02, its function, and how to use it.

Be sure to carefully read and fully understand this manual before using the product. Additionally, store this manual in a safe place so that you can retrieve it whenever necessary.

Related Manuals

All the following PDF manuals can be found and opened in the help file of KV STUDIO. Additionally, the latest versions of the PDF manuals can be downloaded from the KEYENCE website.

Name	Description	Binding
RDB Connection User's Manual	This manual. Explains the RDB connection available for KV-XCM02, as well as the procedures and specifications for connecting to the RDB.	None (Only PDF)
KV-XCM02 User's Manual	Explains the system configuration, specifications, and methods for creating ladder programs for the "KV-XCM02".	None (Only PDF)
KV-8000 Series User's Manual	System configuration and specifications of the "KV-8000 series". Describes how to create a ladder program.	None (Only PDF)
KV-7000 Series User's Manual	System configuration and specifications of the "KV-7000 series". Describes how to create a ladder program.	None (Only PDF)
KV-8000/7000/5000/3000/1000 Series KV Nano Series Instruction Reference Manual	Explains each instruction that is available in ladder programming.	None (Only PDF)
KV-8000 Series ST Language Programming Manual	Describes how to create an ST Language program as well as the operators, control statements, and functions that can be used.	None (Only PDF)
KV-8000/7000/5000/3000/1000 Series KV Nano Series Script Programming Manual	Explains the script program creation method and possible operators, control statements, and functions.	None (Only PDF)
KV STUDIO User's Manual	Explains the operation method for "KV STUDIO".	None (Only PDF)

Software License and Copyright

The sample project used in the RDB connection function of this product also includes software components targeted for open source.

Refer to the "RDBC_copyright" function included in the sample project for open source target software components.

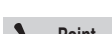
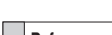
Registered Trademarks

- Oracle, Oracle Database, and MySQL are trademarks or registered trademarks of Oracle Corporation, its subsidiaries, and affiliates in the United States and other countries.
- Microsoft, Windows, SQL Server, and other Microsoft product names are trademarks or registered trademarks of Microsoft Corporation in the United States and/or other countries.
- Excel is a trademark or registered trademark of Microsoft Corporation in the United States and/or other countries.
- PostgreSQL is a trademark or registered trademark of PostgreSQL in the United States and/or other countries.
- MariaDB is a trademark or registered trademark of MariaDB Corporation Ab, its subsidiaries and affiliates in the United States and other countries.

Safety Precautions

Symbols

The following symbols are used to help you identify important elements at one glance. Please be sure to read it.

	Indicates a hazardous situation that, if not avoided, will result in death or serious injury.
	Indicates a hazardous situation that, if not avoided, could result in death or serious injury.
	Indicates a hazardous situation that, if not avoided, could result in minor or moderate injury.
	Indicates a situation that, if not avoided, could result in product damage as well as property damage.
	Indicates cautions and limitations that must be followed during operation.
	Indicates caution for error-prone operations.
	Indicates tips for better understanding or useful information.

General Precautions

	- Do not use the product with the purpose of protecting humans or parts of the human body. - This device is not intended for use as an explosion-proof product. Do not use this device in a hazardous location or in a location that has a potentially explosive atmosphere. - Do not use this product in an application that may cause death, serious injury or serious property damage due to a failure with this product should occur, such as nuclear power plants, on aircraft, trains, ships, or vehicles, used within medical equipment, playground equipment, roller coasters and other rides, etc.
	- Provide a safety circuit that bypasses the PLC to enable failsafe operation of the entire system in the event that the PLC fails. - Malfunction of the output circuit or internal circuit may prevent proper control operation. Be sure to provide a safety circuit in the control systems where circuit malfunctions may lead to fire or other serious accidents. - Prior to installing this product, perform sufficient risk assessment of the entire device to ensure that appropriate risk reduction has been made. Provide appropriate protective fail-safe measures on the machine independent from this product in case a failure with this product should occur.
	- Before using this device, verify its functionality and performance at the startup and/or during operation. - The use of our products other than those specified in this manual may compromise the protection provided by the products.
	- Operation and performance cannot be guaranteed for products that are used in a manner other than those specified in the specification, or those that have been modified. - Utilizing our products in combination with other devices only after careful consideration, since the product may fail to satisfy its functionality and performance capabilities as a result of the conditions and environment in which it is used.

CE Marking/UL Standards

For CE marking limitations and UL compliance restrictions, see the operation manual for KV-XCM02.

How to Use This Manual

This section describes the terms/symbols used in the manual.

Terminology

The following terminologies are used with the exception of some of the terminologies described in this manual.

Terminology	Description
RDB	Abbreviation for relational database. A database for managing multiple associated tables in one place.
SQL	The language used to process data in RDB. Data can be exchanged with RDB by sending the syntax (SQL statement) written in SQL to RDB.
SQL statement	A command written in SQL. In the KV-XCM02, a dedicated function block automatically generates and interprets SQL statements.
INSERT	The SQL statement to use when inserting data into the database.
SELECT	The SQL statement used to search and retrieve data from the database.
UPDATE	The SQL statement used to update/modify the data in the database.
DELETE	A SQL statement used to search and retrieve data from the database.
TRUNCATE	The SQL statement used to delete all data in the database.
WHERE clause	A part of the SQL statement to narrow down the criteria when searching, updating, or deleting data.
ORDER BY clause	A part of the SQL statement that specifies the order in which data is processed when searched, updated, or deleted.
LIMIT clause	A part of the SQL statement that specifies the number of differences in a data to process when searching, updating, or deleting data.
OFFSET clause	A part of the SQL statement that specifies how many of the first pieces of data are skipped when searching, updating, or deleting data.

About Symbols

Menus and buttons in this manual are indicated using the following symbols.

Symbol	Description
" "	Menu item. Can be selected from the menu bar.
[]	Names of windows and items.
{ }	Buttons with text descriptions in windows for various operations, such as executing and canceling.
Ctrl	Keys on the keyboard.

Contents

Preface	2
Related Manuals	2
Software License and Copyright	2
Registered Trademarks	2
Safety Precautions	3
Symbols	3
General Precautions	3
CE Marking/UL Standards	3
How to Use This Manual	4
Terminology	4
About Symbols	4
Contents	5

Chapter1 Overview of the Unit

RDB	1-1
1-1 Features	1-2
1-2 Terminology Description	1-3
1-3 System Configuration	1-4
Flow of Connecting to the RDB	1-5
1-4 Specifications	1-6
Supported RDB	1-6
Corresponded Function	1-6

Chapter2 DB Mapping Configuration

What is the DB Mapping Configuration?	2-1
2-1 SQL File for Table Creation	2-2
How to Output SQL File for Table Creation	2-2
2-2 DB Mapfile	2-3
DB Mapfile Specifications	2-3
Creating a DB Mapfile	2-4
Reading SQL Files for Creating Multiple Tables	2-6
2-3 PLC Data Type	2-7
PostgreSQL RDB Data Types	2-8
Microsoft SQL Server RDB Data Types	2-9
MySQL RDB Data Types	2-10
MariaDB RDB Data Types	2-11
Oracle Database RDB Data Types	2-12

Chapter3 RDB Connection Settings

3-1 Program Image	3-1
3-2 Program Structure	3-1
3-3 Importing Programs	3-2
3-4 Folder Structure	3-3
3-5 Login Settings	3-4
Account Settings	3-4
Root Folder Path Setting	3-5

Chapter4 RDB Connection Function

4-1 Flow of Operation	4-1
4-2 Various Function	4-2
Login Function	4-2
Automatic Login Function	4-2
Character Code Conversion Function	4-2
SQL Execution Function	4-2
SQL Execution Interruption Function	4-2
SQL Timeout Function	4-2
System Log Function	4-3
Stored Procedure Execution Function	4-3
Buffer Resend Function	4-5
Multiple Connections	4-6

Chapter5 Function Block

5-1 Overview of Function Blocks Used in the RDB Connection Function	5-1
5-2 Management Function Block	5-2
RDBC_manage : RDBC Manage	5-2
Argument Description	5-3
Time Chart	5-5
5-3 Mapping Setting Read Function Block	5-7
RDBC_cfgRelation : Read DB Mapfile	5-7
Argument Description	5-7
Time Chart	5-8
5-4 SQL Execution Function Block	5-9
RDBC_select : SQL Execution: SELECT	5-9
Argument Description	5-10
Time Chart	5-11
RDBC_insert : SQL Execution: INSERT	5-12
Argument Description	5-12
Time Chart	5-13
RDBC_update : SQL Execution: UPDATE	5-14
Argument Description	5-14
Time Chart	5-15
RDBC_delete : SQL Execution: DELETE	5-16
Argument Description	5-16
Time Chart	5-16
RDBC_truncate : SQL Execution: TRUNCATE	5-17
Argument Description	5-17
Time Chart	5-17
RDBC_storedProcedure : SQL Execution:	
Stored Procedure	5-18
Argument Description	5-18
Operation in the Event of an Error	5-19
Time Chart	5-20
5-5 Auxiliary Function Block	5-21
RDBC_getErrMsg : Retrieve Error Message	5-21
Argument Description	5-21
RDBC_getSQLSTATE : Retrieve SQLSTATE	5-22
Argument Description	5-22
RDBC_getPlcTime : Retrieve Timestamp	5-23
Argument Description	5-23

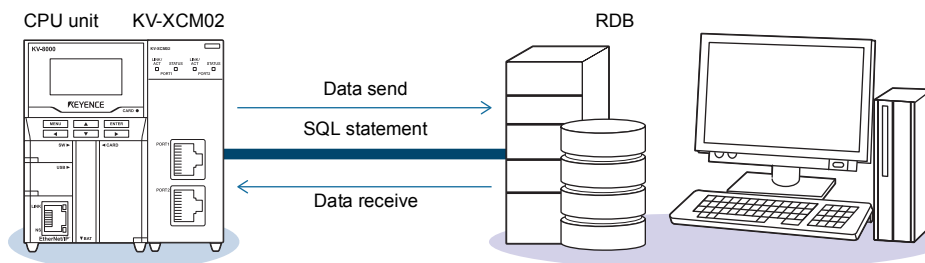
Appendix

1	Error Code	A-1
	Error Types	A-1
	System Error Code.....	A-2
	Setting File Error Code.....	A-2
	Communication Error Code.....	A-3
	SQL Execution Error Code.....	A-4
2	Restriction	A-5
	Common Restrictions of the RDB Connection	
	Function.....	A-5
	Restrictions when using PostgreSQL.....	A-5
	Restrictions when using Microsoft SQL Server.....	A-5
	Restrictions when using MySQL and MariaDB	A-5
	Restrictions when using Oracle Database	A-5
3	Quotation/String Identifier for Each RDB	A-6
4	SQL Command Description	A-7
	INSERT	A-7
	SELECT	A-7
	UPDATE.....	A-7
	DELETE	A-7
	TRUNCATE.....	A-7
	WHERE Clause.....	A-7
	ORDER BY Clause	A-8
	LIMIT Clause.....	A-8
	OFFSET Clause.....	A-9
5	List of Devices Used	A-10
	List of Devices Used	A-10
	Batch Replacement Devices	A-14
6	Troubleshooting.....	A-15
	Relay Does Not Turn ON During UPG Execution	A-15
	Relay Does Not Turn ON During Online	A-15

Chapter1 Overview of the Unit

With "KV-XCM02", data can be exchanged with a relational database (RDB). Using a dedicated function block (FB) on the ladder program, record insert, update, search, and delete can be executed against the RDB.

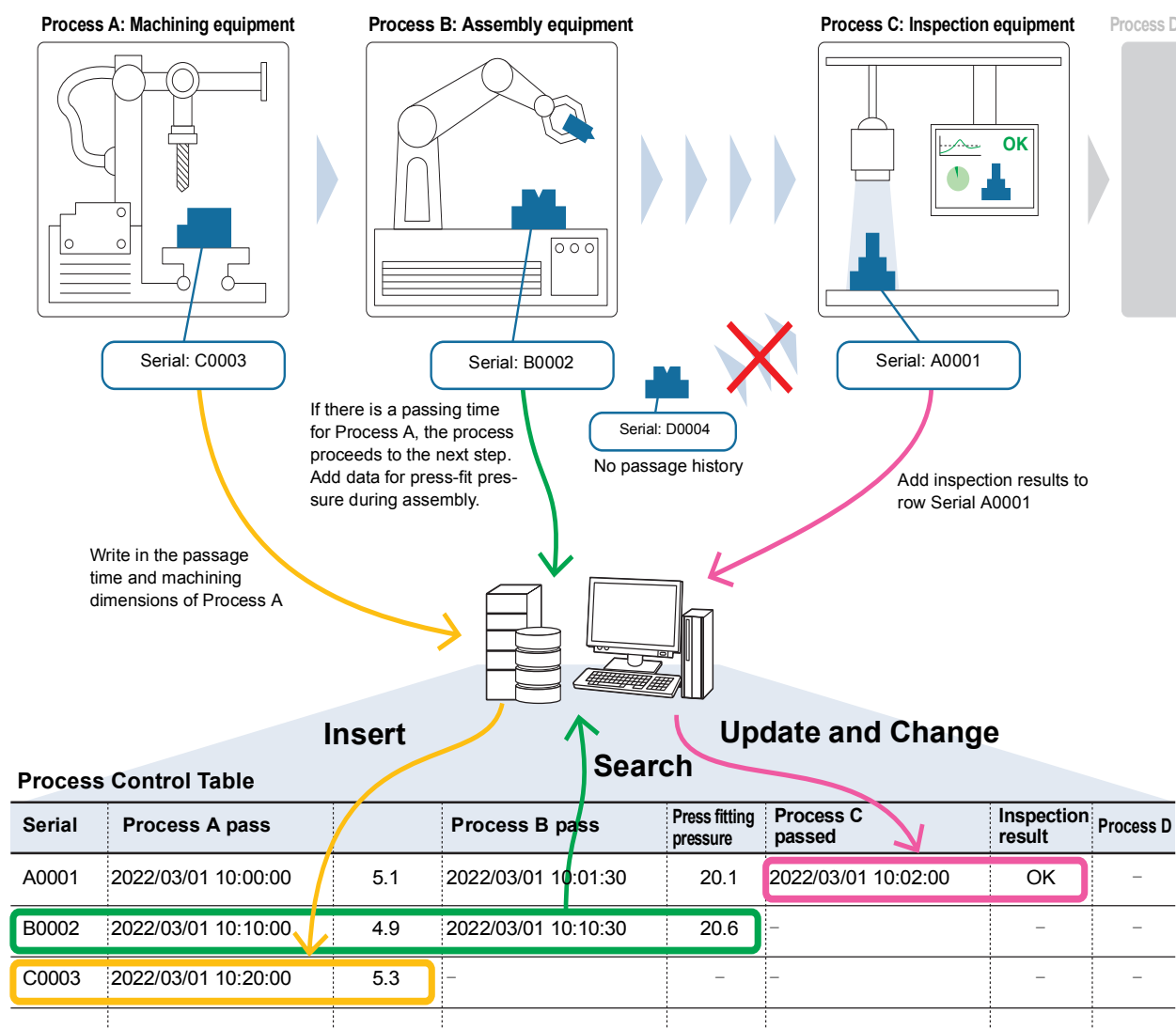
<KV-XCM02> issues SQL statements to process data via TCP/IP communication with the RDB. Since a dedicated FB is provided for issuing SQL statements, the ladder program only needs to control the storage and relay of the data to be processed to exchange data with the RDB.



RDB

A relational database (RDB) is a collection of multiple tables that are associated with each other. Each table consists of columns and rows (records). RDB has excellent searchability, even when large amounts of data are stored, as data in multiple tables can be easily modified. It is used in a variety of systems, from large-scale to small-scale systems.

The following is an example of recording in the database of manufacturing conditions from each piece of equipment tied to serial numbers and confirming the passage of the previous process when a workpiece is submitted. Since the process management table built in the RDB can be directly accessed from each process, it can easily record manufacturing conditions tied to serial numbers and manage process passage.



1-1 Features

This section describes the features of the RDB connection function on <KV-XCM02>.

- **No need for dedicated programs or applications**

The PLC performs TCP/IP communication with the RDB, so installing a separate dedicated program or application on the server is not needed.

- **Access database only by ladder program**

Programming to access the database is only done by the ladder program. Function block (FB) is provided for database connection, data processing, and data retrieval, making it easy to program.

Sample project is also provided and executed without specialized knowledge of SQL.

- **Programs can be customized and capitalized.**

Since basic SQL is provided as FB, data processing sequences can be built and capitalized in the same way as a regular ladder program.

- **Up to 4 different RDBs can be connected simultaneously**

A single "KV-XCM02" can connect up to 4 RDBs simultaneously. Even if one RDB stops functioning due to failure, the system can register data in another RDB.

- **Supports up to 99 different table structures**

A single "KV-XCM02" can support up to 99 different table structures. The system can be constructed to access multiple tables independently, such as a traceability table, equipment monitoring table, and variety information table.

- **Data can be guaranteed even in the event of communication failure**

The contents of SQL statements when communication with the RDB is interrupted are saved once, and when communication is restored, the saved SQL statements are automatically issued. Data can be exchanged without loss in the event of communication errors or RDB server downtime.

1-2 Terminology Description

This section describes the terms related to the RDB connection function.

● RDB

Abbreviation for relational database. A database that manages multiple data tables (tabular data) in relation to one another.

● Schema

A hierarchy for managing tables. A schema consists of one or more tables.

● Table

A tabular representation of multiple data in columns and rows (records).

● Primary key

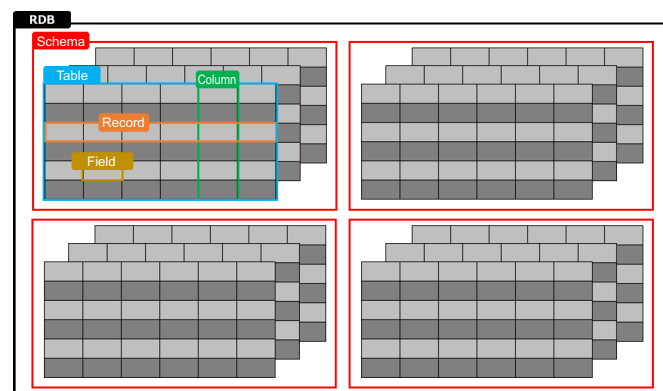
In RDB, this is the column where no data duplication is accepted. Primary keys serve to identify data uniquely.

For example, there can be duplicate names and departments in the following table, but no duplicate employee numbers. Thus, the columns that are not allowed to duplicate but identify data uniquely are set as primary keys.

Employee No.	Name	Department
12345	Taro Tanaka	Human Resources Department
23456	Hanako Yamada	Sales Department
34567	Jiro Suzuki	Development Department
45678	Saburo Sato	Development Department

● Field

The smallest unit of data that makes up a table.



● SQL

The language used to process data in RDB. Data can be exchanged with RDB by sending the syntax (SQL statement) written in SQL to RDB. With KV-XCM02, the dedicated FB automatically issues SQL statements, so there is no need to create SQL statements.

● INSERT

SQL for inserting or adding records to a table.

● UPDATE

SQL to update or change data existing in a table.

● SELECT

SQL to retrieve data in a table.

● DELETE

SQL to delete data existing in a table.

● TRUNCATE

SQL to delete all data in a table.

● Stored Procedure

A procedure is a process that combines several SQL statements into one.

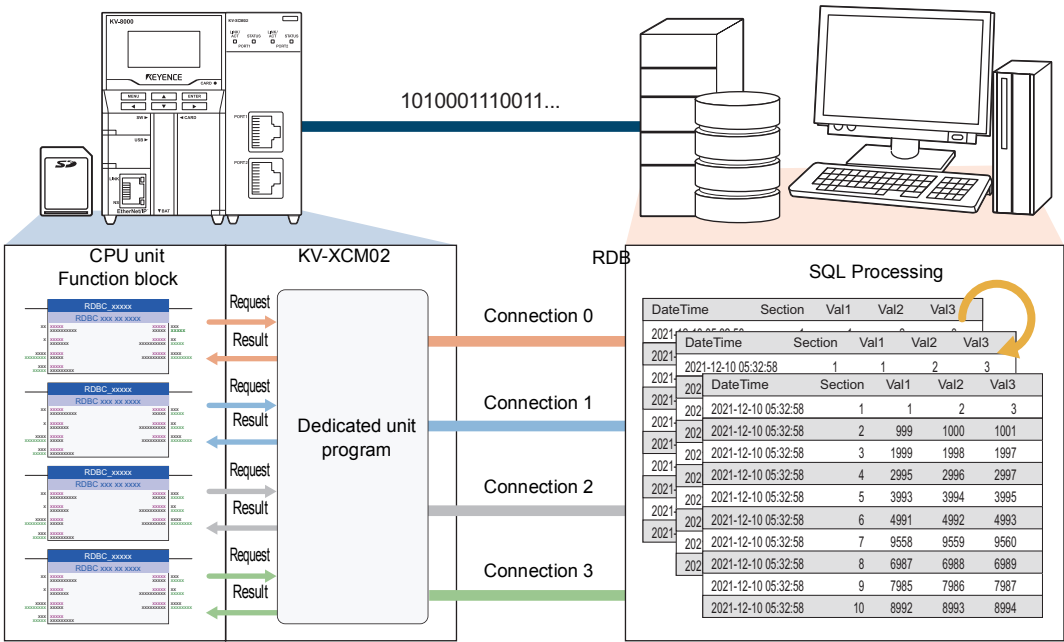
A similar term is a stored function. The only difference between a stored procedure and a stored function is the presence or absence of a return value at runtime.

- Stored procedure: Return value exists at runtime
- Stored function: No return value exist at runtime

In this manual, stored procedures and stored functions are collectively referred to as stored procedures.

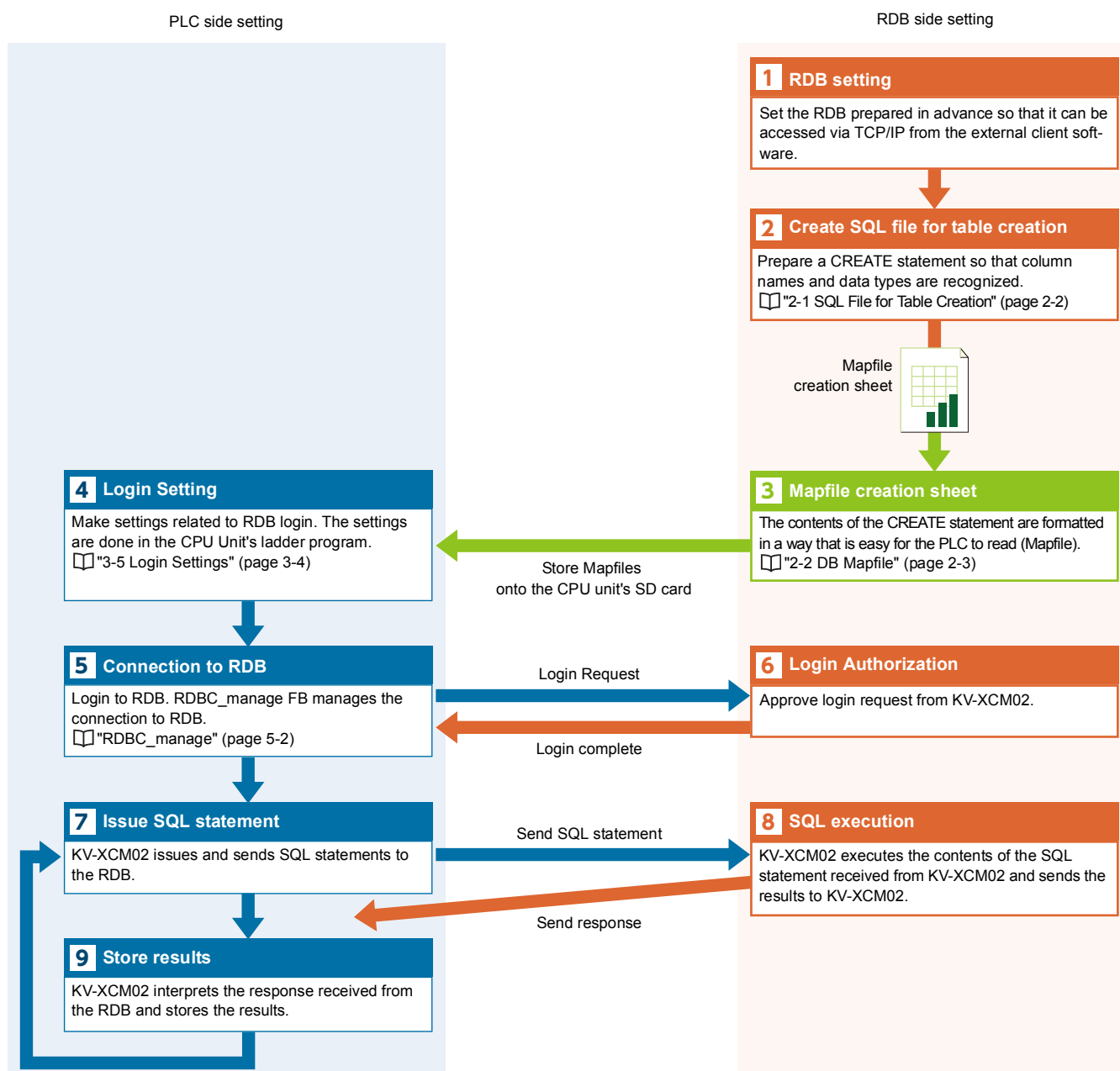
1-3 System Configuration

The KV-XCM02's RDB connection function allows up to 4 connections to the RDB by using connections 0 to 3.



- **Function block (FB)**
A program that runs internally in the CPU unit. It passes the specified data to or receives data from the KV-XCM02 unit program.
- **Unit program**
A program that runs internally in the "KV-XCM02. Creates and issues SQL statements based on data received from the CPU unit's FB. It also interprets SQL statements received from the RDB and delivers the data to the CPU unit.
- **DB Mapfile**
This file describes the relationship between the data types of the columns set in the RDB and the data in the PLC. Data is stored in the CPU according to the contents of the DB Mapfile. The DB Mapfile can be created on the Mapfile creation sheet.
□ "Creating a DB Mapfile" (page 2-4)

Flow of Connecting to the RDB



1-4 Specifications

Supported RDB

RDB	Support Version
PostgreSQL ^{*1}	7.4 or later
Microsoft SQL Server	2008 R2, 2012, 2014, 2016, 2017, 2019
MySQL ^{*2}	5.5, 5.6, 5.7, 8.0
MariaDB	5.5.1, 10.3.14
Oracle Database ^{*3}	11g ^{*4} , 12c, 18c, 19c

*1 Password authentication procedure supports plain text passwords and MD5 encrypted passwords. As the default password authentication procedure is scram-sha-256 for PostgreSQL 13 or later, please change it to plain text password or MD5 encrypted password.

The character set (character code) on the client and server side should be set to UTF-8.

The parameter "datestyle" should not be altered from the default value "ISO".

*2 The password authentication procedure supports Standard.

The character set (character code) on the client and server side should be set to UTF-8.

*3 When using "oracle_fdw", it is necessary to set the PostgreSQL and Oracle table associations on the client PC.

If the REST connection is used by HTTP, please set the following on the RDB side.

•Enable Oracle REST Data Service (ORDS) (released 17.3 or later)

•Disable HTTPS connections (set to use OAuth2 in a non-HTTPS environment)

•Activate REST-enabled SQL services

REST connections via HTTP are supported only in Ver. 12c or later. Also, the HTTP authentication procedure supports Basic authentication.

*4 Ver. 11g can be connected only when "oracle_fdw" is used.

Corresponded Function

Item	Specifications
Supported CPU Unit	KV-8000(A) ^{*5} KV-7500/7300 (function version 2.3 or later)
Supported expansion unit	KV-XCM02 (function version 1.1 or later)
Supported KV STUDIO version	Ver.11.5 or later
Transport protocol	TCP/IP ^{*1}
Maximum number of connections	4
Function block name	RDBC_manage (status management) RDBC_cfgRelation (read DB Mapfile) RDBC_select (record search execution) RDBC_insert (record insert execution) RDBC_update (record update execution) RDBC_delete (record delete execution) RDBC_truncate (deletion of all records) RDBC_storedProcedure (stored procedure execution) RDBC_getErrMsg (get error messages) RDBC_getSQLSTATE (get SQLSTATE) RDBC_getPlcTime (get timestamp)
Supported SQL	SELECT ^{*2} , INSERT ^{*3} , UPDATE, DELETE, TRUNCATE, Stored procedure/Stored function execute
Additional clauses that can be specified	WHERE, ORDER BY, LIMIT, OFFSET
Maximum supported table structures	99 (total columns less than 20,000)
Maximum supported columns	1024 (Microsoft SQL Server) 1000 (PostgreSQL, MySQL, MariaDB, Oracle Database)
Maximum record storage size	Up to 200,000 bytes ^{*4}
Supported data types	Numeric type, date/time type, text string type
Stored procedure execution function	Arguments: Total of 100 input, input/output, and output arguments Return value: Outputs as a text string type of 199Byte or less Result set: Depends on the RDB type Supported RDB: PostgreSQL, SQL Server, MySQL, MariaDB
Buffer resend function	Supported SQL statement: INSERT (multi-row INSERT also possible), UPDATE Maximum buffer capacity: 1 GB per connection Buffer storage location: SD card in the CPU unit
Other functions	System log creation

*1 When using Oracle Database, HTTP is used as the application protocol.

*2 To eliminate duplicate records, "SELECT DISTINCT" is also supported.

*3 Multi-row INSERT is also supported. Only KV-8000 is supported.

*4 Maximum size of data storage area to be reserved by the CPU unit.

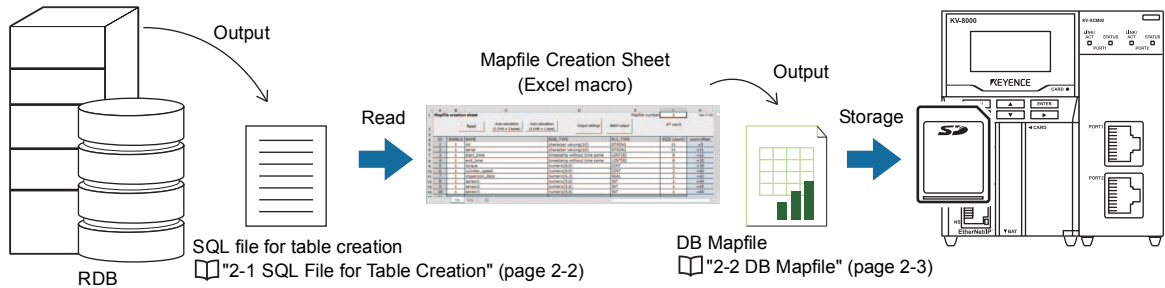
*5 Function version 2.0 or later is required when using the variable version of the sample project.

Chapter2 DB Mapping Configuration

This section provides an overview of the DB mapping configuration detail.

What is the DB Mapping Configuration?

DB mapping configuration is a setting that matches the data type of a table column with the data area of the PLC. SQL (CREATE statement) files for table creation are output from RDB, and the Mapfile creation sheet is used to convert the RDB data structure so that it can be interpreted by the PLC. By saving the converted DB Mapfile to the CPU unit's SD card, the PLC can issue and interpret SQL statements in accordance with the RDB data structure.



For example, suppose the following table structure is defined in RDB

Column name	Serial code	Product type code	Temperature	Date/time
RDB data type	character(8)	integer	numerical	TimeStamp

DB mapping configuration is a setting to match the table structure and data area so that the data type on the RDB side can be used on the PLC side (the following is a case where "character" is a double-byte character).

Column name	Serial code	Product type code	Temperature	Date time
RDB data type	character(8)	integer	numerical	TimeStamp
PLC data type	STRING[8]	DINT	REAL	UINT[8]
PLC device	[N] *1 to [N] + 8	[N] + 9 to [N] + 10	[N] + 11 to [N] + 12	[N] + 13 to [N] + 20

*1 [N]: Leading device to store data

2-1 SQL File for Table Creation

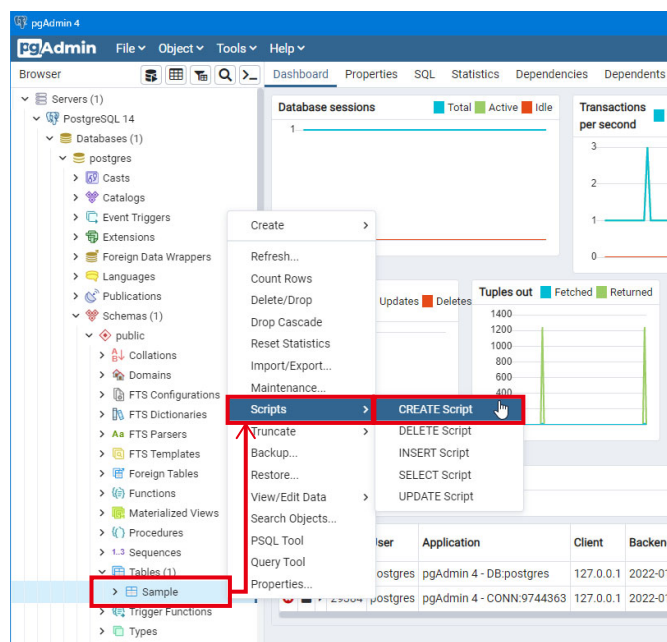
A table creation SQL (CREATE statement) file contains the output of SQL statements (CREATE statements) used to create RDB tables. The column names and data types in the CREATE statement for the created table are used in the Mapfile creation sheet.

How to Output SQL File for Table Creation

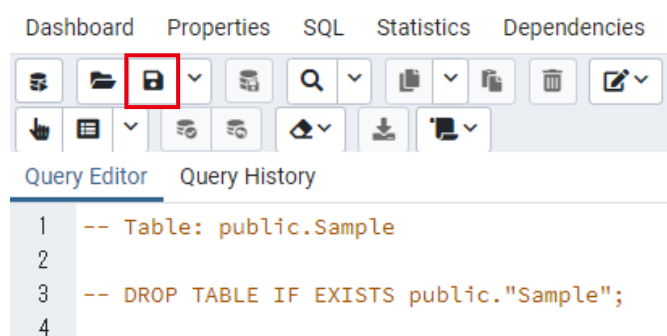
This section explains how to output SQL files for table creation from RDB, using PostgreSQL as an example.

- **CREATE statements output from RDB other than PostgreSQL can also be used in the Mapfile creation sheet. For details on how to output CREATE statements, please refer to the manual of each RDB.**
- **Append any extension to the SQL file for table creation. The extension is required for use in the Mapfile creation sheet.**

- 1 Right-click on the table to be output in the object browser and select [Scripts] - [CREATE Scripts] from the menu.



- 2 Select the "Save File" icon on the query editor.

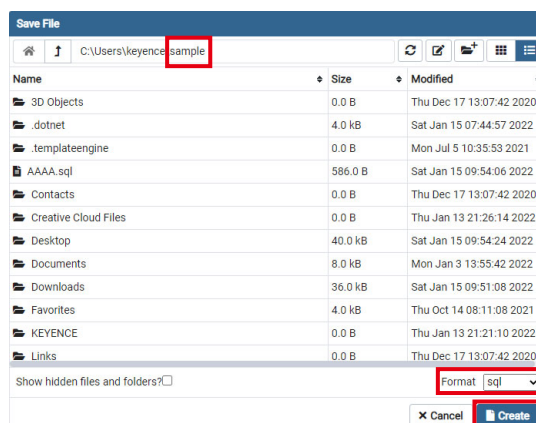


A save dialog box will appear.

- 3 Specify the path of the folder to save the file.



- 4 Enter the folder path followed by the name of the file, select "sql" as "format", and then click "Create".

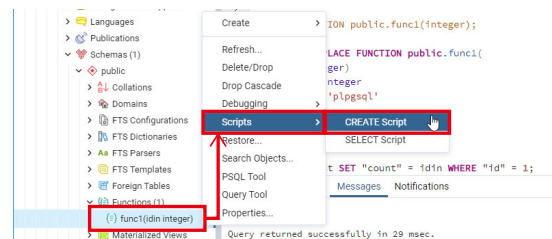


- Reference**
- Entering a file name will enable you to select "Create".
 - Selecting "sql" as the "format" will add an extension to the SQL file for the table creation so that it can be used in the Mapfile creation sheet.

- 5 The SQL file will be imported and used in the mapping sheet.

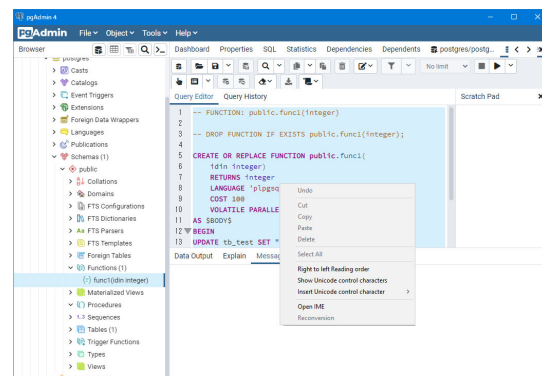
Reference When using stored procedures, in addition to the above SQL file for table creation, it is necessary to output a stored definition table file in the following procedure.

1. From the object browser, right-click on the stored procedure to be output and select [Scripts] - [CREATE scripts].



2. Copy all SQL statements on the activated query editor, paste them into a text file, then save.

- * Be sure to select "UTF-8" as the character encoding when saving.



2-2 DB Mapfile

DB Mapfiles are SQL files for creating DB tables output from the RDB side, converted using the Mapfile creation sheet.

By reading the DB Mapfile, the data structure of the RDB side can be mapped to the data area in the PLC, and the SQL statements are issued and interpreted on the PLC side.

ID	ENABLE	NAME	RDB_TYPE	PLC_TYPE	SIZE (word)	word offset
1	1	lot	character varying(10)	STRING	11	+0
2	1	serial	character varying(10)	STRING	11	+11
3	1	start_time	timestamp without time zone	UINT[8]	8	+22
4	1	end_time	timestamp without time zone	UINT[8]	8	+30
5	1	torque	numeric(8,0)	DINT	2	+38
6	1	cylinder_speed	numeric(8,0)	DINT	2	+40
7	1	inspection_data	numeric(6,3)	REAL	2	+42
8	1	sensor1	numeric(5,0)	INT	1	+44
9	1	sensor2	numeric(5,0)	INT	1	+45
10	1	sensor3	numeric(5,0)	INT	1	+46

The RDB connection function can read up to 99 DB Mapfiles and switch between them by specifying the file number to issue SQL statements with different settings.

By loading the DB Mapfile in advance, it can be used as follows

- By specifying the DB Mapfile number and executing SELECT, the columns to be retrieved can be changed.
- By specifying the DB Mapfile number and executing INSERT/UPDATE, the column to store the data can be changed.

DB Mapfile Specifications

The specifications of the DB Mapfile are as follows

Item	Description
File name	columnRelationCfgXX.txt (XX is a two-digit number from 01 to 99)
Delimiter	TAB
Element	Element 1: Registered column name Element 2: PLC data type Element 3: Occupied size (words) *1 to 1000 Element 4: RDB data type Element 5: Word offset
Character code	UTF-16
New-line code	CR+LF
Registered column name	64 characters or less
Special note	A line break is also required on the last line



Point

- Registered column names are case-sensitive.
- The registered column name must match both the RDB table definition and stored procedure definition.
- When using Oracle Database, only single-byte alphanumeric characters, underscore (_), dollar sign (\$), and sharp (#) capital letters can be used to register column names.
- In Oracle Database 12cR1 or earlier, the registered column name must be 29 characters or less.

Format

(Line 1) Registered column name1<tab>PLC data type<tab>Occupied size<tab>RDB data type<tab>Word offset<CR+LF>
(2nd line) Registration column name2<tab>PLC data type<tab>Occupied size<tab>RDB data type<tab>Word offset<CR+LF>

: :

Example:

```
inspDateTime<tab>UINT[8]<tab>8<tab>datetime2(7)<tab>0<CR+LF>
productSerial<tab>STRING<tab>12<tab>varchar(11)<tab>8<CR+LF>
inspResultVal<tab>INT<tab>2<tab>smallint<tab>20<CR+LF>
inspResult<tab>STRING<tab>3<tab>varchar(2)<tab>22<CR+LF>
```

*1 In the example format, <tab> indicates a horizontal tab. <CR+LF> indicates a line break.

Creating a DB Mapfile

DB Mapfiles are created by loading SQL files for table creation onto a Mapfile creation sheet.



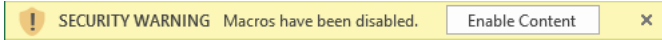
Point

- The Mapfile creation sheet is an Excel file with macro extension (.xlsx). It can be obtained from the Keyence website.
- The Mapfile creation sheet works with Microsoft Excel 2010 or later.

1 Launch the Mapfile creation sheet.

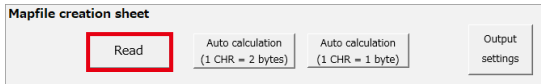


Click "Enable Content" to activate the VBA macros if a security warning appears.



2 Click "Read" to read the SQL file to create the table.

□ "2-1 SQL File for Table Creation" (page 2-2)



The information from the SQL file will be expanded onto the Mapfile creation sheet.

Mapfile creation sheet						Mapfile number	1	Ver.7.00
Read Auto calculation (1 CHR = 2 bytes) Auto calculation (1 CHR = 1 byte) Output settings Batch output						0 word		
ID	ENABLE	NAME	RDB_TYPE	PLC_TYPE	SIZE (word)	word offset		
1	1	lot	character varying(10)			+0		
2	1	serial	character varying(10)			+0		
3	1	start_time	timestamp without time zone			+0		
4	1	end_time	timestamp without time zone			+0		
5	1	torque	numeric(8,0)			+0		
6	1	cylinder_speed	numeric(8,0)			+0		
7	1	inspection_data	numeric(6,3)			+0		
8	1	sensor1	numeric(5,0)			+0		
9	1	sensor2	numeric(5,0)			+0		
10	1	sensor3	numeric(5,0)			+0		

3 Click "Auto Calculate". The PLC data type and size will be automatically calculated.



If working with data that contains double-byte characters (1 character = 2 bytes), click "Auto Calculate". To handle data containing only single-byte characters, click "Auto Calculate" (1 character = 1 byte) to reduce the number of words occupied.

ID	ENABLE	NAME	RDB_TYPE	PLC_TYPE	SIZE (word)	word offset
1	1	lot	character varying(10)	STRING	11	+0
2	1	serial	character varying(10)	STRING	11	+11
3	1	start_time	timestamp without time zone	UINT[8]	8	+22
4	1	end_time	timestamp without time zone	UINT[8]	8	+30
5	1	torque	numeric(8,0)	DINT	2	+38
6	1	cylinder_speed	numeric(8,0)	DINT	2	+40
7	1	inspection_data	numeric(6,3)	REAL	2	+42
8	1	sensor1	numeric(5,0)	INT	1	+44
9	1	sensor2	numeric(5,0)	INT	1	+45
10	1	sensor3	numeric(5,0)	INT	1	+46

No.	Name	Description
1	ENABLE	Specifies whether to output the target rows as a DB Mapfile. 0: The target row is grayed out, and the row is not output to the DB Mapfile. 1: Target rows are output to the DB Mapfile. Immediately after read, all rows are set to 1. For the same table, if the columns to be processed are different, a Mapfile is created for each. For example, to retrieve data for all columns using a SELECT statement, create a Mapfile with all rows set to 1. To search only specific columns of data or update only specific columns of data, create a Mapfile with 1 specified only for the target rows.
2	NAME	Column name.
3	RDB_TYPE	Data type of the column.
4	PLC_TYPE	Data type of PLC. Click on "Auto Calculate" to display. The Mapfile creation sheet will automatically specify the PLC data type from the RDB data type. □ "2-3 PLC Data Type" (page 2-7)
5	SIZE	Data size. Click on "Auto Calculate" to display. When the PLC_TYPE is STRING, it can be changed to any SIZE (word).
6	word offset	Offset from the beginning. Click on "Auto Calculate" to display. When storing data in a ladder, store data in order from the top according to the offset.

4 Specify the file number and click "Output Settings".

A save dialog box will appear to output the DB Mapfile to a folder.

Auto calculation (1 CHR = 2 bytes)		Auto calculation (1 CHR = 1 byte)		Output settings	Batch output	Mapfile number 1	0 word
---------------------------------------	--	--------------------------------------	--	--------------------	-----------------	---	--------



Point

The file number must be within 2 digits.

If specify a three-digit number or more, the last two digits will be used as the file number.

5 Stores the output DB Mapfile in the specified folder on the SD card.

root directory > RDBC_SDK > cfg

* When the root folder for RDB connection function is "RDBC_SDK"

"3-4 Folder Structure" (page 3-3)



Point

Before using the RDB connection function, please store the DB Mapfile in the above cfg folder.



Reference

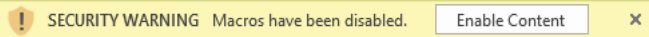
The Mapfile creation sheet also reads the output DB Mapfile later. To change or modify the DB Mapfile once output, please read the DB Mapfile and perform the change or modification.

Reading SQL Files for Creating Multiple Tables

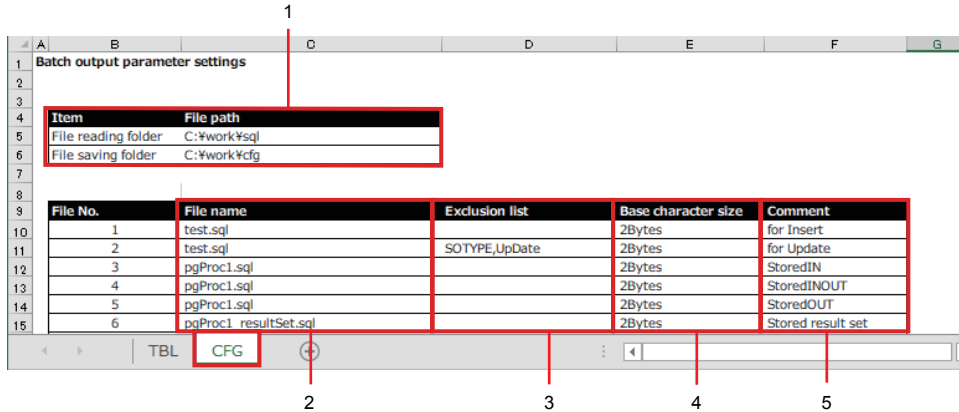
The Mapfile creation sheet allows reading SQL files for creating multiple tables at once and automatically creates DB Mapfiles. This section explains the process.

1 Launch the Mapfile creation sheet.

 Click "Enable Content" to activate the VBA macros if a security warning appears.



2 Open the "CFG" sheet and set the file path, file name, etc.



No.	Description
1	Settings for reading and outputting files. File reading folder: Specify the folder that contains the SQL files for table creation. File saving folder: Specify the target folder where the DB Mapfiles are output.
2	Specify the file name of the SQL file for table creation to be read.
3	Specify column names to exclude from SELECT/INSERT/UPDATE/STORED PROCEDURE arguments.
4	Select the basic character size for string type columns from 2 bytes/1 byte. For ASCII one-byte characters, select 1 byte to reduce the number of words occupied.
5	Enter a comment. If there are multiple files, entering the purpose of use will make it easier to manage them since the Mapfile creation sheet can be opened to check them later.

3 Open the "TBL" sheet, enter the file number and click "Batch Output".

Mapfile creation sheet						
Read Auto calculation (1 CHR = 2 bytes) Auto calculation (1 CHR = 1 byte) Output settings Batch output				Mapfile number	1	Ver.7.00
				47 word		
ID	ENABLE	NAME	RDB_TYPE	PLC_TYPE	SIZE (word)	word offset
1	1	lot	character varying(10)	STRING	11	+0
2	1	serial	character varying(10)	STRING	11	+11
3	1	start_time	timestamp without time zone	UINT[8]	8	+22
4	1	end_time	timestamp without time zone	UINT[8]	8	+30
5	1	torque	numeric(8,0)	DINT	2	+38
6	1	cylinder_speed	numeric(8,0)	DINT	2	+40
7	1	inspection_data	numeric(6,3)	REAL	2	+42
8	1	sensor1	numeric(5,0)	INT	1	+44
9	1	sensor2	numeric(5,0)	INT	1	+45
10	1	sensor3	numeric(5,0)	INT	1	+46

 The file will be numbered consecutively, with the number specified in the "set file number" field at the top.

2-3 PLC Data Type

Among the data types that can be used in the RDB, those that the CPU unit cannot support are processed as separate data types. This section describes the data types that are replaced in the Mapfile creation sheet for the RDB data structure file.

The PLC data types supported by the RDB connection function are as follows.

Type of data		PLC data type
Value	Bits	BOOL[16] ^{*1}
	16-bit signed integer	INT
	16-bit unsigned integer	UINT
	32-bit signed integer	DINT
	32-bit unsigned integer	UDINT
	64-bit signed integer	LINT ^{*4}
	64-bit unsigned integer	ULINT ^{*4}
	Single precision floating-point real number	REAL
	Double-precision floating-point real number	LREAL
Date/time ^{*2}		UINT[8]
Character string ^{*3}		STRING

*1 Only the least significant bit is turned ON/OFF. Upper 15 bits are reserved area.

*2 The date/time type is a word array (8 elements).
Assuming that the leading data is DM100
December 31, 2018 23:58:59.999 is stored as follows.
DM100: 2018
DM101: 12
DM102: 31
DM103: 23
DM104: 58
DM105: 59
DM106: 999
DM107: 0 (Reserved area. Please store 0)

Date and time type data can be created with the "RDBC_getPlcTime" FB.

□ "RDBC_getPlcTime" (page 5-23)

*3 If a Mapfile creation sheet is used, the string type will have the following size

- When the basic character size is 2 bytes
1 character = 2 bytes, plus 2 bytes of the terminating NULL
Example) STRING[20] → 2 bytes x 20 + 2 = 42 bytes (21 words)

- When the basic character size is 1 bytes
1 character = 1 byte, plus 1 byte of the terminating NULL
Example) STRING[20] → 1 bytes x 20 + 1 = 21 bytes (11 words)

*4 KV-8000/7000 series CPU units do not have LINT or ULINT type. The CPU unit expands the data into a 1-word word array, passes to "KV-XCM02", and then processes it as the above data type in "KV-XCM02".

For the RDB data types that can be used in "KV-XCM02," please refer to the PLC data types corresponding to each RDB data type on the next page.

□ "PostgreSQL RDB Data Types" (page 2-8)

□ "Microsoft SQL Server RDB Data Types" (page 2-9)

□ "MySQL RDB Data Types" (page 2-10)

□ "MariaDB RDB Data Types" (page 2-11)

□ "Oracle Database RDB Data Types" (page 2-12)

-  **Point**
- RDB data types not listed in the table on the next page are not supported.
 - KV-XCM02 does not support NaN and inf in its RDB connection function.

PostgreSQL RDB Data Types

Type	RDB data type	PLC data type
Value	bigint bigserial	LINT (64-bit signed integer) LREAL (double precision floating-point real number)
	integer serial	DINT (32-bit signed integer)
	smallint smallserial	INT (16-bit signed integer)
	boolean	BOOL[16] (16 bits)*5
	decimal (to 5, N) numeric (to 5, N)	INT (16-bit signed integer)*1 REAL (single-precision floating-point real number)*2
	decimal (to 7, N) numeric (to 7, N)	DINT (32-bit signed integer)*1 REAL (single-precision floating-point real number)*2
	decimal (to 10, N) numeric (to 10, N)	DINT (32-bit signed integer)*1 LREAL (double precision floating-point real number)*2
	decimal (to, N) numeric (to, N)	LINT (64-bit signed integer)*1 LREAL (double precision floating-point real number)*2
	double precision	LREAL (double precision floating-point real number)
	real	REAL (single-precision floating-point real number)
Date/time	date/time (without time zone) / time with time zone / timestamp (without time zone) / timestamp with time zone	UINT[8] (date and time)*3
Character string	text / character*6 / character varying / json / jsonb	STRING (Character string)*4

*1 Depending on the settings, the range handled by RDB data types and PLC data types do not necessarily match.

If a number outside the range of the PLC data type is retrieved (SELECT), it will overflow.

When overflowing in the positive direction, the maximum positive value is stored in the CPU unit, and when overflowing in the negative direction, the maximum negative value is stored in the CPU unit.

Inserting (INSERT/UPDATE) a number outside the value range of the RDB data type may result in an error.

*2 If the number of decimal places is specified in decimal or numeric and a floating-point real number is specified for the PLC data type, the number is rounded to the nearest significant digit when converted from the PLC data type to the RDB data type.

If an integer is specified for the PLC data type in decimal or numeric, only the integer portion of the RDB data type is targeted, and values after the decimal point are truncated.

*3 Stores [year/month/day/hour/minute/second/millisecond/reservation (1 word)] in 8 consecutive words. The size is fixed. Time zone information is not included.

*4 NULL is appended to the end of the string. The data size of the STRING should be set to an appropriate value including NULL according to the character code to be stored. A maximum of 2000 bytes (1000 words) can be specified. If the number of bytes exceeds 2000 bytes (1000 words), the excess is truncated, and NULL is stored at the end. If text, json, or jsonb is specified, the data size is fixed and occupies 2000 bytes (1000 words).

*5 Only the least significant bit is turned ON/OFF. Upper 15 bits are reserved area.

*6 When data of the STRING type is INSERT into a column of CHARACTER type, spaces are appended until the number of bytes of CHARACTER type is satisfied.

Example: If a 4-byte string "ABCD" is INSERT into a column of character(8), a space is appended to the last 4 bytes of "ABCD".

If data is retrieved with SELECT in this state, the string with spaces will be retrieved. Therefore, it is recommended to use CHARACTER VARIATION, which does not append spaces.

Microsoft SQL Server RDB Data Types

Type	RDB data type	PLC data type
Value	bigint	LINT (64-bit signed integer) LREAL (double precision floating-point real number)
	int	DINT (32-bit signed integer)
	smallint	INT (16-bit signed integer)
	tinyint	INT (16-bit signed integer) ^{*1}
	bit	BOOL[16] (16 bits) ^{*7}
	decimal (to 5, N) numeric (to 5, N)	INT (16-bit signed integer) ^{*1} REAL (single-precision floating-point real number) ^{*2}
	decimal (6 to 7, N) numeric (6 to 7, N)	DINT (32-bit signed integer) ^{*1} REAL (single-precision floating-point real number) ^{*2}
	decimal (8 to 10, N) numeric (8 to 10, N)	DINT (32-bit signed integer) ^{*1} LREAL (double precision floating-point real number) ^{*2}
	decimal (11 to, N) numeric (11 to, N)	LINT (64-bit signed integer) ^{*1} LREAL (double precision floating-point real number) ^{*2}
	float	LREAL (double precision floating-point real number)
	real	REAL (single-precision floating-point real number)
	money	LREAL (double precision floating-point real number) ^{*3}
	smallmoney	REAL (single-precision floating-point real number) ^{*4}
Date/time	date/datetime/time datetime2 datetimeoffset smalldatetime	UINT[8] (date and time) ^{*5}
Character string	char/varchar/nchar/nvarchar/text/ntext	STRING (Character string) ^{*6}

*1 Depending on the settings, the range handled by RDB data types and PLC data types do not necessarily match.

If a number outside the range of the PLC data type is retrieved (SELECT), it will overflow. When overflowing in the positive direction, the maximum positive value is stored in the CPU unit, and when overflowing in the negative direction, the maximum negative value is stored in the CPU unit.

Inserting (INSERT/UPDATE) a number outside the value range of the RDB data type may result in an error.

*2 If the number of decimal places is specified in decimal or numeric and a floating-point real number is specified for the PLC data type, the number is rounded to the nearest significant digit when converted from the PLC data type to the RDB data type.

If an integer is specified for the PLC data type in decimal or numeric, only the integer portion of the RDB data type is targeted, and values after the decimal point are truncated.

*3 The number of significant digits is 15. The value is rounded to the fifth decimal when writing to RDB in SQL Execution FB.

Example: If 1.23456 is stored in the CPU unit, writes 1.2346.

*4 The number of significant digits is 7. The value is rounded to the fifth decimal when writing to RDB in SQL Execution FB.

Example: If 1.23456 is stored in the CPU unit, writes 1.2346.

*5 Stores [year/month/day/hour/minute/second/millisecond/reservation (1 word)] in 8 consecutive words. The size is fixed. Time zone information is not included.

Dates and times that can be retrieved are up to 35:59:59:999 milliseconds on December 31, 2998.

*6 NULL is appended to the end of the string. The data size of the STRING should be set to an appropriate value including NULL according to the character code to be stored. A maximum of 2000 bytes (1000 words) can be specified. If the number of bytes exceeds 2000 bytes (1000 words), the excess is truncated, and NULL is stored at the end. If text, ntext is specified, the data size is fixed and occupies 2000 bytes (1000 words).

*7 Only the least significant bit is turned ON/OFF. Upper 15 bits are reserved area.

MySQL RDB Data Types

Type	RDB data type	PLC data type
Value	BIGINT	LINT (64-bit signed integer) ULINT (64-bit unsigned integer) LREAL (double precision floating-point real number)
	INT/MEDIUMINT	DINT (32-bit signed integer) UDINT (32-bit unsigned integer)
	SMALLINT/TINYINT/BOOL/BOOLEAN	INT (16-bit signed integer) UINT (16-bit unsigned integer)
	BIT(1)	BOOL[16] (16 bits) ^{*5}
	DECIMAL (to 5, N) ^{*7}	INT (16-bit signed integer) ^{*1} UINT (16-bit unsigned integer) ^{*1} REAL (single-precision floating-point real number) ^{*2}
	DECIMAL (6 to 7, N) ^{*7}	DINT (32-bit signed integer) ^{*1} UDINT (32-bit unsigned integer) ^{*1} REAL (single-precision floating-point real number) ^{*2}
	DECIMAL (8 to 10, N) ^{*7}	DINT (32-bit signed integer) ^{*1} UDINT (32-bit unsigned integer) ^{*1} LREAL (double precision floating-point real number) ^{*2}
	DECIMAL (11 to, N) ^{*7}	LINT (64-bit signed integer) ^{*1} ULINT (64-bit unsigned integer) ^{*1} LREAL (double precision floating-point real number) ^{*2}
	DOUBLE	LREAL (double precision floating-point real number)
	FLOAT	LREAL (double-precision floating-point real number) ^{*6}
Date/time	DATE/DATETIME/TIMESTAMP/TIME	UINT[8] (date and time) ^{*3}
Character string	CHAR/VARCHAR/TINYTEXT/TEXT/ MEDIUMTEXT/LONGTEXT/JSON	STRING (Character string) ^{*4}

*1 Depending on the settings, the range handled by RDB data types and PLC data types do not necessarily match.

If a number outside the range of the PLC data type is retrieved (SELECT), it will overflow. When overflowing in the positive direction, the maximum positive value is stored in the CPU unit, and when overflowing in the negative direction, the maximum negative value is stored in the CPU unit.

Inserting (INSERT/UPDATE) a number outside the value range of the RDB data type may result in an error.

*2 If the number of decimal places is specified in DECIMAL and a floating-point real number is specified for the PLC data type, the number is rounded to the nearest significant digit when converted from the PLC data type to the RDB data type.

If an integer is specified for the PLC data type in DECIMAL, only the integer part of the RDB data type is applicable, and the decimal point is truncated.

*3 Stores [year/month/day/hour/minute/second/millisecond/reservation (1 word)] in 8 consecutive words. The size is fixed. Time zone information is truncated.

*4 NULL is appended to the end of the string. The data size of the STRING should be set to an appropriate value including NULL according to the character code to be stored. A maximum of 2000 bytes (1000 words) can be specified. If the number of bytes exceeds 2000 bytes (1000 words), the excess is truncated, and NULL is stored at the end.

If TEXT, MEDIUMTEXT, LONGTEXT, or JSON is specified, the data size is fixed and occupies 2000 bytes (1000 words). If TINYTEXT is specified, the data size is fixed and occupies 1022 bytes (511 words).

*5 Only the least significant bit is turned ON/OFF. Upper 15 bits are reserved area.

*6 LREAL will be assigned to PLC_TYPE when read in the mapfile creation sheet. Change PLC_TYPE to REAL if necessary.

*7 When a column of DECIMAL type with an unsigned setting is read out in the mapfile creation sheet, PLC_TYPE is assigned to a data type with a sign. Change PLC_TYPE if necessary.

MariaDB RDB Data Types

Type	RDB data type	PLC data type
Number	BIGINT	LINT (64-bit signed integer) ULINT (64-bit unsigned integer) LREAL (double precision floating-point real number)
	INT/MEDIUMINT	DINT (32-bit signed integer) UDINT (32-bit unsigned integer)
	SMALLINT/TINYINT/BOOL/BOOLEAN	INT (16-bit signed integer) UINT (16-bit unsigned integer)
	BIT(1)	BOOL[16] (16 bits) ^{*5}
	DECIMAL (to 5, N) ^{*7}	INT (16-bit signed integer) ^{*1} UINT (16-bit unsigned integer) ^{*1} REAL (single-precision floating-point real number) ^{*2}
	DECIMAL (6 to 7, N) ^{*7}	DINT (32-bit signed integer) ^{*1} UDINT (32-bit unsigned integer) ^{*1} REAL (single-precision floating-point real number) ^{*2}
	DECIMAL (8 to 10, N) ^{*7}	DINT (32-bit signed integer) ^{*1} UDINT (32-bit unsigned integer) ^{*1} LREAL (double precision floating-point real number) ^{*2}
	DECIMAL (11 to, N) ^{*7}	LINT (64-bit signed integer) ^{*1} ULINT (64-bit unsigned integer) ^{*1} LREAL (double precision floating-point real number) ^{*2}
	DOUBLE	LREAL (double precision floating-point real number)
	FLOAT	LREAL (double-precision floating-point real number) ^{*6}
Date/time	DATE/DATETIME/TIMESTAMP/TIME	UINT[8] (date and time) ^{*3}
Character string	CHAR/VARCHAR/TINYTEXT/TEXT/ MEDIUMTEXT/LONGTEXT	STRING (Character string) ^{*4}

*1 Depending on the settings, the range handled by RDB data types and PLC data types do not necessarily match.

If a number outside the range of the PLC data type is retrieved (SELECT), it will overflow. When overflowing in the positive direction, the maximum positive value is stored in the CPU unit, and when overflowing in the negative direction, the maximum negative value is stored in the CPU unit.

Inserting (INSERT/UPDATE) a number outside the value range of the RDB data type may result in an error.

*2 If the number of decimal places is specified in DECIMAL and a floating-point real number is specified for the PLC data type, the number is rounded to the nearest significant digit when converted from the PLC data type to the RDB data type.

If an integer is specified for the PLC data type in DECIMAL, only the integer part of the RDB data type is applicable, and the decimal point is truncated.

*3 Stores [year/month/day/hour/minute/second/millisecond/reservation (1 word)] in 8 consecutive words. The size is fixed. Time zone information is not included.

*4 NULL is appended to the end of the string. The data size of the STRING should be set to an appropriate value including NULL according to the character code to be stored. A maximum of 2000 bytes (1000 words) can be specified. If TEXT, MEDIUMTEXT, or LONGTEXT is specified, the data size is fixed and occupies 2000 bytes (1000 words). If the number of bytes exceeds 2000 bytes (1000 words), the excess is truncated, and NULL is stored at the end. If TINYTEXT is specified, the data size is fixed and occupies 1022 bytes (511 words).

*5 Only the least significant bit is turned ON/OFF. Upper 15 bits are reserved area.

*6 LREAL will be assigned to PLC_TYPE when read in the mapfile creation sheet. Change PLC_TYPE to REAL if necessary.

*7 When a column of DECIMAL type with an unsigned setting is read out in the mapfile creation sheet, PLC_TYPE is assigned to a data type with a sign. Change PLC_TYPE if necessary.

Oracle Database RDB Data Types

Type	RDB data type	PLC data type
Number	NUMBER (to 5, N)	INT (16-bit signed integer) ^{*1} UINT (16-bit unsigned integer) ^{*1} REAL (single-precision floating-point real number) ^{*2}
	NUMBER (6 to 7, N)	DINT (32-bit signed integer) ^{*1} UDINT (32-bit unsigned integer) ^{*1} REAL (single-precision floating-point real number) ^{*2}
	NUMBER (8 to 10, N)	DINT (32-bit signed integer) ^{*1} UDINT (32-bit unsigned integer) ^{*1} LREAL (double precision floating-point real number) ^{*2}
	NUMBER (11 to, N)	LINT (64-bit signed integer) ^{*1*5} ULINT (64-bit unsigned integer) ^{*1*5} LREAL (double precision floating-point real number) ^{*2}
	FLOAT	LREAL (double precision floating-point real number)
	BINARY_FLOAT	REAL (single-precision floating-point real number)
	BINARY_DOUBLE	LREAL (double precision floating-point real number)
Date/time	TIMESTAMP	UINT[8] (date and time) ^{*3}
Character string	VARCHAR2/NVARCHAR2/CHAR/NCHAR	STRING (Character string) ^{*4}

^{*1} Depending on the settings, the range handled by RDB data types and PLC data types do not necessarily match.

If a number outside the range of the PLC data type is retrieved (SELECT), it will overflow. When overflowing in the positive direction, the maximum positive value is stored in the CPU unit, and when overflowing in the negative direction, the maximum negative value is stored in the CPU unit.

Inserting (INSERT/UPDATE) a number outside the value range of the RDB data type may result in an error.

^{*2} If the number of decimal places is specified in NUMBER and a floating-point real number is specified for the PLC data type, the number is rounded to the nearest significant digit when converted from the PLC data type to the RDB data type.

If an integer is specified for the PLC data type in NUMBER, only the integer part of the RDB data type is applicable, and the decimal point is truncated.

^{*3} Stores [year/month/day/hour/minute/second/millisecond/reservation (1 word)] in 8 consecutive words. The size is fixed. Time zone information is not included.

^{*4} NULL is appended to the end of the string. The data size of the STRING should be set to an appropriate value including NULL according to the character code to be stored. A maximum of 2000 bytes (1000 words) can be specified. If the number of bytes exceeds 2000 bytes (1000 words), the excess is truncated, and NULL is stored at the end.

^{*5} The range of values that can be SELECT is as follows.

LINT : -9,007,199,254,740,991 to 9,007,199,254,740,991

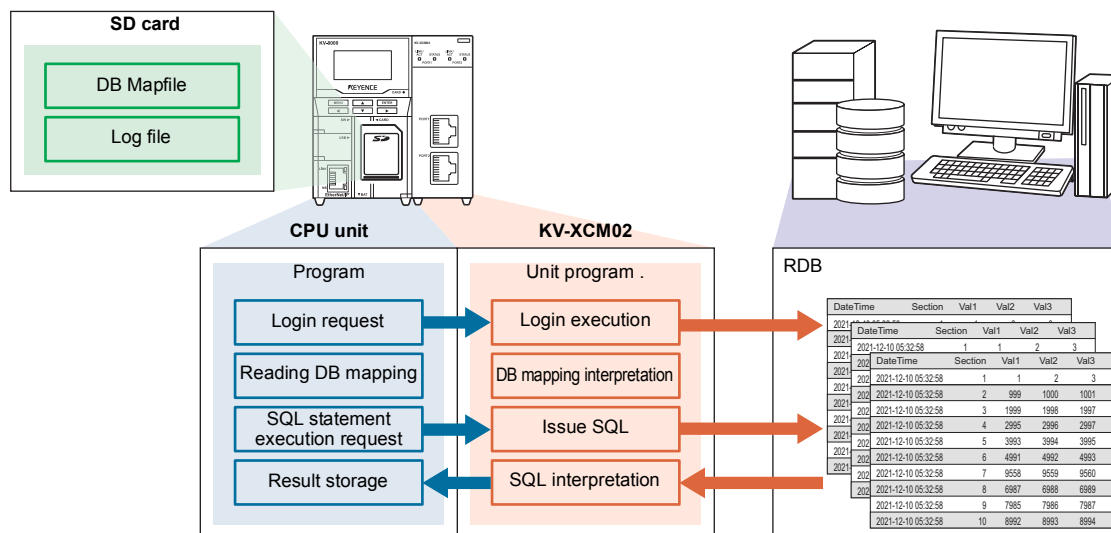
ULINT : 0 to 9,007,199,254,740,991

Chapter3 RDB Connection Settings

This section describes the settings required to connect to RDB using the RDB connection function.

3-1 Program Image

The RDB connection function operates as follows



3-2 Program Structure

The following programs are required for the RDB connection function.

Target	Program Type	Name	Description
CPU unit	Scan module every time	RDBC	Controls RDB connection functions.
		RDBC_management	Controls connection management and reading of DB Mapfile.
	Initialization Module	RDBC_cfg	Performs initial configuration of the RDB.
	Periodic Module	RDBC_timeStamp	Retrieves PLC time information (timestamp).
	Function block	RDBC_cfgRelation	Reads the DB Mapfile.
		RDBC_delete	Creates a DELETE SQL statement.
		RDBC_getPlcTime	Retrieves PLC time information (timestamp).
		RDBC_insert	Creates an INSERT SQL statement.
		RDBC_manage	Controls communication with the RDB.
		RDBC_select	Creates an SELECT SQL statement.
		RDBC_startUpg	Launch the unit program.
		RDBC_storedProcedure	Controls stored procedure execution functions.
		RDBC_truncate	Creates a TRUNCATE SQL statement.
		RDBC_update	Creates an UPDATE SQL statement.
	Function	RDBC_accountCfg	Set login information.
		RDBC_copyright	This section contains information about copyrights and the terms of use (license) for distributing the unit program. Be sure to import it.
		RDBC_getErrMsg	Retrieve RDB error messages.
		RDBC_getSQLSTATE	Retrieve RDB (PostgreSQL) status.
		RDBC_pwdEnc	Obfuscate passwords for login.
KV-XCM02	Unit program .	RDB_client_VXXX.kpo	Communicate with RDB.



Point

- Ensure to import all programs. If there are programs missing, the RDB connection function will not work properly.

☞ "3-3 Importing Programs" (page 3-2)

- RDBC_copyright FUN contains information about copyrights and the conditions of use (license) when distributing unit programs. Be sure to import and check the contents.

3-3 Importing Programs

This section describes the procedure for importing program from a sample project.

The following two sample projects are available

- RDBClient_Device_YYY_VXXX.kpz: Sample project described by device
- RDBClient_Variable_YYY_VXXX.kpz: Sample project described by variables (for KV-8000 series only)
 - * XXX is a number representing the version. For example, in the case of V700, it represents Ver. 7.00.
- YYY is a string representing a language. Contains the following strings.

JPN: Japanese
 ENG: English
 CHS: Chinese (Simplified)
 CHT: Chinese (Traditional)
 KOR: Korean
 US: English (U.S.)

Reference Please download the sample project from the following website.
www.keyence.co.jp/kv_database_soft

Here is an example of the procedure for importing the RDBClient_Device_ENG_VXXX.kpz (device version) sample project. The variable version should be read as RDBClient_Variable_ENG_VXXX.kpz.

1 Open the source RDBClient_Device_ENG_VXXX.kpz and destination project in <KV STUDIO> respectively.

Point The sample project can be opened in "KV STUDIO" of Ver. 11.5 or later.

2 Confirm the device range used in the RDBClient_Device_ENG_VXXX project.

If the device range overlaps with that of the project to be imported, replace all devices used in the RDBClient_Device_ENG_VXXX project with the range of devices not used in the project being imported.

□ "5 List of Devices Used" (page A-10)

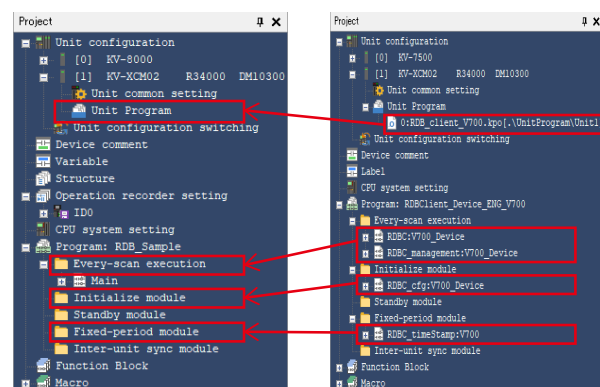
3 Import each of the following programs into the destination project by drag & drop.

Please Import according to the original program type when importing.

Program type	Program name
Unit program	RDB_client_VXXX.kpo
Scan module every time	RDBC
	RDBC_management
Initialization Module	RDBC_cfg
Periodic Module	RDBC_timeStamp

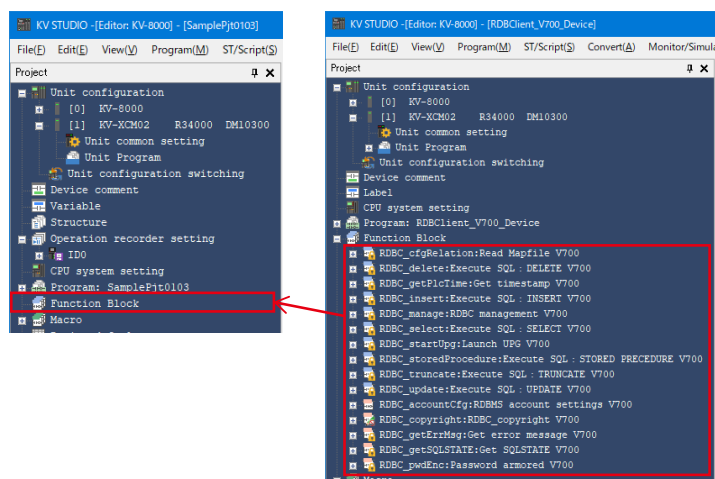
Import source project

Import source project:
RDBClient_Device_ENG_VXXX



Point Function blocks and functions are imported together when modules are imported.

Reference If only the FBs has been upgraded, only the FBs can be replaced.

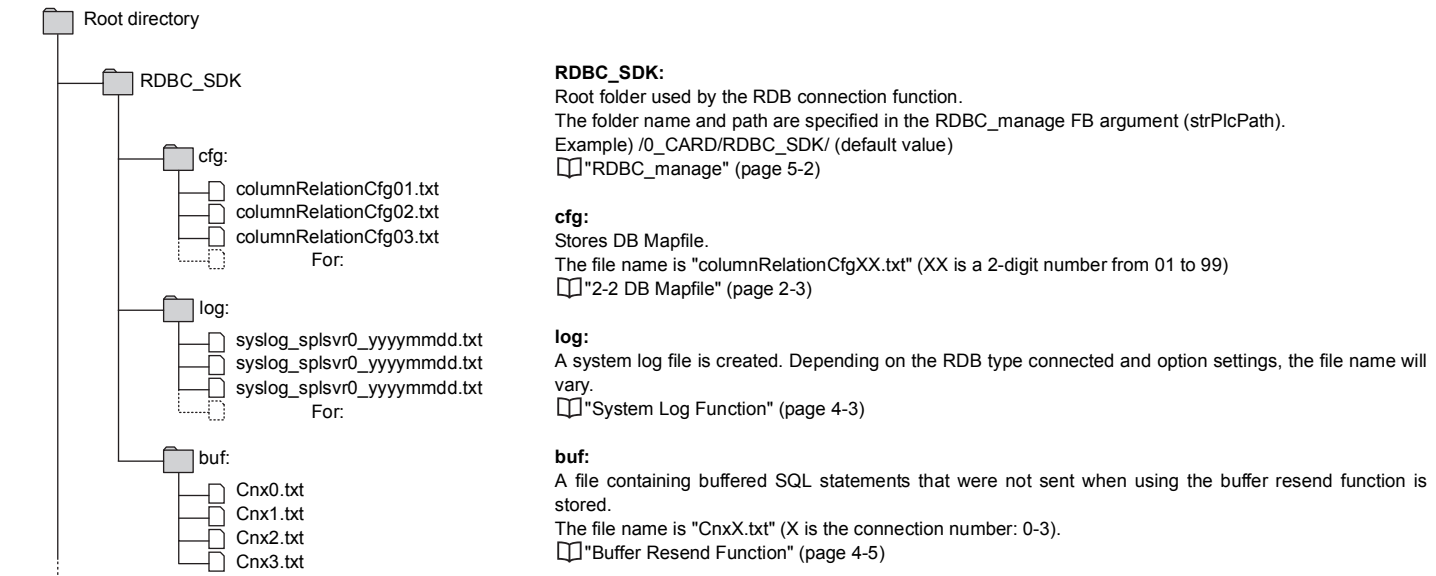


3-4 Folder Structure

This section describes the folder structure of the SD card inserted in the CPU unit.
The folder structure is as follows.

Reference

As the folders up to the root directory > RDBC_SDK are specified in the argument (strPlcPath) of RDBC_manage FB, the destination folder can be changed by changing the folder path to be stored in the argument.
"RDBC_manage" (page 5-2)



Point

- The log and buf folders are automatically created when RDBC_manage FB is launched.
- DB Mapfile should be stored in the cfg folder beforehand.
- When multiple units of "KV-XCM02" are connected to one CPU unit, specify different root folders in the argument strPlcPath of RDBC_manage FB.
Example) RDBC_SDK01, RDBC_SDK02...

3-5 Login Settings

When using the RDB connection function, first log into the RDB server.

This section describes the settings required for login.

Point In addition to the login settings in <KV STUDIO>, the RDB side must also be configured to allow TCP/IP access from external client software.

To perform a login, the following settings must be made in advance.

- Account settings such as username are required for login
- Set the location of configuration and log files to be used after login
- Set the target character set to be converted for use after login.

Account Settings

Account information such as username and password must be set for login.

Account information is set in RDBC_accountCfg. RDBC_accountCfg is a function executed inside RDBC_manage FB.

Set the following configuration points in the function according to the settings of the RDB server.

As the RDB connection function can access 4 RDBs simultaneously, 4 settings are provided for connection numbers 0 to 3.

- Edit the setting for the connection number to be used and change the always OFF execution condition.

- Enter the name of the database to be connected in DatabaseName.

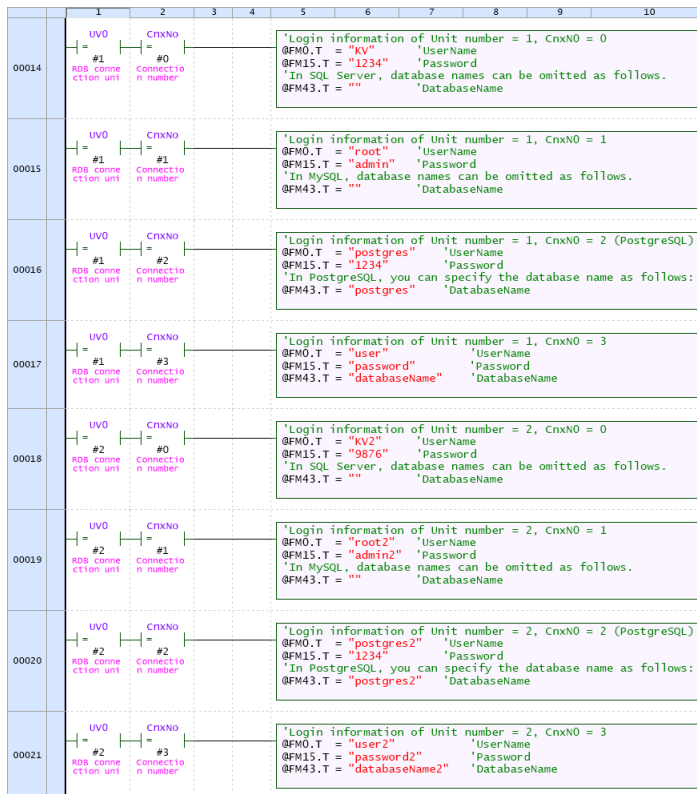
Point	- The username can be up to 29 single-byte alphanumeric characters, the password up to 127 single-byte alphanumeric characters, and the database name and URL up to 59 single-byte alphanumeric characters. - The database name setting is required or not for each RDB.												
	<table> <tr> <th>(RDB)</th><th>(Database name)</th></tr> <tr> <td>PostgreSQL</td><td>Required</td></tr> <tr> <td>Microsoft SQL Server</td><td>Required</td></tr> <tr> <td>MySQL</td><td>Optional</td></tr> <tr> <td>MariaDB</td><td>Optional</td></tr> <tr> <td>Oracle Database</td><td>Specify part of the URL</td></tr> </table>	(RDB)	(Database name)	PostgreSQL	Required	Microsoft SQL Server	Required	MySQL	Optional	MariaDB	Optional	Oracle Database	Specify part of the URL
(RDB)	(Database name)												
PostgreSQL	Required												
Microsoft SQL Server	Required												
MySQL	Optional												
MariaDB	Optional												
Oracle Database	Specify part of the URL												

Reference To prevent others from seeing the account information, set a password for RDBC_accountCfg after configuration.

📖 "KV STUDIO User's Manual"

Reference

Even when using multiple <KV-XCM02> for one CPU unit, use a single RDBC_accountCfg. Account information can be set individually for each unit/connection number. In such a case, use the ladder program of RDBC_accountCfg as shown below, adding a comparison contact using the unit number as an execution condition.



When connecting to Oracle Database, communicate with RDB via HTTP. In that case, please enter the URL to connect to DatabaseName in the following format.
(Example 1) When accessing /ords/ordtest/_/sql without using a proxy server

```
'Login information of CnxNo = 3
@FM0.T = "C#KV" 'UserName
@FM15.T = "1234" 'Password
@FM43.T = "ords/ordtest/_/sql" 'DatabaseName
```

(Example 2) Accessing http://192.168.0.100:8080/ords/ordtest/_/sql using a proxy server

```
'Login information of CnxNo = 3
@FM0.T = "C#KV" 'UserName
@FM15.T = "1234" 'Password
@FM43.T = "http://192.168.0.100:8080/ords/ordtest/_/sql" 'DatabaseName
```

Root Folder Path Setting

Set the folder path (root folder path) for the folder that stores the configuration file of the RDB connection function used after login and the log file at runtime. This setting is common to all 4 connections managed by a single "KV-XCM02".

☞ "3-4 Folder Structure" (page 3-3)

Point

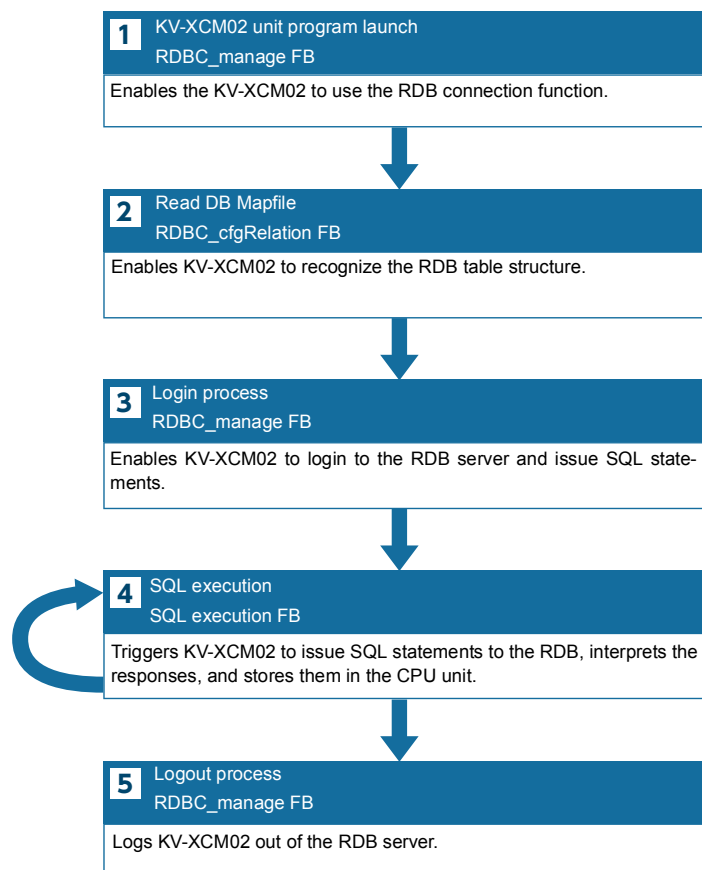
- The root folder path is specified with the argument strPlcPath of RDBC_manage FB.
Append "/" to the beginning and end of the path, and set the path within 40 bytes, including the terminating null character.
Example 1: RDBC_SDK folder on SD card "/0_CARD/RDBC_SDK/"
Example 2: RDBC_SDK folder in CPU memory "/1_CPUMEM/RDBC_SDK/"
When using multiple connections on a single "KV-XCM02", RDBC_manage FB must be placed and configured for the number of connections. Even in this case, specify a common root folder path for strPlcPath.
- If multiple "KV-XCM02" units are connected to one CPU unit, specify a different root folder for strPlcPath.

Chapter4 RDB Connection Function

This section describes the procedure for connecting to RDB and exchanging data.

4-1 Flow of Operation

The RDB connection function operates as follows.



1 Launches the unit program of <KV-XCM02>.

Execute RDBC_manage FB to launch the "KV-XCM02" unit program.

📖 "RDBC_manage" (page 5-2)

2 Read DB Mapfile

Read the DB Mapfile using RDBC_cfgRelation FB.

Reads the DB Mapfile from the cfg folder of the configured root folder. DB Mapfile can also be read after the login process is executed.

📖 "RDBC_cfgRelation" (page 5-7)

3 Login process

Turn ON the RDBC_manage FB login request relay to log in to the RDB.

Set the root folder of RDB connection function, IP address, port number, and login information of RDB and then login.

📖 "RDBC_manage" (page 5-2)

4 SQL execution

Execute SQL using the following function blocks

Function block name	Description	Related pages
RDBC_select	The FB to execute SELECT. Used to retrieve and read data from RDB that matches the specified conditions.	5-9
RDBC_insert	The FB to execute INSERT. Used to insert or add specified data to the RDB.	5-12
RDBC_update	The FB to execute UPDATE. Used to update or modify data in the RDB that matches the specified conditions.	5-14
RDBC_delete	The FB to execute DELETE. Used to delete data that matches the specified condition from the RDB.	5-16
RDBC_truncate	The FB to execute TRUNCATE. Used to delete all data in the specified table.	5-17
RDBC_storedProcedure	Executes a stored procedure defined in RDB.	5-18

Executes SQL statements such as record retrieval and registration.

At the time of execution, the target data is switched by specifying the DB Mapfile number. Therefore, to execute SQL while switching the target data, it is necessary to read the DB Mapfile for that amount in advance. Up to 99 DB Mapfiles can be read.

5 Logout process

Execute a logout request from RDBC_manage FB to log out of the RDB.

After executing login, there is no issue in continuing to execute SQL without logging out.

📖 "RDBC_manage" (page 5-2)

4-2 Various Function

This section describes the functions available in the RDB connection function.

Login Function

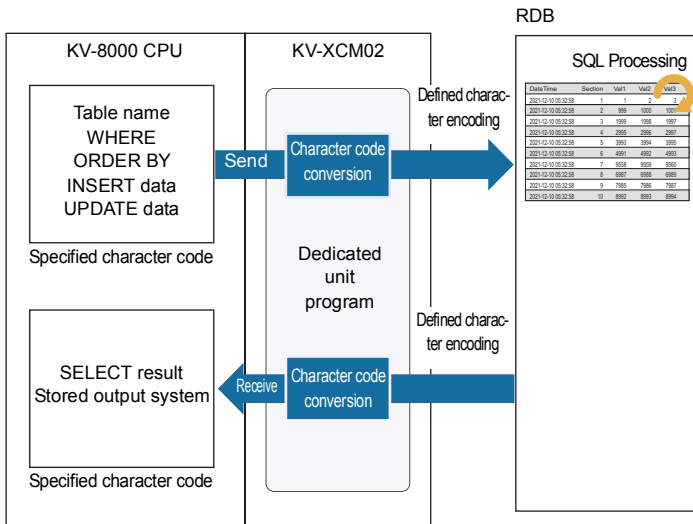
Logging into the RDB is done via the RDBC_manage FB login request relay. When logging in, connection settings such as RDB type and RDB IP address are passed from the CPU unit to the KV-XCM02's RDB connection function.

Automatic Login Function

Once login has been completed, if SQL is executed after logout from the server due to session timeout or processing interruption, the RDB connection function will take over the connection settings from the most recent login and automatically log in to execute SQL.

Character Code Conversion Function

Communication between "KV-XCM02" and RDB is based on the character encoding defined in the RDB specifications. The RDB connection function can convert character codes between "KV-XCM02" and the CPU unit.



The character code for sending strings in the CPU unit to the RDB and the character code for storing strings received from the RDB in the CPU unit can be specified separately. The character code conversion targets are schema name, table name, SELECT result data, INSERT (UPDATE) target data, stored procedure input/output/output arguments, result set and return value, WHERE clause, and ORDER clause. Shift-JIS (SJIS), Big5, GB2312, KS_X_1001, UTF-8 (UTF8), and UTF-16 (UTF16) are supported.

Character code conversion target

- String data sent to RDB

Data to be converted	Description
Table name	Character code of the string specified in strTblName
WHERE clause	Character code of the string specified in strWhere
ORDER clause	Character code of the string specified in strOrder
Data to be INSERT Data to be UPDATE	Character code of STRING type among the target data specified in arrValIn

- Incoming string data from RDB

Data to be converted	Description
SELECT result data Stored output system	Character code of STRING type among the target data specified in arrValOut (except for some data types*)

Point

- Microsoft SQL Server's char/varchar/text data types do not convert the character code of the received string, and the data received from the RDB is stored in the CPU unit as is. When using the character code conversion function, the data type should be nchar/nvarchar/text.
- When handling double-byte characters (Japanese) in the KV series, specify SJIS.
- The SELECT result can be displayed on the VT series by matching the character encoding of the character display component of the VT series with the character code to be converted by the character code conversion function. The character encodings that can be used in the VT series character display components are as follows
 - SJIS(Shift_JIS)
 - UNICODE(UTF-16)
 - Big5
 - GBK(G2312)
 However, Big5 and GBK can be used only with VT STUDIO Ver. 8.22 or later and the VT5 series.

SQL Execution Function

By turning on bSQLReq of each SQL execution FB, an SQL execution request is made to "KV-XCM02".

When the request is successfully accepted, and execution is completed, processing completion (bSQLComp) is turned ON.

SQL Execution Interruption Function

This function is used to interrupt SQL execution in progress.

Each SQL execution FB has an interrupt request (bSQLAbt) to abort SQL execution. After the interruption process is completed, the user is automatically logged out of the RDB.

SQL Timeout Function

It monitors the time from when the SQL execution request turns ON until the processing completion turns ON, detects a timeout when the set timeout time (duTimeOut) elapses, turns ON the SQL execution error (bAppErr), and ends the processing. When a timeout occurs, SQL processing execution is interrupted and logged out of the RDB after completion.

System Log Function

This function logs the failed SQL requests to "KV-XCM02".

The error code and error message for each RDB can be referenced to confirm the cause of the SQL error.

The system log file is created under

/0_CARD/RDBC_SDK/log/

The log folder is automatically created when RDBC_manage FB is launched.

 **The "/0_CARD/RDBC_SDK/" part can be set in RDBC_manage FB. To specify CPU memory, use /0_CARD/ as /1_CPUMEM/.**

File path

The file name is [sysLog "RDB server type" X.log] (X is the connection number: 0-3).

The extension (.log) can be prefixed with "_" (date/time information)".

The information to be added can be optionally specified as year, month, day, hour, minute, and second.

- Example of file path
/0_CARD/RDBC_SDK/log/syslog_mysql0_2021121906.log
(System log file as of 6:00, 12/19/2021)

Format

The format of the log is as follows.

```
(event category)<blank>(time of occurrence)<blank>(communication error code)
<tab>(error code for each RDB vendor)<tab>(SQL execution error code)
<tab>(SQL statement where the error occurred)<tab>(error message)
```

- Log example

```
err 2022-03-02T11:55:44.000Z 0 102 0 SELECT TOP (1)
processTimes FROM test.dbo.testInsp WHERE inspDateTime >=
'2022-02-22 19:20:21' ORDER BY NGWORD Incorrect syntax near
'processTimes DESC processTimes'.
```

The character encoding of the log file is created in the following character encoding used by each RDB.

RDB	Character code
PostgreSQL	UTF-8
Microsoft SQL Server	UTF-16
MySQL	UTF-8
MariaDB	UTF-8
Oracle Database	UTF-8

Stored Procedure Execution Function

KV-XCM02's RDB connection function allows users to execute stored procedures (including stored functions). A procedure is a function defined by combining multiple SQL statements on the RDB side. A procedure has a return value, while a function does not.

Since multiple processes can be executed together, it has the following advantages over controlling INSERT/SELECT/UPDATE/DELETE and other processes combined in a ladder program.

- Complex processing can be performed
- Ladder sequences can be simplified
- Reduces processing time related to communication
- Data can be processed in a consistent manner

The specifications for stored procedures in the RDB connection function are as follows

Item	Description
Supported RDB	PostgreSQL Microsoft SQL Server MySQL MariaDB
Number of stored procedure arguments*1	Input argument: 100 items Input/output arguments: 100 items Output argument: 100 items Maximum total 100 items Up to 200,000 Bytes each*3
Return value of stored procedure*1	1st item (output after type conversion to string (maximum 199Byte))
Stored procedure result set*1	1,000 items of one type*2 (multiple records can be obtained) Up to 200,000 Bytes*3
Supported data type	Numeric, date/time, string  "2-3 PLC Data Type" (page 2-7)

*1 There are restrictions on the number of arguments that can be combined depending on the RDB type.

*2 In the case of Microsoft SQL Server, the number of items is 1024.

*3 The maximum size of the data storage area to be allocated by the CPU unit.

 **Stored procedures for Oracle Database are not supported.**

Specification of stored procedure execution function of each RDB

This section describes the specifications for each RDB when using the stored procedure execution function.

Please also check the restrictions that need to be considered for each RDB.

 "2 Restriction" (page A-5)

PostgreSQL

Item	Description
Supported stored procedures	Functions (user-defined functions) created by PL/pgSQL
How to specify a stored procedure name	When specifying a schema: "Schema name". "Stored procedure name" When omitting the current schema: "Stored procedure name"
Other precautions	- Functions that omit argument names on the RDB side are not supported. - When using a result set, prepare only one OUT argument of type REFCURSOR and assign the result set to its cursor. At this time, set the DB Mapfile ID of the result set to the argument arrCfgNo+3 (result set) of RDBC_stored-Procedure FB. - When using a result set, all input/output arguments, output arguments, and return values are not available. - When a result set is not used, input/output arguments, output arguments, and return values are available. However, if I/O arguments or output arguments are used, the return value cannot be used. The return value can be used only when neither input/output arguments nor output arguments are used. - Using the default values for input arguments set on the RDB side - To use default values for specific arguments →Delete the relevant arguments from the Mapfile creation sheet. - When default values are used for all arguments (including the case where only one argument is used) →Do not use the Mapfile creation sheet - Default values set on the RDB side cannot be used for input/output arguments.

Reference

The following is an example of a PL/pgSQL script that accepts a result set in a PostgreSQL function.

```
Function name: usf_sel_tbl (public schema)
IN argument: (1) in_int (integer type)
OUT argument: (1) out_ref (refcursor type)
Operation: The record with the "id" column of the table name "public.test" matches the IN argument in_int are retrieved.

CREATE OR REPLACE FUNCTION public.usf_sel_tbl(
    in_int integer,
    OUT out_ref refcursor)
RETURNS refcursor
LANGUAGE 'plpgsql'

AS $BODY$
BEGIN
    OPEN out_ref FOR SELECT * FROM "public.test"
    WHERE "id" = in_int;

END;
$BODY$;
```

MySQL • MariaDB

Item	Description
Supported stored procedures	Stored function
How to specify a stored procedure name	When specifying a schema: `schema name`. `stored procedure name` When omitting the current schema: `stored procedure name`.
Other precautions	- As the specification of the stored function in MySQL and MariaDB, only input parameters and return values can be set. Therefore, use only the input arguments and return values of RDBC_storedProcedure FB. Input/output arguments, output arguments, and result sets cannot be used. - The BIT() type cannot be used for the return value.

Microsoft SQL Server

Item	Description
Supported stored procedures	Stored procedures created by Transact-SQL
How to specify a stored procedure name	When specifying a schema: [Schema name]. [Stored procedure name] When omitting the current schema: [Stored procedure name]
Other precautions	- The argument name of SQL Server stored procedures must be prefixed with @. In the DB Mapfile, register the string of stored procedure argument name minus @ as argument name. - Specify the parameters declared with Output in the SQL Server stored procedure as input/output arguments or output arguments in RDBC_storedProcedure FB. - The maximum amount of data (I/O/output arguments and result set) that can be received using a stored procedure is approximately 25,000 Bytes. Attempting to receive data larger than this will result in connection error 150120. - Only one of the input/output and output arguments should be used. They cannot be used in combination. - If the stored value of the input/output argument, output argument, or result set is NULL, the error code 150000 is generated. - Using the default values for input arguments set on the RDB side - To use default values for specific arguments →Delete the relevant arguments from the Mapfile creation sheet. - When default values are used for all arguments (including the case where only one argument is used) →Do not use the Mapfile creation sheet - Default values set on the RDB side cannot be used for input/output arguments. - When the "time" type is obtained, "1900-01-01" is stored in the date field. - When acquiring string type data with input/output arguments, the data is treated as nvarchar, regardless of the data column definition/stored argument definition, and the character code conversion is always performed according to the received character code.

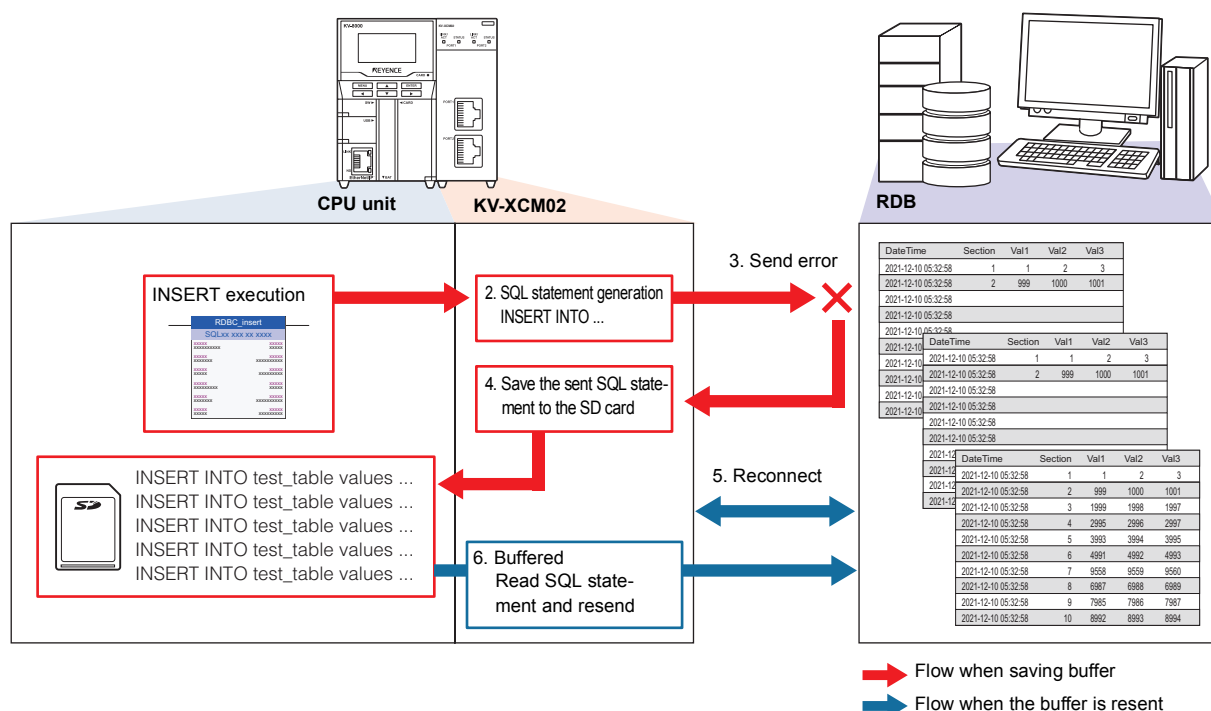
Buffer Resend Function

The buffer resend function is a function that saves SQL statements to be sent when the network between "KV-XCM02" and the RDB is disconnected or when the RDB is down, and resends after recovery.

The buffer resend function is configured in RDBC manage FB.

📖 "RDBC manage" (page 5-2)

Operational Overview



- **When saving buffer**

If an SQL statement cannot be correctly sent to the RDB server due to network disconnection or server down during send, the SQL statement is appended to a file on the SD card. The file is created for each connection and is called a buffer file.

- **Buffer resend**

When the failure is restored, and the RDB is reconnected, the SQL sentences stored in the SD card are read and resent in order from the top. The resend timing can be controlled by the ladder program.

 Refer to the RDBC manage FB for a time chart of the buffer resend function.

📖 "RDBC manage" (page 5-2)

Specifications of the buffer resend function

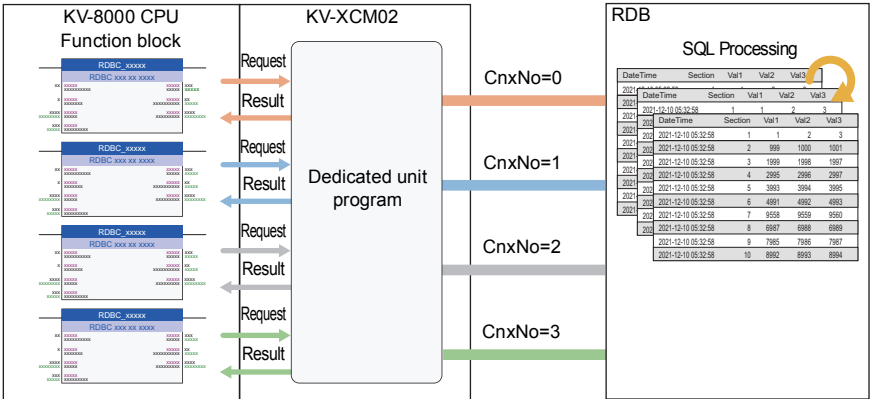
Item	Description
Supported SQL statement	INSERT (multi-row INSERT is also acceptable) UPDATE
Set Unit	Set enable/disable per connection
Buffer capacity	1GB per connection If the buffer size exceeds 1GB, error 100102 (buffer full) will occur.
Buffer storage location	SD card Save in the buf folder under the folder specified by the argument (strPlcPath) of the RDBC_manage function block Example: /0_CARD/RDBC_SDK/buf/Cnx0.txt (for connection 0)
Conditions to be saved in the buffer file	If any of the following occurs when executing the corresponding SQL statement on a connection with the buffer resend feature enabled, the SQL statement is appended to the buffer file. • When a timeout error occurs • When a communication error occurs (including when executed while a communication error occurs) • If the buffer already exists When errors other than the above occur, such as SQL execution errors, SQL statements are not saved to the buffer.
Conditions when a buffered SQL statement is resent	While the login relay is ON and the buffer resend permission relay is ON on a connection where the buffer resend function is enabled. * For Oracle Database, while the buffer resend permission relay is turned ON.
If an error occurs during buffer resend	• During timeout error or communication error Resend is aborted, and the relay is turned OFF during login (except in the case of Oracle Database). During this time, the error code is not stored. During the error, it is saved to the buffer file again. • In case of SQL execution error Ignore the error and continue resend.

Item	Description
If the corresponding SQL statement (INSERT/UPDATE) is executed during buffer resend	Buffer resend is interrupted, and the outgoing SQL statement is appended to the buffer file. Then resumes retransmission from the buffer.
If a SQL statement other than the corresponding SQL statement (e.g., SELECT) is executed during buffer resend	Resend from the buffer is given priority. It will not be executed until all buffers have been retransmitted or buffer resend permission is turned OFF. If a request for execution other than supported SQL is made during buffer resend, the request may be timed out. Therefore, please do not request execution other than the corresponding SQL during buffer resend.
Timing for clearing buffer files	Buffer files are deleted during the following. • All buffer SQL resend are completed • Completion of buffer clear by buffer clear request Otherwise, if not logged into the RDB, remove the SD card and delete the corresponding file.

Multiple Connections

The RDB connection function allows up to 4 connections to a single "KV-XCM02" unit. By having multiple connections to one RDB, different SQL processes can be executed in parallel, or connections can be made to different RDBs.

In SQL execution FB, the connection number is specified by CnxNo (range: 0 to 3).
The processes of SQL execution FBs with different CnxNo do not interfere with each other and are processed in parallel.



For example, if connection number 0 is used to periodically write data (INSERT) and connection number 1 is used to retrieve data (SELECT) at the time of variety switching, the INSERT process can be executed concurrently even while the SELECT is being executed, thus reducing time variation.

- Point
- **When using the RDB connection function, be sure to use connection 0.**
When using multiple connections, use multiple connection numbers including connection 0.
 - **When using multiple connections, place FBs with different CnxNo for the number of connections.**
For execution control arguments, set an independent device/variable for each connection.
Set the same values for UnitNo, UpgNo, and strPlcPath in the configuration arguments.
uRDBType and arrCfgCnx need not be set for each connection.
To set separately for each connection, set the self port number of arrCfgCnx to 0 (automatic).

Chapter5 Function Block

This section describes each function block (FB) used in the RDB connection function.

For details on setting the FB and how to set arguments, refer to the user's manual of the CPU unit used.

5-1 Overview of Function Blocks Used in the RDB Connection Function

The following FBs are used in the RDB connection function.

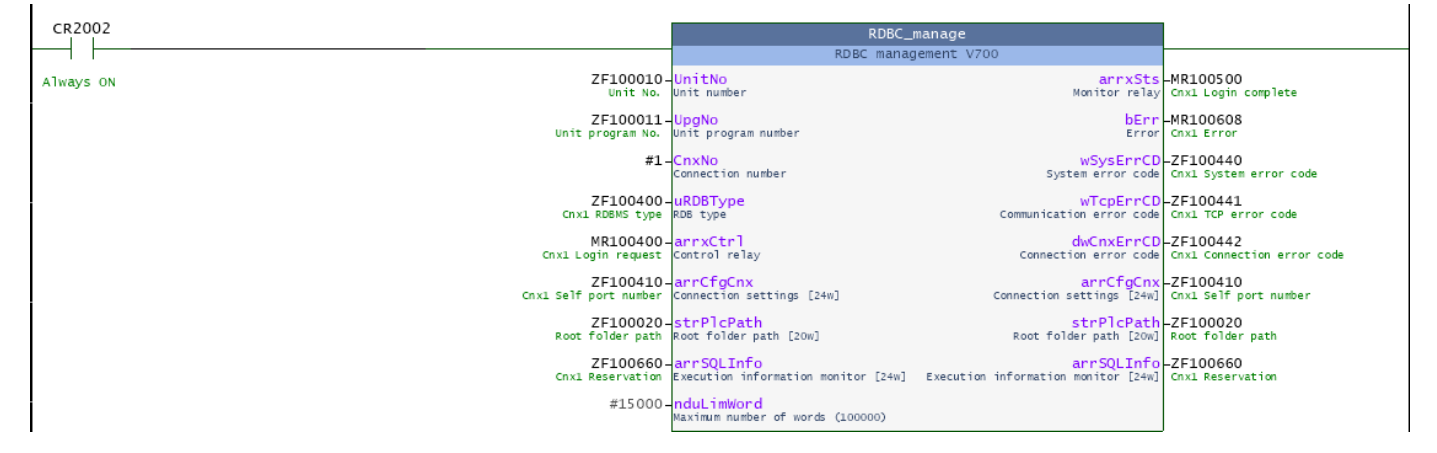
FB type	Function	Function block Program name	Command description	Related page
Management FB	RDBC manage	RDBC_manage	Controls the connection to the RDB, sets the connection destination, root folder path, etc.	5-2
Read mapping setting FB	Read DB Mapfile	RDBC_cfgRelation	Reads the DB Mapfile from the CPU Unit's memory card.	5-7
SQL execution FB	SQL execution: SELECT	RDBC_select	Retrieves records from the RDB matching the condition.	5-9
	SQL execution: INSERT	RDBC_insert	Register records matching the condition in the RDB.	5-12
	SQL execute : UPDATE	RDBC_update	Updates records on the RDB matching the condition.	5-14
	SQL execute: DELETE	RDBC_delete	Deletes records from the RDB matching the condition.	5-16
	SQL Execute: TRUNCATE	RDBC_truncate	Deletes all records in the specified table.	5-17
	SQL execute: Stored procedure	RDBC_storedProcedure	Executes stored procedure to obtain INOUT/OUT arguments, result set, and return value.	5-18
Obtain auxiliary information FB	Retrieve error message	RDBC_getErrMsg	Get RDB error message.	5-21
	Retrieve SQLSTATE	RDBC_getSQLSTATE	Get RDB status.	5-22
	Get timestamp	RDBC_getPlcTime	Creates data containing date/time timestamps.	5-23



- Point**
- In addition to the above, there are other FBs used internally in the system.
 - For the SQL execute FB, when two or more SQL processes are requested simultaneously, the other SQL process is executed after the first started process is completed. Hence, when using multiple types of SQL execute FBs, there is no need for exclusive processing of execution requests in the ladder program. Even if multiple requests are turned ON in the same scan, they are processed according to the execution order of the requests (execution order of FBs).

5-2 Management Function Block

RDBC_manage	RDBC Manage	Controls the connection to the RDB, sets the connection destination, root folder path, etc.
-------------	-------------	---



- Point
- RDBC_manage should be deployed with the condition that it is always ON.
 - RDBC_manage is placed basis the number of connections to be used.
Connection 0 must be used.
When using multiple connections, use multiple connection numbers, including connection 0.
 - KV-XCM02's permission relay is used inside RDBC_manage. Do not operate the "KV-XCM02" permission relay from the ladder program, etc., as some functions may not work correctly.

Argument Description

The description of each argument is as follows

● UnitNo: Unit number (IN/UINT)

Specify the unit number of "KV-XCM02.

● UpgNo: Unit program number (IN/UINT)

Specifies the program number of the unit program xcm_RDB_Client_Vxxx.kpo. Setting range is 0 to 255.

When the execution condition of RDBC_manage FB is turned ON, the unit program specified by UpgNo is automatically started. If the launch is successful, UPG startup (arrxSts.bUpgBusy) is turned ON. If the launch fails, the system error (arrxSts.bSysErr) turns ON.

● CnxNo: Connection number (IN/UINT)

Specifies the connection number used by the RDB connection function.

The range is 0 to 3.

Up to 4 connections can be used simultaneously in the RDB connection function.

● uRDBType: RDB type (IN/UINT)

Specifies the type of RDB to connect.

No.	Type
0	Microsoft SQL Server
10	MySQL5.7 to 8.0, MariaDB (10.2 or later)
11	MySQL5.5 to 5.6, MariaDB (10.1 or earlier)
20	PostgreSQL
30	Oracle Database
35	Oracle Database (using proxy server)

● arrxCtrl: Control relay (IN/UINT)

Specifies the processing request signal from the ladder program to RDBC_manage FB.

Bit position	Item	Description
bit0	Login request	A signal to request login to RDB
bit1	Logout request	A signal to request logout from RDB The RDB connection function logs out after all tasks currently waiting to be processed are completed.
bit2	Buffer resend permission	Turns ON when sending buffered SQL statements to the server. If the arrxSts buffer existence relay is turned ON after login is completed, turn this bit ON when need to resend. To resend automatically, create a ladder program that turns this bit ON under the AND condition of the online relay and the buffer existence relay of arrxSts.
bit3	Buffer Clear request	Turns ON when clearing a buffered SQL statement (all buffered SQL statements are automatically cleared even when they are resent). Hold until buffer clear complete is turned ON. However, it cannot be executed while the buffer saving process or buffer resend process is ON. When the buffer clear is performed, the buffered SQL statements are deleted, the current buffer stored SQL count, current buffer file read offset, and current buffer utilization are cleared to 0. If the SD card is removed and the buffer file is deleted, the number of SQL currently stored in the buffer will not be cleared, so please clear the buffer.
Bit4 to 12	Reservation	-
bit13	Communication error clear	Clears communication errors at the rising edge of the error clear signal. All requests awaiting processing are canceled.
bit14	Connection error clear	Clears the connection error at the rising edge of the error clear signal.
bit15	Reservation	-

Point

- Login and error clearing are interrupted by the SQL execution requests that are waiting to be processed.
- Logout by logout request is executed after all SQL executions waiting to be processed at the time of the request have been completed.
- The login/logout request should be held until the corresponding completion relay is turned ON.
- Error clear should be held ON until the corresponding error signal turns OFF.

● arrxSts: Monitor relay (OUT/UINT)

Specifies various monitor relays.

Offset	Item	Description
bit0	Login complete	Turns ON after the login execution process is completed. When the login request is turned OFF, the login completion is also turned OFF.
bit1	Logout complete	Turns ON after the logout execution process is complete. When the login request is turned OFF, the Logout completion is also turned OFF.
bit2	Online	Turns ON during online.
bit3	UPG running	It turns ON while the unit program for the RDB connection function is running.
bit4	System Error	Turns ON when an error occurs during unit program launch or system log folder creation.
bit5	Communication error	Turns ON when a communication error occurs.
bit6	Connection error	Turns ON when an error occurs in the connection process (login/logout).
bit7	Reservation	-
bit8	Buffer existence	Is ON while a buffered SQL statement is present. At the start of the operation, it is OFF regardless of the presence or absence of a buffer. It is updated when a login request is executed.
bit9	Reservation	-
bit10	Enable buffer clear	Turns ON when buffer clear can be executed.
bit11	Buffer clear complete	Turns ON when buffer clear is completed. Turns OFF when arrxCtrl's buffer clear request turns OFF.
bit12-15	Reservation	-

Point

When Oracle Database is used, it does not turn ON during login.

● bErr: Error (OUT/BOOL)

Output error status. Turns ON when bSysErr, bTcpErr, or bCnxErr is ON.

● wSysErrCD: System error code (OUT/INT)

Outputs error codes for unit program launch and system log folder creation.

□ "1 Error Code" (page A-1)

● wTcpErrCD: Communication error code (OUT/INT)

Outputs error codes related to socket communication.

□ "1 Error Code" (page A-1)

● dwCnxErrCD: Connection error code (OUT/UDINT)

Outputs error codes for login/logout.

□ "1 Error Code" (page A-1)

● arrCfgCnx: Connection settings [24w] (IN-OUT/UINT [24])

Set the contents of the connection settings.

Offset	Item	Description
+0	Self port No.	Setting range: 1 to 65535 If 0 is specified, a port number is automatically assigned.
+1	Connection destination IP address 1	Specifies the 1st byte of the connection destination IP address. ○○○.***.***.***
+2	Connection destination IP address 2	Specifies the 2nd byte of the connection destination IP address. ***.○○○.***.***
+3	Connection destination IP address 3	Specifies the 3rd byte of the connection destination IP address. ***.***.○○○.***
+4	Connection destination IP address 4	Specifies the 4th byte of the connection destination IP address. ***.***.***.○○○
+5	Connection destination port number	Specifies the connection destination TCP port number.
+6	Ethernet port used	Specify the Ethernet port (1 to 2) used by the RDB connection function. If specified out of range, it is treated as 1.
+7 to 9	Reservation	-
+10	Connection timeout	Specifies the timeout period for login requests (unit: ms). When set to 0, it is treated as 10000.
+11	Reservation	-
+12	Buffer resend function Enable	Specifies whether to use the buffer resend function. • 0: Not use • 1: Use Reads the specified value at login request.
+13 to 16	Reservation	Unused.
+17	Character code of sending string data	Specifies the character code for converting character strings sent from the PLC. • 1: SJIS • 2: UTF-8 • 3: UTF-16 (upper/lower byte swap) • 4: UTF-16 • 5: BIG5 • 6: GB2312 • 7: KS_X_1001 If a value outside the range is specified, a connection error occurs.
+18	Character code of received string data	Specifies the character code for converting strings received from the RDB. • 1: SJIS • 2: UTF-8 • 3: UTF-16 (upper/lower byte swap) • 4: UTF-16 • 5: BIG5 • 6: GB2312 • 7: KS_X_1001 If a value outside the range is specified, a connection error occurs.
+19	Enable system logging function	Specifies whether or not to leave a log when the system logging function fails. • 0: No logging • 1: Keep log
+20	System log name options	Specifies time information to be appended to the end of the file name to be created. • bit 0: seconds • bit 1: minutes • bit 2: hour • bit 3: day • bit 4: month • bit 5: year Turn ON each bit corresponding to the information to be added.
+21	GMT offset	Specify the GMT offset when connecting to Oracle Database (REST connection). For Japan, +9 hours is set. Invalid when using RDB other than Oracle Database.
+22 to 23	Reservation	-

Point

- When login is executed, items set in arrCfgCnx are reflected in the RDB connection function. Not reflected at the time of automatic login. The setting is not reflected in the RDB connection function even if it is changed when it is in a state other than when the login is executed.
- When storing the UTF-16 string "ABC" in DM0-3, the byte order differs between LE (little-endian) and BE (big-endian).
 - When storing in UTF-16LE
DM0 = 0x4100, DM1 = 0x4200, DM2 = 0x4300, DM3 = 0x0000
 - When storing in UTF-16BE
DM0 = 0x0041, DM1 = 0x0042, DM2 = 0x0043, DM3 = 0x0000

● strPlcPath: root folder path [20w] (IN-OUT/STRING [40])

Specify the root folder where the DB Mapfile and system log file are stored.

Up to 40 characters, including NULL, can be set. Append "/" to the beginning and end of the path.

The setting is reflected when the unit program is started and when logging in (the setting is not reflected when auto-login).

Point

- The root folder path is common when using multiple connections with one "KV-XCM02" unit.
- When using multiple units of "KV-XCM02", specify a different root folder path for each unit.

● arrSQLInfo: Execution Information Monitor [24w] (IN-OUT/UINT [24])

Outputs information about RDB connection and SQL execution.

Offset	Item	Description
+0 to 17	Reservation	-
+18 to 19	Current buffer accumulation SQL number	The number of SQLs accumulated in the buffer is stored. The number is added by 1 when buffer save is completed and is subtracted by 1 when buffer resend is completed.
+20 to 21	Current buffer file reading offset	The file pointer of the position where the retransmission was completed is stored. Do not change the stored value.
+22	Current buffer usage rate [0.01%]	The buffer usage rate is stored. It is updated when a buffer is saved (written). The buffer file is not cleared until all resend is completed, so the usage rate does not decrease during resend. When all resend are completed, the usage rate is set to 0.
+23	Reservation	-

● nduLimWord: Maximum number of words (100,000) (IN/UDINT)

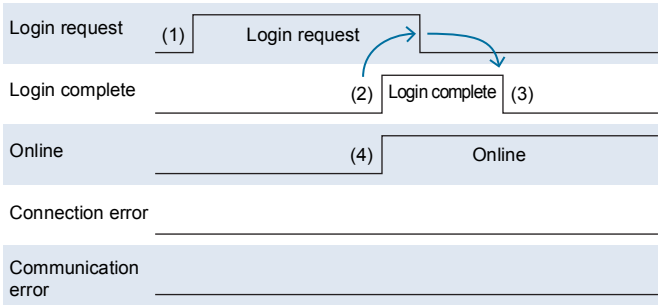
Sets the maximum record storage word size when using SELECT/INSERT and the result set of stored procedures. Up to 100,000 words (200,000 bytes) can be set. *1

Different sizes can be used for different connections. If left blank, the default value of 15,000 words (30,000 bytes) is used.

*1 For Oracle connections, the SQL statement to be sent must be within 100,000 Bytes. The outgoing SQL statement includes "table name," "column name," and "INSERT data."

Time Chart

Login Process

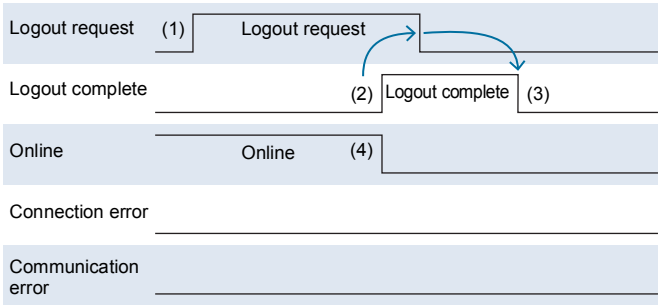


- (1) Turn ON the login request to start the process.
Hold ON until completion.
- (2) When processing is successful, the login complete turns ON.
If login is successfully completed, connection error and communication error will be OFF.
- (3) Confirm the login completion, and turn OFF the login request.
When the login request is turned OFF, the login complete is turned OFF.
- (4) When online, the online turns ON.

Point

Please execute the first login after starting RDBC_manage FB.
At the time of login, connection settings are registered in the RDB connection function. After that, even if SQL execution is requested from the logout state, the login process will be performed automatically, but the connection settings of the most recent login will be applied.

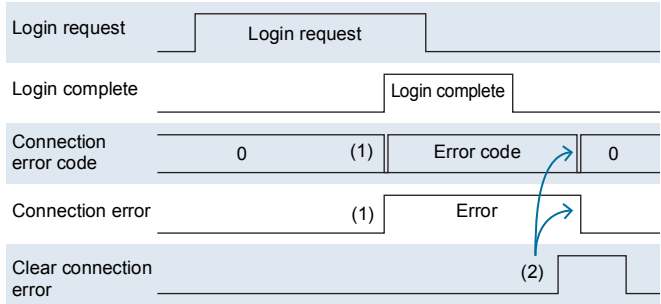
Logout Process



- (1) Turn ON the logout request to start the process.
Hold ON until completion.
- (2) When processing is successful, the logout complete turns ON.
If logout is successfully completed, connection error and communication error will be turned OFF.
- (3) Confirm the logout completion and turn OFF the logout request.
When the logout request is turned OFF, the logout completion is turned OFF.
- (4) When logout, the online turns OFF.

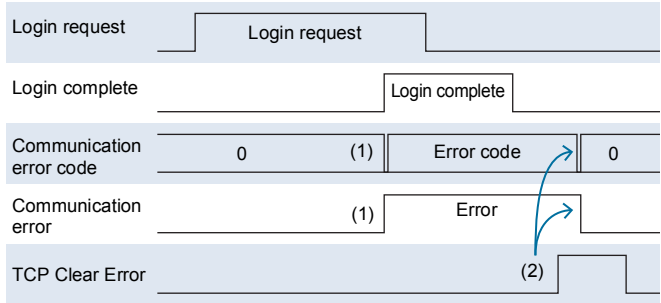
Login Failed

● Connection error



- (1) If an error occurs during login, the error code is stored in the connection error code, and the connection error turns ON when login is completed.
- (2) When the connection error clear is turned ON, the connection error code and connection error are cleared.

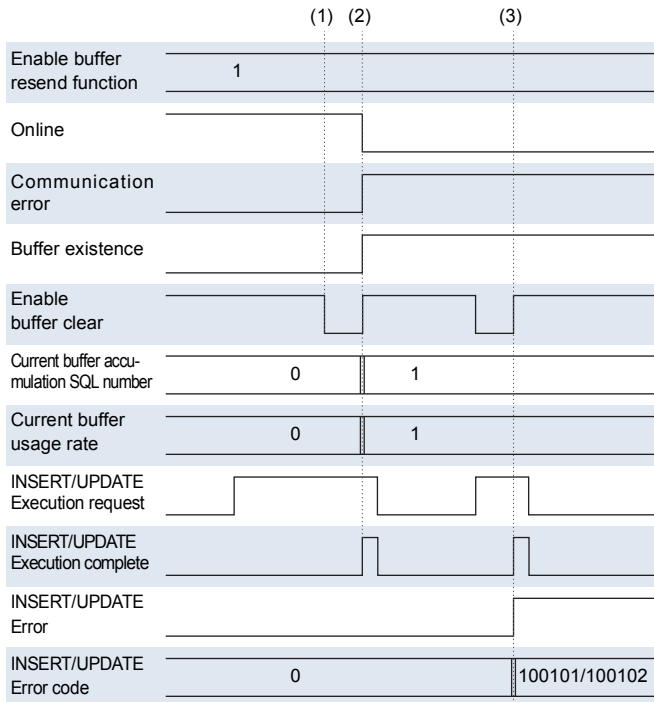
● Communication error



- (1) If login fails, a communication error may occur when login completion is turned on instead of (or at the same time) during connection error. In such cases, the error code is stored in the communication error code.
- (2) When TCP error clear is turned ON, the communication error code and communication error are cleared.

Buffer Resend Function

When saving buffer



- (1) If a communication error occurs after turning ON the INSERT/UPDATE execution request, the buffer clear enable is turned OFF and the SQL statement save process begins.
- (2) When the save process is successfully completed, the buffer existence and buffer clear enable are turned ON, the current buffer accumulation SQL number stored in the buffer is incremented by 1, and the current buffer usage rate is updated. Also, when the INSERT/UPDATE execution is completed, the communication error is turned ON, and online is turned OFF.

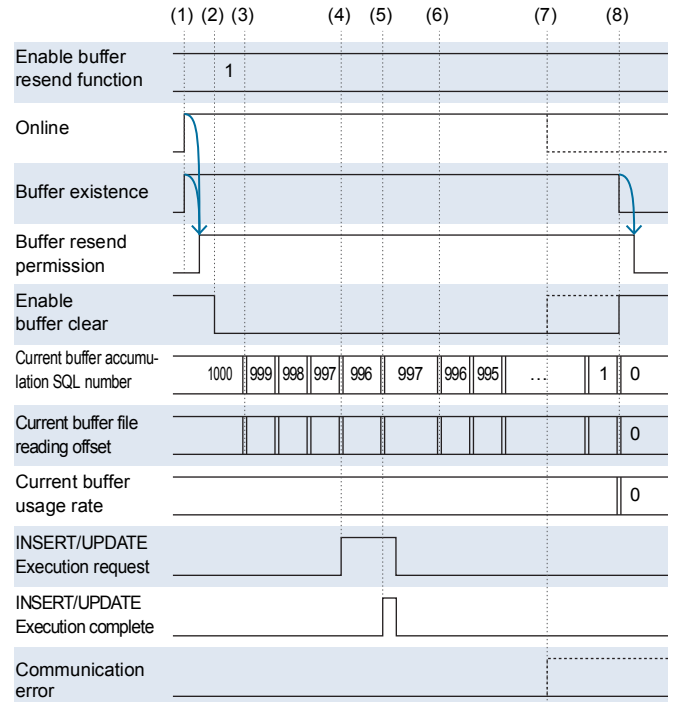


Point

The communication error is turned ON when a communication error occurs, and the SQL statement save process is completed.

- (3) If an error occurs in the save process, an INSERT/UPDATE error occurs and the error code is stored. In the case of a full buffer, the error code is 100102, and in the case of other errors (SD card not inserted, no buf folder, etc.), the error code is 100101.

When resending buffer



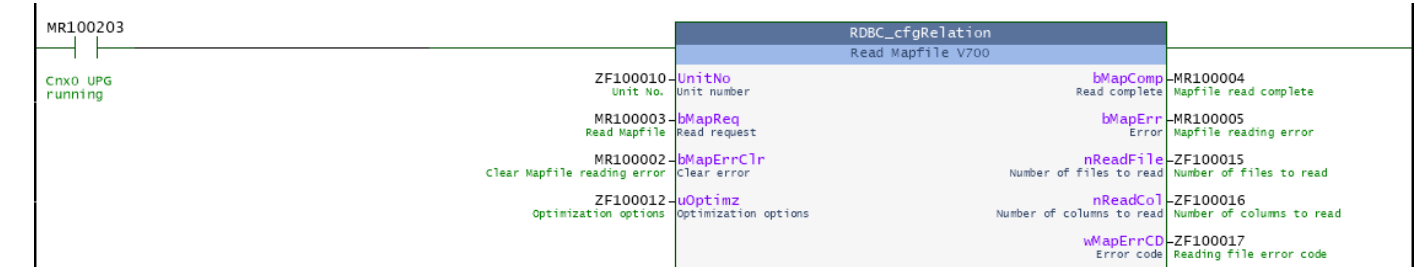
- (1) Check the ON during login and the ON of the buffer existence, and then turn ON the buffer resend permission (ladder).
- (2) Buffer clear enable turns OFF when buffer resend process starts.
- (3) Each time a resend is completed, the current buffer accumulation SQL number is set to -1, and the current buffer file reading offset is updated.
- (4) If the INSERT/UPDATE execution request is turned ON during the buffer resend process, the resend is interrupted for a moment when the resend process being executed is completed, and the send SQL statement is added to the buffer file.
- (5) When the save process is successfully completed, the current buffer accumulation SQL number is incremented by 1, and the INSERT/UPDATE execution completion is turned ON.
- (6) The buffer resend process is resumed.
- (7) If a communication error occurs during the buffer resend process, online is turned OFF, and the communication error is turned ON.
- (8) When all buffer SQL resend are completed, the buffer file is deleted, the buffer existence is turned OFF, and the current buffer accumulation SQL number, the current buffer reading offset, and the current buffer usage rate are cleared to 0. When buffer existence is OFF, the buffer resend permission is turned OFF.

5-3 Mapping Setting Read Function Block

RDBC_cfgRelation	Read DB Mapfile	Reads the DB Mapfile from the CPU Unit's memory card.
-------------------------	------------------------	---

Reads the DB Mapfile from the CPU Unit's memory card. This FB must be executed beforehand when performing SQL execution (INSERT/UPDATE/SELECT). A single execution reads up to 99 setting files.

DB Mapfile is a common setting among multiple connections of the RDB connection function. Even if multiple connections are used, only one RDBC_cfgRelation file is required.



- Point**
- The execution condition is "UPG running" in RDBC_manage, so please turn ON the login request of RDBC_manage and execute the unit program beforehand.
 - Store the DB Mapfile in the cfg folder directly under the root folder path (strPlcPath) in advance.
DB Mapfile name = "columnRelationCfgXX.txt" (XX is a 2-digit number from 01 to 99)
Readout is performed starting with XX at 01 and continuing up to 99.
 - Even if there is a missing number, all files numbered up to 99 are read.

Argument Description

The description of each argument is as follows

● UnitNo: Unit number (IN/UINT)

Specify the unit number of "KV-XCM02".

● bMapReq: Read request (IN/BOOL)

Specifies DB Mapfile read request.

Hold until completion.

If there is an SQL execution request waiting to be processed, it will be interrupted and processed.

If there are settings already read, they are all initialized.

● bMapErrClr: Error clear (IN/BOOL)

Specifies the error clear request when reading DB Mapfile.

When turned ON, wMapErrCD is cleared to 0 and bMapErr is turned OFF.

● bMapComp: Read complete (OUT/BOOL)

Outputs DB Mapfile reading complete.

Turns ON when the DB Mapfile is successfully read.

Turns OFF when bMapReq is turned OFF.

● bMapErr: Error (OUT/BOOL)

Outputs DB Mapfile reading error.

Turns ON when a value other than 0 is stored in wMapErrCD.

Turns OFF when bMapErrClr turns ON.

● nReadFile: Number of files to read (OUT/UINT)

Outputs the number of DB Mapfiles read.

Outputs 1 to 99 when readout is completed.

● nReadCol: Number of columns to read (OUT/UINT)

Outputs the number of columns read from the DB Mapfile.

The number is output from 1 to 20,000 when the readout is completed.

● wMapErrCD: Error code (OUT/INT)

Outputs the read error code of the DB Mapfile.

A value is entered when the readout process is completed. When the process ends normally, 0 is stored.

□ "1 Error Code" (page A-1)

● uOptimiz: Optimization options (IN/UINT)

Specifies whether to perform optimization processing according to the RDB to be connected when reading the setting file. Optimization speeds up SQL execution to the target RDB.

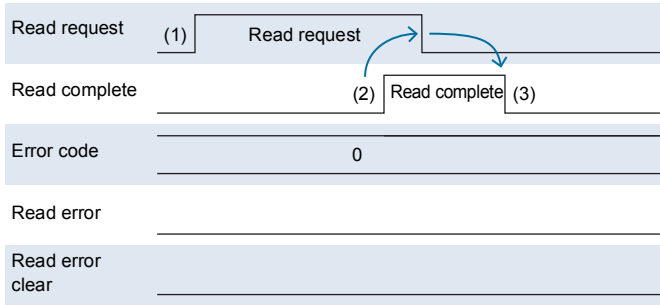
uOptimiz value	SQL Server	RDB other than the SQL Server
0 to 9	Accelerate.	No acceleration.
10 or more	No acceleration.	Accelerate.

Point

- Optimization options are reflected at the time of reading the DB Mapfile.
- Optimization options are the same for all connections. Cannot be set per connection.

Time Chart

Read request (normal end)

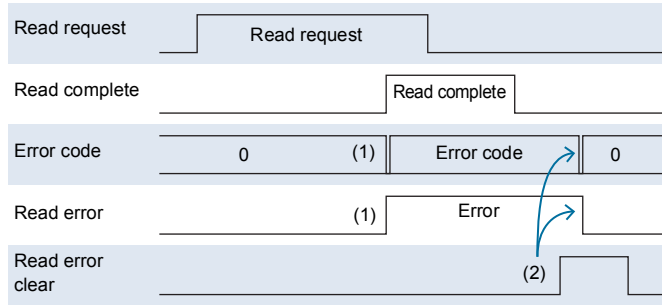


- (1) When read request is turned ON, processing begins. Hold ON until completion.
- (2) When processing is successfully completed, the readout completion is turned ON, and the setting readout error is turned OFF.
- (3) Confirm the read complete and turn OFF the read request. When the read request is turned OFF, the read complete is turned OFF.

Point

- While the read request is in progress, the read error clear should be turned OFF.
- DB Mapfile should be stored in the cfg folder directly under strPlcPath.
- DB Mapfile is read in order from No. 01. Even if there is a missing number, all files numbered up to 99 are read.

Read request (abnormal end)



- (1) If an error occurs during processing, the error code is stored, and the read error turns ON.
- (2) When read error clear is turned ON, the error code and read error are cleared.

5-4 SQL Execution Function Block

RDBC_select

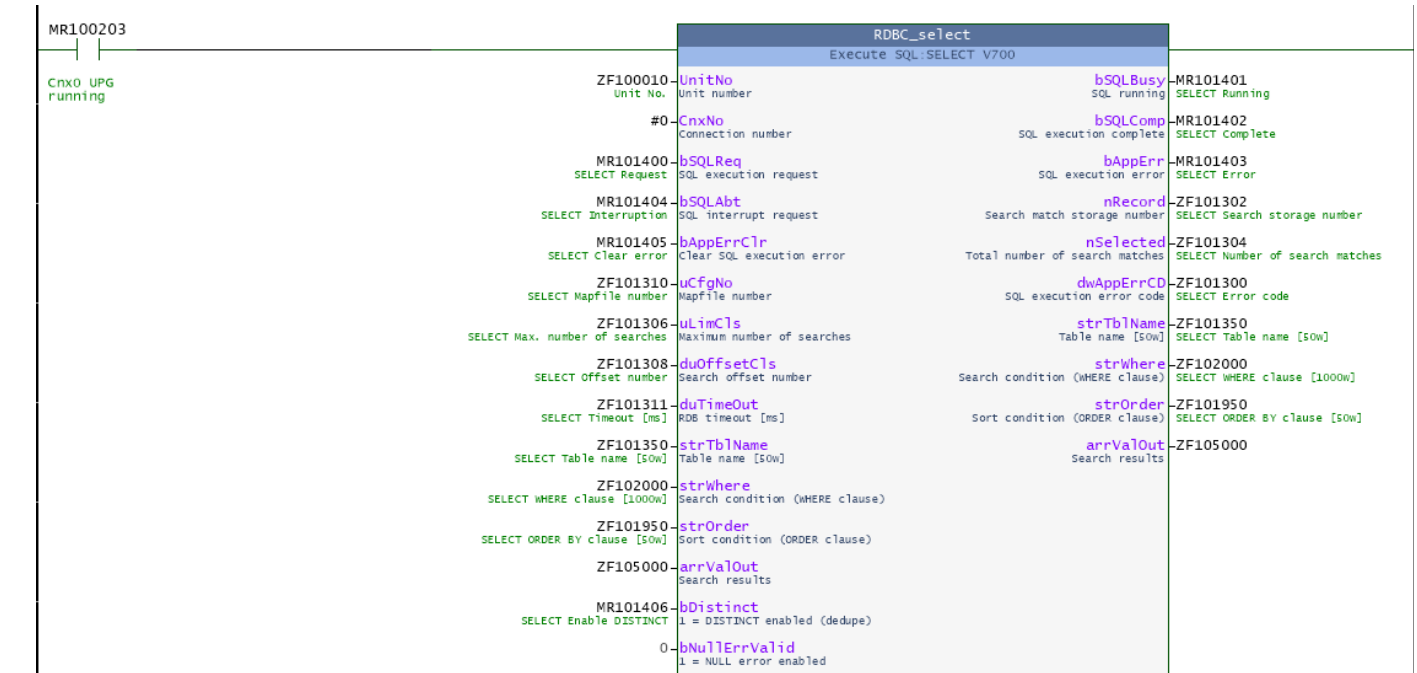
SQL Execution: SELECT

Retrieves records from the RDB matching the condition.

"KV-XCM02" can execute the SQL (SELECT) by specifying the DB Mapfile number (uCfgNo) and search conditions, and then retrieve specific data from the RDB according to the SELECT statement generated by "KV-XCM02".

The record data retrieved by the SELECT statement will be converted to the PLC data type by "KV-XCM02" based on the information in the specified DB Mapfile, and the data will be stored with "arrValOut" at the beginning.

Important To execute this FB, the DB Mapfile reading process (📖 "RDBC_cfgRelation" (page 5-7)) must be performed in advance.



- Point**
- The condition for execution is "UPG running" in RDBC_manage, so please turn ON the login request in RDBC_manage and execute the unit program in advance.
 - Place SQL execution FBs with different connection numbers to execute SQL for different connections.
 - RDBC_select stores the search results in one record at a time in a structure according to the DB Mapfile. When multiple records are retrieved, data is stored in record units in the arrValOut (search results). Records that cannot be stored up to the last column are not stored in arrValOut.
 - Example 1: When 20 records are retrieved with a data size of 10 words per record
Records are stored in the first 200 words of arrValOut, such as the 1st record in the 10th word from the beginning, the 2nd record in the 10th word from the 10th word, the 3rd record in the 10th word from the 30th word, and so on.
 - Example 2: If the data size per record is 101 words and 5000 records meet the search criteria
The default size of arrValOut is 15,000 words, so 148 records of data are stored in arrValOut. The 149th and subsequent record data that exceed the size of arrValOut, will not be stored.
The value stored are the first 14,948 (101 x 148) words.

Argument Description

The description of each argument is as follows

- **UnitNo: Unit number (IN/UINT)**

Specify the unit number of "KV-XCM02".

- **CnxNo: Connection number (IN/UINT)**

Specify the connection number to use (specified range: 0 to 3).
The RDB connection function can use up to 4 connections simultaneously.

- **bSQLReq: SQL execution request (IN/BOOL)**

Specifies the SQL execution request.
Hold until bSQLComp turns ON.

- **bSQLAbt: SQL interrupt request (IN/BOOL)**

Specifies the SQL interruption request.
Turns ON to interrupt SQL execution request. Hold until bSQLReq and bSQL-Busy are turned OFF.

- **bAppErrClr: SQL execution error clear (IN/BOOL)**

Specifies the SQL execution error clearing.
When bAppErrClr turns ON, dwAppErrCD stores 0 and bAppErr turns OFF.
Hold until bAppErr turns OFF.

- **uCfgNo: Mapfile number (IN/UINT)**

Specify the DB Mapfile ID to be used (specified range: 1-99).

- **uLimCls: Maximum number of searches (IN/UINT)**

Specifies the number of records to search.
If 0 is specified, no maximum limit is set. Automatically creates a LIMIT clause or syntax equivalent to a LIMIT clause with the specified number of records as the maximum.
□ "LIMIT Clause" (page A-8)

- **duOffsetCls: Search offset number (IN/UDINT)**

Specifies the number of records to skip without searching.
Automatically creates an OFFSET clause with the specified number of records as the offset value. If strOrder is not specified, it is processed as 0.
For example, to process from the 5th record, set 4 as 4 records are skipped.
□ "OFFSET Clause" (page A-9)

- **duTimeOut: RDB timeout [ms] (IN/UDINT)**

Specifies the SQL execution timeout period.
If bSQLComp does not turn on within the time specified by duTimeOut after bSQLReq turns ON, the program times out and exits with an error.
If a timeout occurs, SQL processing is aborted, and the user is logged out of the RDB.

- **bSQLBusy: SQL running (OUT/BOOL)**

Outputs SQL being executed.
Turns ON when a SQL running request is accepted by <KV-XCM02> Turns OFF when the SQL execution process is completed or interrupted.

- **bSQLComp: SQL execution complete (OUT/BOOL)**

Outputs the completion of SQL processing.
Turns ON when SQL processing ends normally/abnormally or SQL processing is interrupted.
Turns OFF when both bSQLReq and bSQLAbt are turned OFF.

- **bAppErr: SQL execution error (OUT/BOOL)**

Outputs SQL execution errors. Turns ON when an SQL execution error occurs.

- **nRecord: Search match storage number (OUT/DINT)**

Of the number of records that match the search criteria, only those that can be stored in arrValOut are counted.

- **nSelected: Total number of search matches (OUT/DINT)**

Of the number of records that matched the search criteria, those that could not be stored in arrValOut are also counted.
If the value of nSelected is greater than the value of nRecord, it means that there are search results that could not be stored.

- **dwAppErrCD: SQL execution error code (OUT/UDINT)**

Output SQL execution error code.
• 0: Normal end
• Value other than 0 (error code): Abnormal end

- **strTblName: Table name [50w] (IN-OUT/STRING [100])**

Specifies the table name and the schema name. Up to 100 bytes (including NULL) can be specified.
For example, to specify the Product table in the Sales schema, specify "Sales.Product".
□ "3 Quotation/String Identifier for Each RDB" (page A-6)

- **strWhere: Search condition (WHERE clause) (IN-OUT/STRING[2000])**

Specifies search conditions.
The specified string is automatically generated as a clause under "WHERE". Up to 2,000 bytes (including NULLs) can be specified.
To use double quotation marks as characters in the WHERE clause, use "".
□ "WHERE Clause" (page A-7)

- **strOrder: Sort condition (ORDER clause) (IN-OUT/STRING[100])**

Specifies the order in which search results are stored.
The specified string is automatically generated as a clause under "ORDER BY". Up to 100 bytes (including NULL) can be specified.
To use double quotation marks as characters in the ORDER BY clause, use "".
□ "ORDER BY Clause" (page A-8)

- **arrValOut: Search results (IN-OUT/UINT [MAX 100000WORD])**

Specifies where search results are to be stored. When specifying a device, DM, EM, FM, ZF, or W can be specified.
Data is stored as per the specified Mapfile.
Do not retrieve data larger than the size of the storage destination.

- **bDistinct: 1=DISTINCT enabled (dedupe) (IN/BOOL)**

Specifies whether duplicate records are eliminated.
If bDistinct is ON, duplicate records are eliminated.



A duplicate record is a record in which the values of all target columns match.

For example, the following table, "name", specify two columns "kanji" and "romaji" in the DB Mapfile and execute SELECT with the WHERE clause ""kanji" = 'Sato'".

Table name: name

id	kanji	romaji
1	佐藤	sato
2	鈴木	suzuki
3	佐藤	sato
4	佐藤	satoh

If bDistinct=OFF, the following three records are read.

kanji	romaji
佐藤	sato
佐藤	sato
佐藤	satoh

If bDistinct=ON, the following two records are read with duplicates eliminated.

kanji	romaji
佐藤	sato
佐藤	satoh

If bDistinct=ON, nRecord and nSelected will contain 2.



When bDistinct is turned ON, the columns specified in the ORDER BY clause must be columns included in the DB Mapfile. An error will occur if a column not included in the DB Mapfile is specified in the ORDER BY clause.

- **bNullErrValid: 1=NULL error enabled (IN/BOOL)**

Specifies whether or not an error is made when NULL data is SELECT.
The behavior, when NULL data is SELECT, is as follows.

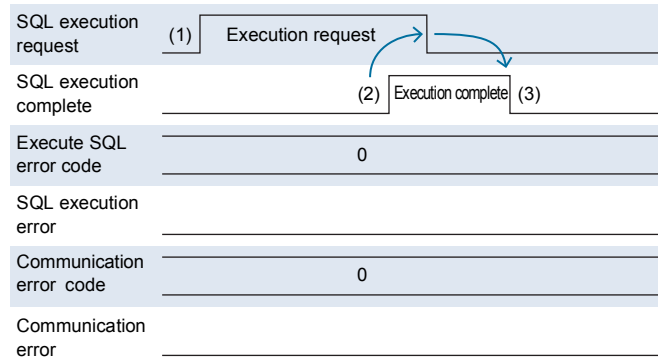
bNullErrValid	Error	Stored value
OFF	Does not occur.	• NULL: Data of character string type • NULL: Data type other than string
ON	Occurs (Error code 150000)	Nothing is stored.

Time Chart

This section describes the time chart for SQL execution FB.

Point The behavior of the time chart of SQL execution FB is common.

SQL Execution Request (normal end)

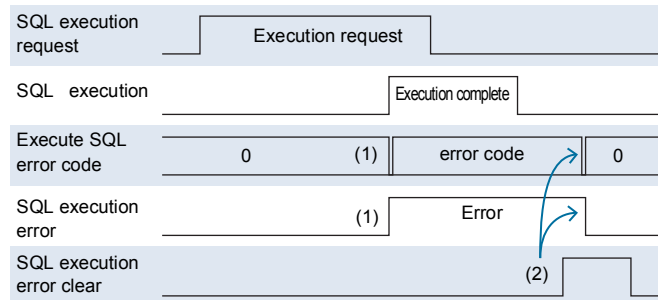


- (1) When the SQL execution request is turned ON, processing begins. Hold ON until completion.
- (2) When it is successfully completed, SQL execution complete is turned ON, and SQL execution error and communication error are turned OFF.
- (3) Confirm SQL execution complete and turn OFF the SQL execution request. When SQL execution request is turned OFF, SQL execution completion is turned OFF.

Point Please turn OFF the SQL execution error clear and communication error clear during the SQL execution request.

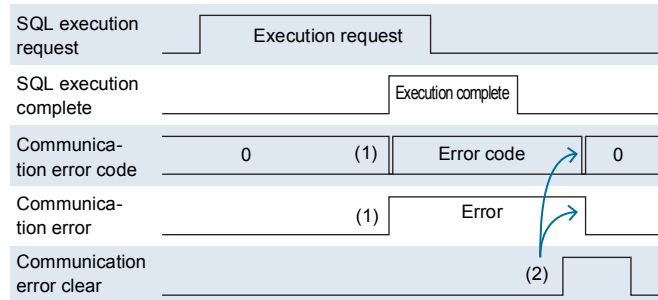
SQL Execution Request (abnormal end)

SQL execution error



- (1) If the SQL processing did not complete normally, the SQL execution error code is stored in the SQL execution error code, and the SQL execution error is turned ON.
- (2) When SQL execution error clear is turned ON, the SQL execution error code and SQL execution error are cleared.

Communication error



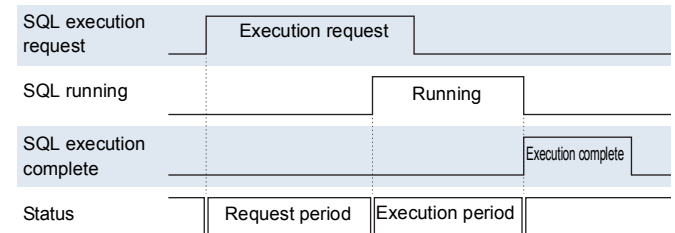
- (1) If a communication error occurs during SQL execution, the communication error code is stored in the Communication Error Code, and the communication error is turned ON.
- (2) When the communication error clear is turned ON, the communication error code and communication error are cleared.

Point When the SQL execution ends abnormally, SQL execution error or communication error turns ON.

SQL Execution Interrupt

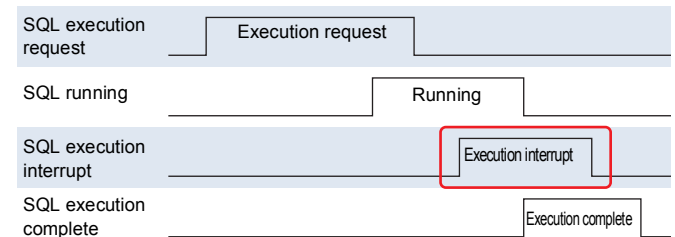
Each SQL execution FB is equipped with an abort function, allowing SQL execution to be interrupted. After the interrupt process is completed, the user is automatically logged out of the RDB.

Turn ON the bSQLReq of each SQL execution FB, then make a SQL execution request to <KV-XCM02>, and when the period until the request is accepted by <KV-XCM02> is the "request period," and the period during SQL execution after the request is accepted, is called the "execution period".



During the request period, the request is canceled and interrupted when the SQL execution request is turned OFF.

Turning ON the SQL interrupt request during the execution period will abort the process.



RDBC_insert

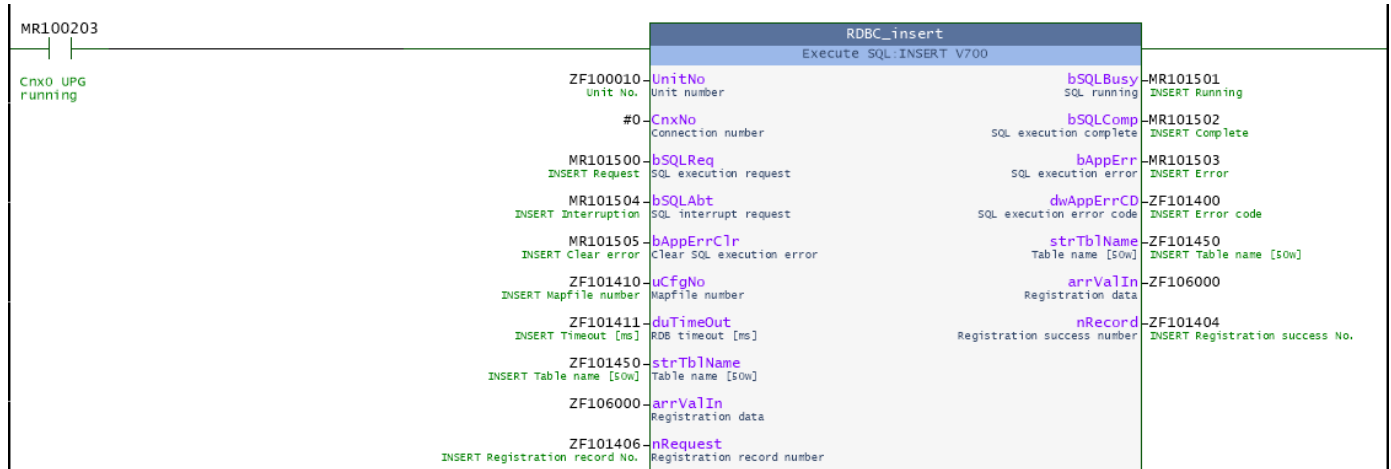
SQL Execution: INSERT

Register records matching the condition in the RDB.

Specify the number of the DB Mapfile (uCfgNo) and the destination of the data to be registered, and request "KV-XCM02" to execute SQL (INSERT) to register the data in RDB according to the INSERT statement generated by "KV-XCM02".

The registration results will be returned to FB from "KV-XCM02".

Important To execute this FB, the DB Mapfile reading process ( "RDBC_cfgRelation" (page 5-7)) must be performed in advance.



- Point**
- Data to be registered should be stored in advance according to the DB Mapfile.
 - The execution condition is "UPG running" in RDBC_manage, so please turn ON the login request of RDBC_manage and execute the unit program beforehand.
 - Place SQL execution FBs with different connection numbers to execute SQL for different connections.

Argument Description

The description of each argument is as follows

- **UnitNo: Unit number (IN/UINT)**
Specify the unit number of "KV-XCM02".
- **CnxNo: Connection number (IN/UINT)**
Specify the connection number to use (specified range: 0 to 3).
The RDB connection function can use up to 4 connections simultaneously.
- **bSQLReq: SQL execution request (IN/BOOL)**
Specifies the SQL execution request.
Hold until bSQLComp turns ON.
- **bSQLAbt: SQL interrupt request (IN/BOOL)**
Specifies the SQL interruption request.
Turns ON to interrupt SQL execution request.
Hold until bSQLReq and bSQLBusy are turned OFF.
- **bAppErrClr: SQL execution error clear (IN/BOOL)**
Specifies the SQL execution error clearing.
When bAppErrClr turns ON, dwAppErrCD stores 0 and bAppErr turns OFF.
Hold until bAppErr turns OFF.
- **uCfgNo: Mapfile number (IN/UINT)**
Specify the DB Mapfile ID to be used (specified range: 1-99).
- **duTimeOut: RDB timeout [ms] (IN/UDINT)**
Specifies the SQL execution timeout period.
If bSQLComp does not turn on within the time specified by duTimeOut after bSQLReq turns ON, the program times out and exits with an error.
If a timeout occurs, SQL processing is aborted, and the user is logged out of the RDB.
- **bSQLBusy: SQL running (OUT/BOOL)**
Outputs SQL being executed.
Turns ON when a SQL running request is accepted by <KV-XCM02>
Turns OFF when the SQL execution process is completed or interrupted.
- **bSQLComp: SQL execution complete (OUT/BOOL)**
Outputs the completion of SQL processing.
Turns ON when SQL processing ends normally/abnormally or SQL processing is interrupted.
Turns OFF when both bSQLReq and bSQLAbt are turned OFF.
- **bAppErr: SQL execution error (OUT/BOOL)**
Outputs SQL execution errors.
Turns ON when an SQL execution error occurs.
- **dwAppErrCD: SQL execution error code (OUT/UDINT)**
Output SQL execution error code.
Outputs 0 for the normal end.
In case of an abnormal end, a value other than 0 (error code) is stored.
- **strTblName: Table name [50w] (IN-OUT/STRING [100])**
Specifies the table name and the schema name. Up to 100 bytes (including NULL) can be specified.
For example, to specify the Product table in the Sales schema, specify "Sales.Product".
 "3 Quotation/String Identifier for Each RDB" (page A-6)

● **arrValln: Registration data (IN-OUT/UINT [MAX 100000WORD])**

Specifies where the registered data is to be stored. When specifying a device, DM, EM, FM, ZF, or W can be specified.
Store data as per the specified Mapfile, beginning with arrValln.

Reference

When registering multiple records, the registered data is stored in record units, with arrValln at the beginning.

Example: When registering 20 records with a data size of 10 words per record,
the 1st record is stored with the first 10 words from the beginning with arrValln, the 2nd record from the 10th word the next10 words, and so on.

Beginning 1st record
Beginning + 10 2nd record
 :
Beginning +190 20th record

● **nRequest: Registration record number (IN/UINT)**

Specifies the number of records to be registered.

● **nRecord: Registration success number (OUT/DINT)**

The number of records registered in the RDB is output.

Time Chart

The behavior of the time chart of the SQL execution FB is common.
📖 "Time Chart" (page 5-11)

RDBC_update

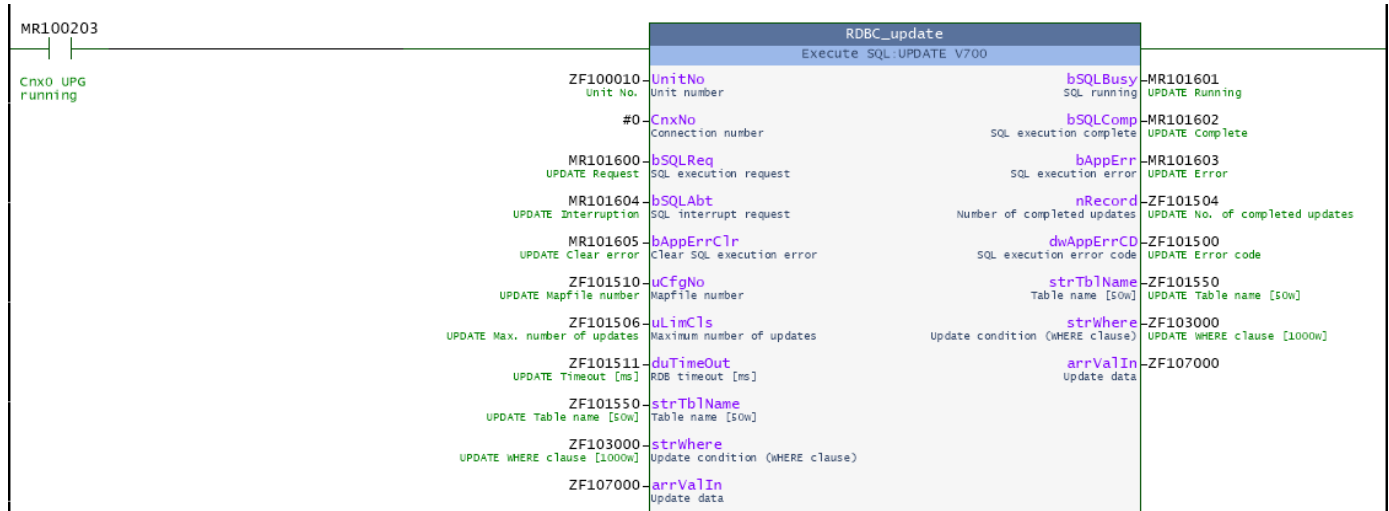
SQL Execution: UPDATE

Updates records on the RDB matching the condition.

Specify the number of DB Mapfile (uCfgNo) and the destination of the data to be updated and request "KV-XCM02" to execute SQL (UPDATE) to update the RDB data according to the UPDATE statement generated by "KV-XCM02".

The result of the update will be returned to FB from "KV-XCM02".

Important To execute this FB, a DB Mapfile read process (☞ "RDBC_cfgRelation" (page 5-7)) must be performed in advance.



- The data for updating must be stored in advance according to the DB Mapfile.
- The execution condition is "UPG running" in RDBC_manage, so please turn ON the login request of RDBC_manage and execute the unit program beforehand.
- Place SQL execution FBs with different connection numbers to execute SQL for different connections.

Argument Description

The description of each argument is as follows

Point The data to be updated must specify the conditions with strWhere or uLimCls. If no condition is specified, all records in the table will be updated.

- **UnitNo: Unit number (IN/UINT)**
Specify the unit number of "KV-XCM02".
- **CnxNo: Connection number (IN/UINT)**
Specify the connection number to use (specified range: 0 to 3).
The RDB connection function can use up to 4 connections simultaneously.
- **bSQLReq: SQL execution request (IN/BOOL)**
Specifies the SQL execution request.
Hold until bSQLComp turns ON.
- **bSQLAbt: SQL interrupt request (IN/BOOL)**
Specifies the SQL interruption request.
Turns ON to interrupt SQL execution request.
Hold until bSQLReq and bSQLBusy are turned OFF.
- **bAppErrClr: SQL execution error clear (IN/BOOL)**
Specifies the SQL execution error clearing.
When bAppErrClr turns ON, dwAppErrCD stores 0 and bAppErr turns OFF.
Hold until bAppErr turns OFF.
- **uCfgNo: Mapfile number (IN/UINT)**
Specify the DB Mapfile ID to be used (specified range: 1 to 99).
- **uLimCls: Maximum number of updates (IN/UINT)**
Specifies the number of records to be updated.
If 0 is specified, no maximum limit is set. Automatically creates a LIMIT clause or the syntax equivalent of a LIMIT clause.
☞ "LIMIT Clause" (page A-8)
- **duTimeOut: RDB timeout [ms] (IN/UDINT)**
Specifies the SQL execution timeout period.
If bSQLComp does not turn on within the time specified by duTimeOut after bSQLReq turns ON, the program times out and exits with an error.
If a timeout occurs, SQL processing is aborted, and the user is logged out of the RDB.
- **bSQLBusy: SQL running (OUT/BOOL)**
Outputs being executed.
Turns ON when a SQL running request is accepted by <KV-XCM02> Turns OFF when the SQL execution process is completed or interrupted.
- **bSQLComp: SQL execution complete (OUT/BOOL)**
Outputs the completion of SQL processing.
Turns ON when SQL processing ends normally/abnormally or SQL processing is interrupted.
Turns OFF when both bSQLReq and bSQLAbt are turned OFF.
- **bAppErr: SQL execution error (OUT/BOOL)**
Outputs SQL execution errors. Turns ON when an SQL execution error occurs.
- **nRecord: Number of completed updates (OUT/DINT)**
The number of records updated is output.

- **dwAppErrCD: SQL execution error code (OUT/UDINT)**

Output SQL execution error code.

- 0: Normal end
- Value other than 0 (error code): Abnormal end

- **strTblName: Table name [50w] (IN-OUT/STRING [100])**

Specifies the table name and the schema name. Up to 100 bytes (including NULL) can be specified.

For example, to specify the Product table in the Sales schema, specify "Sales.Product".

📖 "3 Quotation/String Identifier for Each RDB" (page A-6)

- **strWhere: Update condition (WHERE clause) (IN-OUT/STRING [2000])**

Specify the conditions for the target to be updated.

The specified string is automatically created as a clause under "WHERE". Up to 2,000 bytes (including NULLs) can be specified.

To use double quotation marks as characters in the WHERE clause, use "".

📖 "WHERE Clause" (page A-7)

- **arrValln: Update data (IN-OUT/UINT [MAX 100000WORD])**

Specifies where the updated data is to be stored. When specifying a device, DM, EM, FM, ZF, or W can be specified.

Starting with arrValln, store data as set on the Mapfile creation sheet.

Time Chart

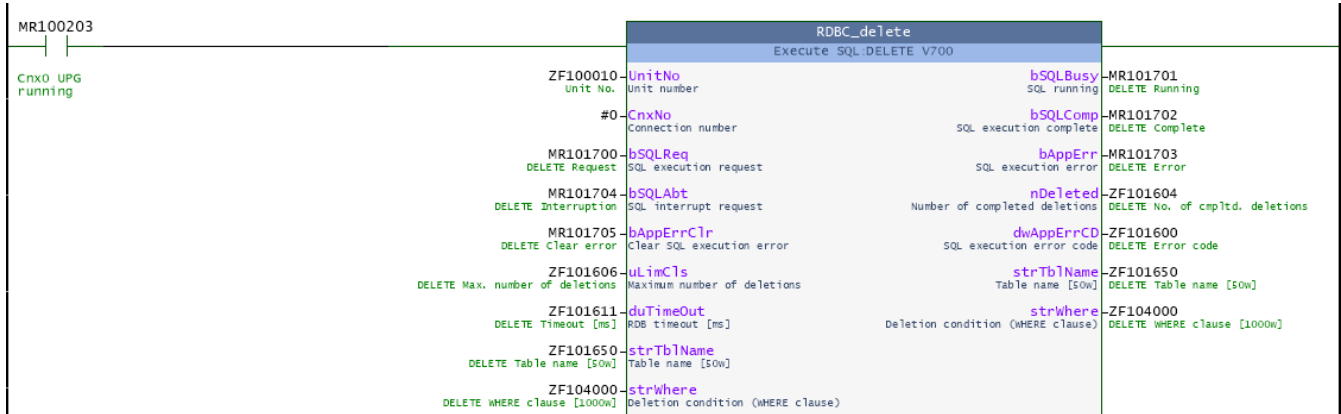
The behavior of the time chart of the SQL execution FB is common.

📖 "Time Chart" (page 5-11)

RDBC_delete

SQL Execution: DELETE

Deletes records from the RDB matching the condition.



- Point**
- The condition for execution is "UPG running" in RDBC_manage, so please turn ON the login request in RDBC_manage and execute the unit program beforehand.
 - Place SQL execution FBs with different connection numbers to execute SQL for different connections.

Argument Description

The description of each argument is as follows

- Point** The data to be deleted must specify the conditions with strWhere or uLimCls. If no condition is specified, all records in the table will be deleted.

- UnitNo: Unit number (IN/UINT)**
Specify the unit number of "KV-XCM02".
- CnxNo: Connection number (IN/UINT)**
Specify the connection number to use (specified range: 0 to 3).
The RDB connection function can use up to 4 connections simultaneously.
- bSQLReq: SQL execution request (IN/BOOL)**
Specifies the SQL execution request.
Hold until bSQLComp turns ON.
- bSQLAbt: SQL interrupt request (IN/BOOL)**
Specifies the SQL interruption request.
Turns ON to interrupt SQL execution request.
Hold until bSQLReq and bSQLBusy are turned OFF.
- bAppErrClr: SQL execution error clear (IN/BOOL)**
Specifies the SQL execution error clearing.
When bAppErrClr turns ON, dwAppErrCD stores 0 and bAppErr turns OFF.
Hold until bAppErr turns OFF.
- uLimCls: Maximum number of deletions (IN/UINT)**
Specifies the number of records to be deleted.
If 0 is specified, no maximum limit is set. Automatically creates a LIMIT clause or the syntax equivalent of a LIMIT clause.
□ "LIMIT Clause" (page A-8)
- duTimeOut: RDB timeout [ms] (IN/UDINT)**
Specifies the SQL execution timeout period.
If bSQLComp does not turn on within the time specified by duTimeOut after bSQLReq turns ON, the program times out and exits with an error.
If a timeout occurs, SQL processing is aborted, and the user is logged out of the RDB.
- bSQLBusy: SQL running (OUT/BOOL)**
Outputs SQL being executed.
Turns ON when a SQL running request is accepted by <KV-XCM02>
Turns OFF when the SQL execution process is completed or interrupted.
- bSQLComp: SQL execution complete (OUT/BOOL)**
Outputs the completion of SQL processing.
Turns ON when SQL processing ends normally/abnormally or SQL processing is interrupted.
Turns OFF when both bSQLReq and bSQLAbt are turned OFF.
- bAppErr: SQL execution error (OUT/BOOL)**
Outputs SQL execution errors. Turns ON when an SQL execution error occurs.
- nDeleted: Number of completed deletions (OUT/DINT)**
The number of records deleted is output.
- dwAppErrCD: SQL execution error code (OUT/UDINT)**
Output SQL execution error code.
 - 0: Normal end
 - Value other than 0 (error code): Abnormal end
- strTblName: Table name [50w] (IN-OUT/STRING [100])**
Specify the table name and the schema name. Up to 100 bytes (including NULL) can be specified.
For example, to specify the Product table in the Sales schema, specify "Sales.Product".
□ "3 Quotation/String Identifier for Each RDB" (page A-6)
- strWhere: Deletion condition (WHERE clause) (IN-OUT/STRING[2000])**
Specifies the condition of the target to be deleted.
The specified string is automatically created as a clause under "WHERE". Up to 2,000 bytes (including NULLs) can be specified.
To use double quotation marks as characters in the WHERE clause, use "".
□ "WHERE Clause" (page A-7)

Time Chart

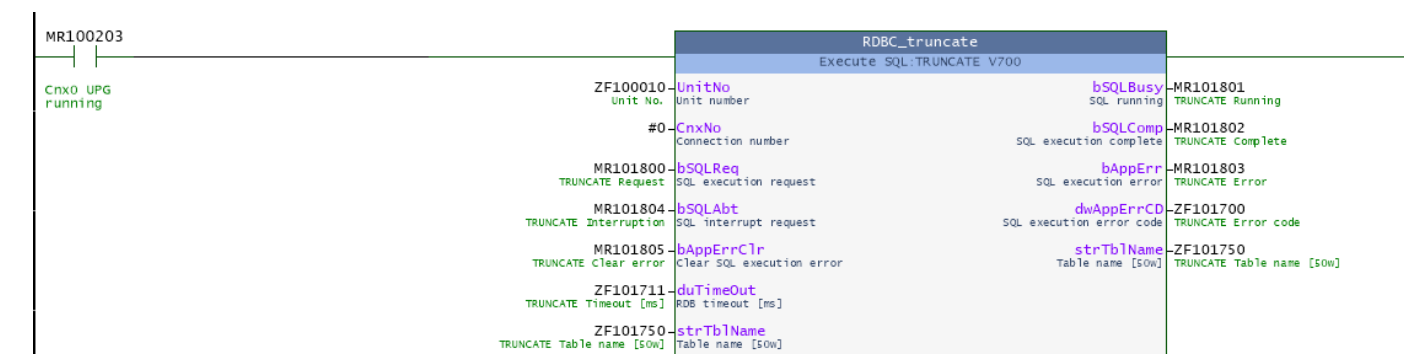
The behavior of the time chart of the SQL execution FB is common.

□ "Time Chart" (page 5-11)

RDBC_truncate

SQL Execution: TRUNCATE

Deletes all records from the specified RDB.



- Point**
- The execution condition is "UPG running" in RDBC_manage, so please turn ON the login request of RDBC_manage and execute the unit program beforehand.
 - Place SQL execution FBs with different connection numbers to execute SQL for different connections.

Argument Description

The description of each argument is as follows

- UnitNo: Unit number (IN/UINT)**
Specify the unit number of "KV-XCM02".
- CnxNo: Connection number (IN/UINT)**
Specify the connection number to use (specified range: 0 to 3).
The RDB connection function can use up to 4 connections simultaneously.
- bSQLReq: SQL execution request (IN/BOOL)**
Specifies the SQL execution request.
Hold until bSQLComp turns ON.
- bSQLAbt: SQL interrupt request (IN/BOOL)**
Specifies the SQL interruption request.
Turns ON to interrupt SQL execution request.
Hold until bSQLReq and bSQLBusy are turned OFF.
- bAppErrClr: SQL execution error clear (IN/BOOL)**
Specifies the SQL execution error clearing.
When bAppErrClr turns ON, dwAppErrCD stores 0 and bAppErr turns OFF.
Hold until bAppErr turns OFF.
- duTimeOut: RDB timeout [ms] (IN/UDINT)**
Specifies the SQL execution timeout period.
If bSQLComp does not turn on within the time specified by duTimeOut after bSQLReq turns ON, the program times out and exits with an error.
If a timeout occurs, SQL processing is aborted, and the user is logged out of the RDB.
- bSQLBusy: SQL running (OUT/BOOL)**
Outputs SQL being executed.
Turns ON when a SQL running request is accepted by <KV-XCM02>
Turns OFF when the SQL execution process is completed or interrupted.
- bSQLComp: SQL execution complete (OUT/BOOL)**
Outputs the completion of SQL processing.
Turns ON when SQL processing ends normally/abnormally or SQL processing is interrupted.
Turns OFF when both bSQLReq and bSQLAbt are turned OFF.
- bAppErr: SQL execution error (OUT/BOOL)**
Outputs SQL execution errors.
Turns ON when an SQL execution error occurs.
- dwAppErrCD: SQL execution error code (OUT/UDINT)**
Output SQL execution error code.
 - 0: Normal end
 - Value other than 0 (error code): Abnormal end
- strTblName: Table name [50w] (IN-OUT/STRING [100])**
Specify the table name and the schema name. Up to 100 bytes (including NULL) can be specified.
For example, to specify the Product table in the Sales schema, specify "Sales.Product".
📖 "3 Quotation/String Identifier for Each RDB" (page A-6)

Time Chart

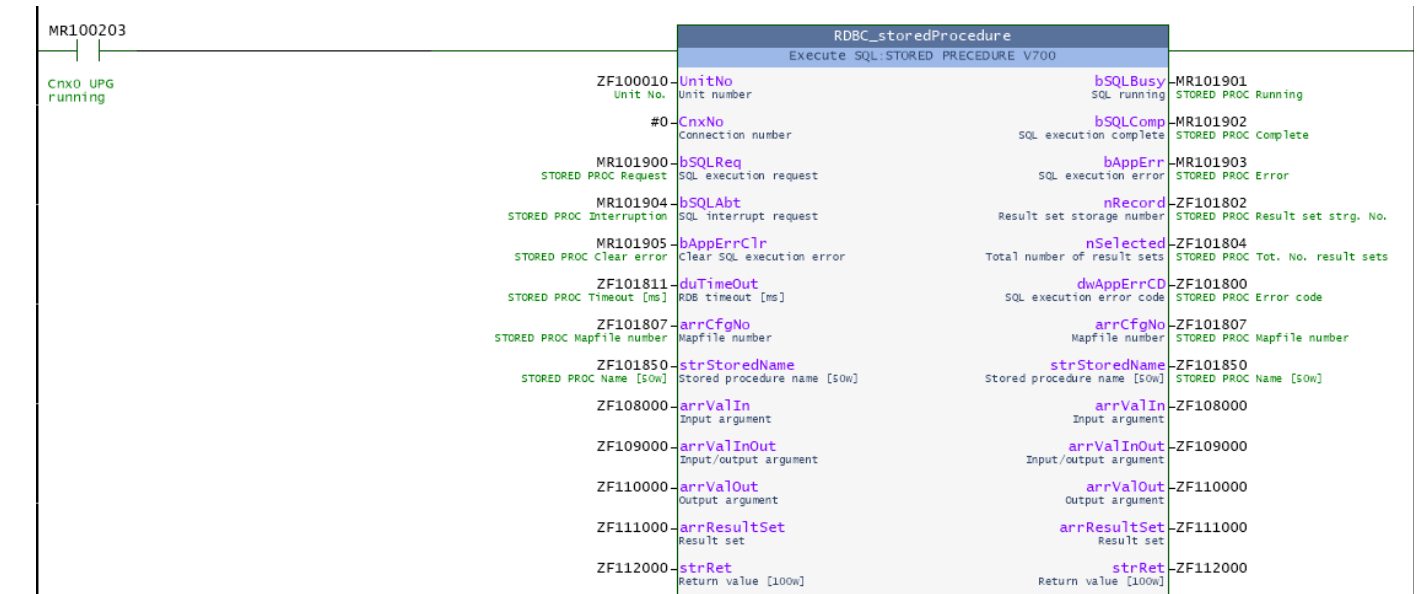
The behavior of the time chart of the SQL execution FB is common.

📖 "Time Chart" (page 5-11)

RDBC_storedProcedure	SQL Execution: Stored Procedure	Execute a stored procedure to obtain various completion bits, INOUT/OUT arguments, result sets, and return values.
-----------------------------	--	--

Set the stored procedure to be executed and the arguments to be passed, and request "KV-XCM02" to execute SQL, then "KV-XCM02" will pass the arguments to RDB to execute the stored procedure.

The results of the stored procedure execution (various complete bits, INOUT/OUT arguments, result set, and return value) will be stored in the CPU unit from "KV-XCM02" based on the information in the specified DB Mapfile.



- Point**
- The execution condition is "UPG running" in RDBC_manage, so please turn ON the login request of RDBC_manage and execute the unit program beforehand.
 - Place SQL execution FBs with different connection numbers to execute SQL for different connections.

Argument Description

The description of each argument is as follows

- UnitNo: Unit number (IN/UINT)**
Specify the unit number of "KV-XCM02".
- CnxNo: Connection number (IN/UINT)**
Specify the connection number to use (specified range: 0 to 3).
The RDB connection function can use up to 4 connections simultaneously.
- bSQLReq: SQL execution request (IN/BOOL)**
Specifies the SQL execution request.
Hold until bSQLComp turns ON.
- bSQLAbt: SQL interrupt request (IN/BOOL)**
Specifies the SQL interruption request.
Turns ON to interrupt SQL execution request.
Hold until bSQLReq and bSQLBusy are turned OFF.
- bAppErrClr: SQL execution error clear (IN/BOOL)**
Specifies the SQL execution error clearing.
When bAppErrClr turns ON, dwAppErrCD stores 0 and bAppErr turns OFF.
Hold until bAppErr turns OFF.
- duTimeOut: RDB timeout [ms] (IN/UDINT)**
Specifies the SQL execution timeout period.
If bSQLComp does not turn on within the time specified by duTimeOut after bSQLReq turns ON, the program times out and exits with an error.
If a timeout occurs, SQL processing is aborted, and the user is logged out of the RDB.
- bSQLBusy: SQL running (OUT/BOOL)**
Outputs SQL being executed.
Turns ON when a SQL running request is accepted by <KV-XCM02> Turns OFF when the SQL execution process is completed or interrupted.
- bSQLComp: SQL execution complete (OUT/BOOL)**
Outputs the completion of SQL processing.
Turns ON when SQL processing ends normally, abnormally, or when SQL processing is interrupted.
Turns OFF when both bSQLReq and bSQLAbt are turned OFF.
- bAppErr: SQL execution error (OUT/BOOL)**
Outputs SQL execution errors.
Turns ON when an SQL execution error occurs.
- nRecord: Result set storage number (OUT/DINT)**
Specifies where the number of records retrieved as a result set is to be stored.
Only records that can be stored in arrResultSet are counted.
- nSelected: Total number of result sets (OUT/DINT)**
Specifies where to store the total number of records retrieved as a result set.
Records that could not be stored in arrValOut are also counted.
If the value of nSelected is greater than the value of nRecord, it means that there are result sets that could not be stored.
- dwAppErrCD: SQL execution error code (OUT/UDINT)**
Output SQL execution error code.
 - 0: Normal end
 - Value other than 0 (error code): Abnormal end

● arrCfgNo: Mapfile number (IN-OUT/INT [4])

Specify the DB Mapfile ID to be used for each argument when executing the stored procedure (specified range: 1-99).

Specify arrCfgNo at the beginning as follows

Offset	Item
+0	DB Mapfile ID for input argument (arrValIn).
+1	DB Mapfile ID for input/output argument (arrValInOut)
+2	DB Mapfile ID for output argument (arrValOut)
+3	DB Mapfile ID of result set (arrResultSet)

Set -1 for arguments not used. The available arguments differ depending on the RDB type.

□ "Stored Procedure Execution Function" (page 4-3)

● strStoredName: Stored procedure name (IN-OUT/STRING [100])

Specify a stored procedure name and the schema name. Up to 100 bytes (including NULL) can be specified.

For example, to specify the procedure Proc for the Sales schema, specify "Sales.Proc".

□ "3 Quotation/String Identifier for Each RDB" (page A-6)

● arrValIn: Input argument (IN-OUT/UINT [MAX 100000WORD])

Specifies where to store the data to be passed to the stored procedure. Can specify as DM, EM, FM, ZF, or W.

Store data as per the specified Mapfile, beginning with arrValIn.

● arrValInOut: Input/output arguments (IN-OUT/UINT [MAX 100000WORD])

Specifies where to store data to be passed to the stored procedure and where to store data after completion of stored procedure execution. Can specify as DM, EM, FM, ZF, or W.

Store data as per the specified Mapfile, starting with arrValInOut.

Do not retrieve data larger than the size of the storage destination.

● arrValOut: Output argument (IN-OUT/UINT [MAX 100000WORD])

Specifies where to store data after completion of stored procedure execution. Can specify as DM, EM, FM, ZF, or W.

Data is output as per the specified Mapfile, starting with arrValOut.

Do not retrieve data larger than the size of the storage destination.

● arrResultSet: Result set (IN-OUT/UINT [MAX 100000WORD])

Specifies where the result set will be stored. Can specify as DM, EM, FM, ZF, or W.

Do not retrieve data larger than the size of the storage destination.

● strRet: Return value [100w] (IN-OUT/STRING [200])

Specifies where the return value is stored. Can specify as DM, EM, FM, ZF, or W.

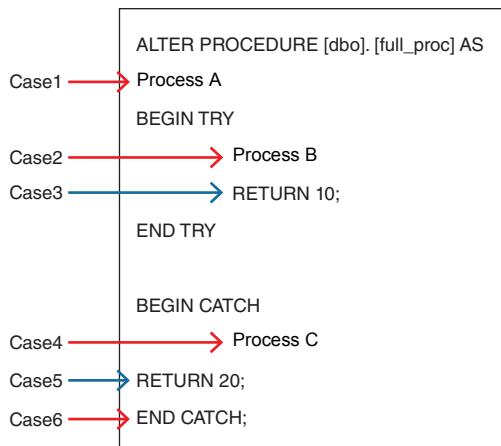
The data type is a character string, up to 200 bytes (including NULL).

Operation in the Event of an Error

This section describes the changes in the value of each argument of the FB when the execution of a stored procedure fails to complete successfully.

Please refer to the RDB specification for how to define stored procedures and how to read the following description examples.

The following description is based on the example when the following stored procedure is executed.



arrResultSet	Value is stored (if any).
--------------	---------------------------

Other remarks: To distinguish from Case 5, confirm the value of the return value, Out argument, etc.

● Case 4: When an exception ends within a CATCH block

Argument	Behavior
bComp	Turn ON
bAppErr	Turn ON
strRet	No change.
arrValInOut	No change.
arrValOut	No change.
arrResultSet	No change.

Other remarks: Confirm the error code and system log.

● Case1: Exception ended outside of TRY block

Argument	Behavior
bComp	Turn ON
bAppErr	Turn ON
strRet	No change.
arrValInOut	No change.
arrValOut	No change.
arrResultSet	No change.

Other remarks: Confirm the error code and system log.

● Case 2: When an exception ends in a TRY block

Move to processing of CATCH block (move to Case 4, Case 5, Case 6).

● Case 3: If the stored procedure ends normally

Argument	Behavior
bComp	Turn ON
bAppErr	Turn OFF
strRet	The result of RETURN (10 in the above example) is stored.
arrValInOut	Value is stored (if any).
arrValOut	Value is stored (if any).

● Case 5: Normal end within a CATCH block

Argument	Behavior
bComp	Turn ON
bAppErr	Turn OFF
strRet	The result of RETURN (20 in the above example) is stored.
arrValInOut	Value is stored (if any).
arrValOut	Value is stored (if any).
arrResultSet	Value is stored (if any).

Other remarks: To distinguish from Case 5, confirm the value of the return value, Out argument, etc.

● Case 6: Failed to finish within the timeout period

Argument	Behavior
bComp	Turn ON
bAppErr	Turn ON
strRet	No change.
arrValInOut	No change.
arrValOut	No change.
arrResultSet	No change.

Other remarks: Confirm the error code and system log.

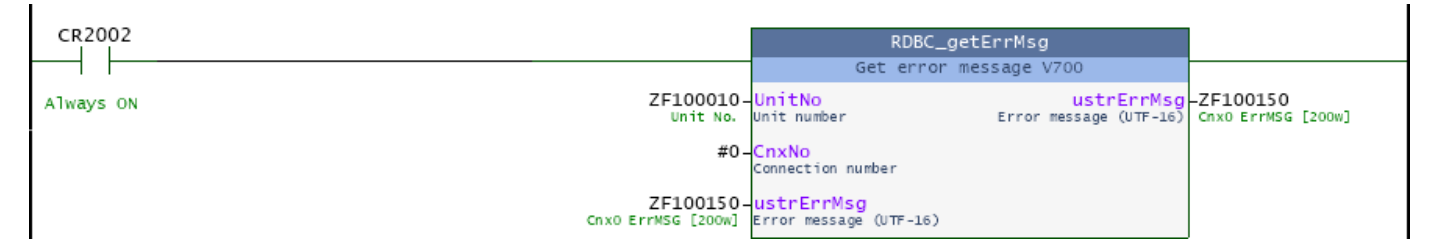
Time Chart

The behavior of the time chart of the SQL execution FB is common.

📖 "Time Chart" (page 5-11)

5-5 Auxiliary Function Block

RDBC_getErrMsg	Retrieve Error Message	Retrieve error messages from RDB.
-----------------------	-------------------------------	-----------------------------------



Argument Description

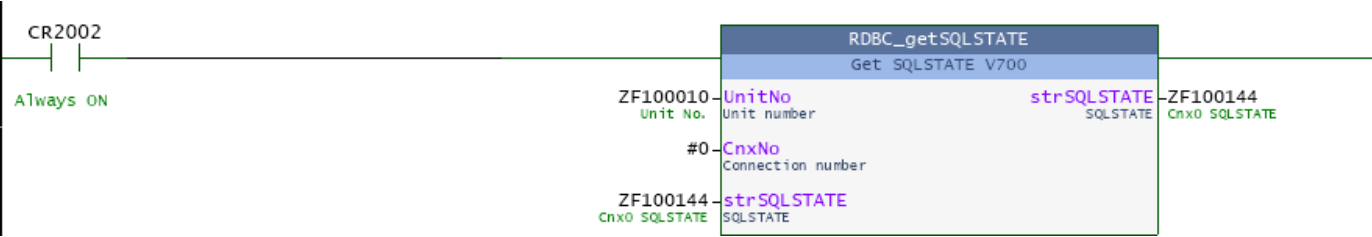
The description of each argument is as follows

- **UnitNo: Unit number (IN/UINT)**
Specify the unit number of "KV-XCM02."
- **CnxNo: Connection number (IN/UINT)**
Specify the connection number to use (specified range: 0 to 3).
The RDB connection function can use up to 4 connections simultaneously.
- **ustrErrMsg: Error message (UTF-16) (IN-OUT/UINT)**
Specifies where error messages are stored.
Retrieves the latest error messages generated in the RDB. Up to 200 words can be retrieved.
When the most recently executed SQL succeeds, the message of the last error message is also retrieved.
The character encoding is fixed to UTF-16.

RDBC_getSQLSTATE	Retrieve SQLSTATE	Retrieve SQLSTATE from RDB (PostgreSQL).
-------------------------	--------------------------	--

 **Point**

If an error occurs when using PostgreSQL, 99999 is always stored in the SQL execution error code of the SQL execution FB. Confirm the details of the error in SQLSTATE. For other RDBs, check the SQL execution error code.



Argument Description

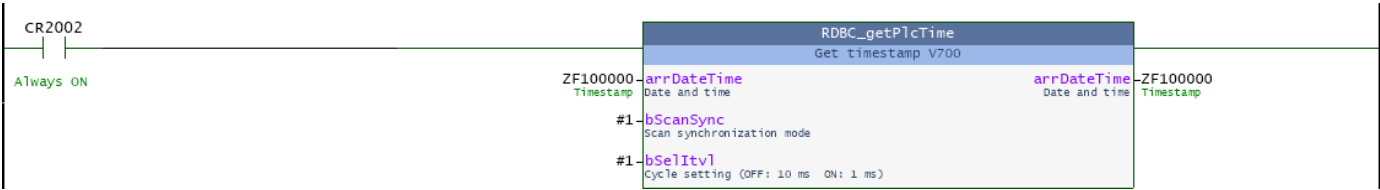
The description of each argument is as follows

- **UnitNo: Unit number (IN/UINT)**
Specify the unit number of "KV-XCM02".
- **CnxNo: Connection number (IN/UINT)**
Specify the connection number to use (specified range: 0 to 3).
The RDB connection function can use up to 4 connections simultaneously.
- **strSQLSTATE: SQLSTATE(IN-OUT/STRING)**
Specifies where SQLSTATE is to be stored.
Retrieve with 5 ASCII characters + NULL (3 words).
Used to confirm detailed error information when connecting to PostgreSQL.
Confirm the PostgreSQL specification for the contents of the information to be stored.

RDBC_getPlcTime	Retrieve Timestamp	Retrieves timestamps up to milliseconds.
------------------------	---------------------------	--

 Point

In the sample project, it is located in the periodic module "RDBC_timeStamp".



Argument Description

The description of each argument is as follows

- arrDateTime: Date and time (IN-OUT/UINT[8])**
Timestamps up to milliseconds are stored.
 - arrDateTime[0]: year
 - arrDateTime[1]: month
 - arrDateTime[2]: day
 - arrDateTime[3]: hour
 - arrDateTime[4]: minutes
 - arrDateTime[5] : seconds
 - arrDateTime[6]: milliseconds
 - arrDateTime[7]: Reserved (0 is stored)
- bScanSync: Scan synchronization mode (IN/BOOL)**
Specifies whether timestamps up to milliseconds are stored synchronously with scans.
 - 1: Synchronize with scan
 - 0: Not synchronized to scanIf not synchronized to scan, clock data at refresh timing is used and is affected by scan time.
- bSelItvl: Cycle setting (OFF:10ms, ON:1ms) (IN/BOOL)**
Specifies the unit of milliseconds.
 - 1: 1 ms unit
 - 0: 10 ms unit

Appendix

1 Error Code

The following error codes exist in the RDB connection function. The following is a list of error codes and their descriptions.

- 📖 "System Error Code" (page A-2)
- 📖 "Setting File Error Code" (page A-2)
- 📖 "Communication Error Code" (page A-3)
- 📖 "SQL Execution Error Code" (page A-4)

Error Types

Errors that occur in RDB are divided into the following types. The names in parentheses () are the name of the FB that detects the error.

Error type	Error signal	Error clear	Error factor	Login status
System Error (RDBC_manage)	bSysErr wSysErrCD	None* ¹	Unit program launch failed Failed to create system log folder	Ongoing
Communication error (RDBC_manage)	bTcpErr wTcpErrCD	bTcpErrClr	STMP communication error* ²	Disconnect
Connection error (RDBC_manage)	bCnxErr dwCnxErrCD	bCnxErrClr	Login failed* ³	Disconnect
SQL execution error (SQL execution FB)	bAppErr dwAppErrCD	bAppErrClr	SQL execution error* ⁴	* ⁵
Setting file error (RDBC_cfgRelation)	bMapErr wMapErrCD	bMapErrClr	Setting file read failed	Ongoing

*¹ When an error occurs due to unit program launch failure, the error signal is cleared upon successful unit program launch.

When an error occurs due to failure to create a system log folder, the error signal is cleared the next time the folder is successfully created.

*² When a TCP communication error occurs, bCnxErr and bAppErr may turn ON simultaneously. In such a case, execute error clear separately for each.

*³ For numbers below 100000, the error code output by RDB is output as is.

*⁴ The error occurs in each SQL execution FB. For numbers below 100000, the error code output by RDB is output as is.

*⁵ For numbers over 150000 disconnects the login status. For numbers below 149999 will continue the login status.

Each error occurs during the following events.

● System Error

Occurs at unit program launch. If an error occurs, confirm the unit program number setting and start RDBC_manage FB again.

● Setting file error

This error occurs when reading the DB Mapfile. If an error occurs, clear the error, confirm the file's existence, file path setting, and file contents, and reread the DB Mapfile.

● Communication error/connection error

Occurs during login/logout. If an error occurs, clear the error and confirm the RDB settings, communication path, and FB settings.

● Communication error/SQL execution error

Occurs during SQL execution.

Communication errors are common to RDBC_manage FB.

SQL execution errors occur in each SQL execution FB.

SQL execution errors should be detected under the OR condition of communication errors and SQL execution errors. If an error occurs, confirm the RDB settings, communication path, FB settings, and SQL syntax, and then clear the error.

System Error Code

The content list of the system error code (wSysErrCD). When normal, 0 is stored.

Failure to start the unit program, failure to create the system folder when launching RDBC_manage FB, etc.

Error code during unit program launch

Code	Description
0	Normal (no error)
1001	Could not be executed as the unit program with the specified program number does not exist. Please check if the unit program number specified in RDBC_manage FB is correct.
1002	No free slots are available for allocation. Stop execution of other unit programs.
1004	A unit program error is occurring. Confirm the unit program error factor before clearing the error.
1100	The unit may be in an abnormal state, such as program data corruption. Please transfer the project again.

Error code when system folder creation fails (only at startup)

Code	Description
-1	Do not have permission to operate the specified folder or file.
-4	The number of open files is exceeded, or file access is in conflict.
-6	File already exists.
-7	Path name is abnormal.
-9	Insufficient storage space or the number of files/folders that can be created.
-14	Storage is not mounted. (SD card is not inserted)
100	Other errors

Setting File Error Code

The content list of the wMapErrCD in RDBC_cfgRelation FB. When normal, 0 is stored.

Code	Description	Remarks
-1	Failed to read/write the file.	The following are possible causes. • The specified file is write-protected. • The memory card is write-protected. • There is something wrong with the file system of the memory card.
-7	The folder does not exist, or the path name is incorrect.	Confirm whether the cfg folder has been deleted or not. Confirm the folder path setting.
-14	Storage is not mounted	Insert SD card.
100	File format is incorrect.	Confirm the format of the DB Mapfile.
102	Column name is too long.	Confirm the format of the DB Mapfile.
103	Incorrect data type on PLC side.	Confirm the format of the DB Mapfile.
104	Excessive string size.	String type columns should be 1000 words or less.
105	The delimiter tab is missing.	Confirm the format of the DB Mapfile.
106	There is no delimiter line break.	A line break is required for each column setting, including the last line.
108	Total column limit in setting file exceeded	Column total must be less than or equal to 20000.
109	Total setting file exceeds the limit.	Mapfiles must be no more than 99.
110	No setting file.	There must be at least one Mapfile.

Communication Error Code

The content list of the communication error code (wTcpErrCD). wTcpErrCD stores 0 when normal. Confirm the communication path such as cable or Ethernet switch, IP address/port number of the communication destination, RDB account settings, RDB status, etc.

Code	Description	Remarks
-1062	The address is already in use.	Please check network settings.
-1064	Processing was performed on a disconnected connection.	Confirm network settings and communication path.
-1090	Timeout	Occurs when a connection cannot be established and the timeout period has expired or when a server response cannot be processed within the timeout period when executing SQL processing. Causes include server down, wrong IP address, wrong port number, or disconnection.
-1115	Unreachable	Occurs when there is no reachable route. Causes include network address and default gateway misconfiguration.
-1053 -1165 -1170	Disconnected by the other device.	Confirm the RDB settings (e.g., automatic logout time setting) and the port number of the server to which access is allowed. The network address may be incorrect. Please check network settings.
-1171	Closed by the other device.	This error occurs when the connection is disconnected from the RDB server. Confirm to see if abnormal processing is taking place or if the server has timed out or is in sleep mode.



Point

**When using Oracle Database, error codes 1 to 999 may be stored. 1 to 999 is the HTTP status code when an HTTP error occurs.
For details on the code, refer to the Oracle Database specifications.**

SQL Execution Error Code

List of SQL execution error codes (dwAppErrCD) and login/logout error codes (dwCnxErrCD) (0 to 199999).

Code	Description	Remarks
0	Normal end	
1 to 99999	The error code output by each RDB is stored.	For the contents of the code, refer to the specifications of each RDB.
100000 to 149999	Error code issued by the RDB connection function. The login status is continued even if an error occurs.	Refer to the table below.
150000 to	Error code issued by the RDB connection function. The user is automatically logged out when an error occurs.	Refer to the table below.



Point

Error code 99999 is always stored when using PostgreSQL. Confirm SQLSTATE for contents of the error.

☐ "RDBC_getSQLSTATE" (page 5-22)

Error for ongoing login status (100000 to 140000)

Code	Description	Remarks
100000	Incorrect RDB server type.	Confirm the arrCfgCnx setting.
100002	arrValln/Out is invalid.	Specify as DM, EM, FM, ZF, or W. Confirm that the device number does not exceed the available range.
100004	uCfgNo specification is invalid.	Confirm whether the DB Mapfile has been read.
100006	Unable to auto-login.	Please login manually with RDBC_manage.
100007	Character code specification is out of range.	Confirm the arrCfgCnx setting.
100008	Maximum record storage word size is incorrect.	Set the maximum record storage word size to a value larger than the size of one record.
100009	SQL execution failed.	SQL could not be executed because automatic login has failed. Please confirm login settings and network status.
100010	Timeout/interruption or request canceled before SQL execution started.	The following may have occurred before KV-XCM02 started processing • The request was canceled. • Interruption request turned ON • Timeout occurred. Increase the timeout period if necessary.
100101	Buffer save error.	Buffer save failed due to factors such as no SD card inserted, no buf folder, write protect, etc.
100102	Buffer is full.	Buffer save failed because the buffer file exceeds 1 GB.
100107	Mapfile settings that cannot be combined during stored procedure execution were specified.	Please check the specification of each RDB. ☐ "Stored Procedure Execution Function" (page 4-3)

Automatic logout error (150000 to)

Code	Description	Remarks
150000	The record contains NULL.	Searched for records containing NULL with bNullErrValid=ON in RDBC_select FB. Records containing NULL were retrieved during STORED PROCEDURE execution when connecting to My SQL. ☐ "RDBC_select" (page 5-9)
150001	Searched for unsupported data type.	Please check the data types that can be processed. Confirm if the data type of each column in the Mapfile matches the data type of each column in the RDB.
150003	Mismatch between RDB data type and PLC data type.	Please specify the correct Mapfile.
150005	Timeout/interruption or request canceled (after the launch of execution).	A request was canceled, an interruption request was turned ON, or a timeout occurred after KV-XCM02 started processing. Increase the timeout time if necessary. If a communication error has occurred, check the contents and remove the error content.
150020	Login was performed by a user with an unsupported authentication procedure.	Please check the authentication procedure of the login user. For PostgreSQL, md5 or password (plain text) is supported. For MySQL/MariaDB, Standard is supported.
150102	More than 1001 columns were detected during SELECT/STORED PROCEDURE execution in PostgreSQL.	When executing a PostgreSQL SELECT/STORED PROCEDURE, the column to be retrieved should be less than 1000.
150120	An invalid token was received in the execution of a stored procedure in the Microsoft SQL Server.	Received data may be too large. Received data should be less than 25000 bytes.

2 Restriction

This section gives a general description on the RDB protocol. This section describes the restrictions of the RDB connection function communication.

Common Restrictions of the RDB Connection Function

Using KV socket

Do not use the following KV sockets when using the following connection numbers.

Connection number (CnxNo)	KV socket numbers that cannot be used
0	0 to 6
1	6 to 13
2	13 to 15
3	None

When using SELECT

The maximum data size that can be stored in arrValOut in a single SELECT execution is the number of words set in nduLimWord in RDBC_manage FB (initial value: 15000 words = 30000 bytes).

If results exceeding this value are received, they are stored up to the capacity that can be stored in record delimiters. nduLimWord can be extended up to 100,000 words (200,000 bytes).

When using INSERT

The maximum data size of arrValIn that can be registered in a single INSERT execution is the number of words set in nduLimWord in RDBC_manage FB (initial value: 15000 words = 30,000 bytes).

If the number of registered rows exceeds this value, the maximum number of records that can be registered by record delimitation is registered. nduLimWord can be extended up to 100,000 words (200,000 bytes).

SELECT to View

SELECT execution to a View is possible. However, the Mapfile creation sheet does not support the ability to read the SQL file for table creation, including the View's CREATE statement, so please create the DB Mapfile manually.

Restrictions when using PostgreSQL

- Since RDB does not output error codes as numerical values but as strings (SQLSTATE), the cause of errors cannot be confirmed from connection errors and SQL execution errors. Use SQLSTATE instead. SQLSTATE can be retrieved with RDBC_getSQLSTATE.
□ "RDBC_getSQLSTATE" (page 5-22)
- The authentication procedure supports plain text passwords and MD5 encrypted passwords.
- Processing limit (LIMIT clause) cannot be set when executing UPDATE and DELETE. The input value of the processing limit is ignored.

Restrictions when using Microsoft SQL Server

- When a char/varchar/text column is retrieved, the character code string specified by the database collation is stored as is. The RDB connection function does not convert the character code.
When handling character strings retrieved by the KV series ladder program, it is recommended to use characters that can be displayed in ASCII code. Also, when using multibyte characters, the data type of the table should be nchar/nvarchar/text.

Restrictions when using MySQL and MariaDB

- Password authentication procedure is supported only for Standard.

Restrictions when using Oracle Database

- The following settings are required on the server side to use REST connections via HTTP.
 - Enable Oracle REST Data Service (ORDS) (Release 17.3 or later)
 - Disable HTTPS connections (configure to use OAuth2 in a non-HTTPS environment)
 - Enable REST-enabled SQL services
- HTTP authentication procedure supports Basic authentication.
- Schema names, table names, and column names should be written using only single-byte alphanumeric characters, underscores (_), dollar signs (\$), and sharp (#).
- In 12cR1 or earlier, column names must be 29 characters or less.
- Send SQL statements must be less than 100,000 bytes.
- The processing limit (LIMIT clause) cannot be set during UPDATE and DELETE execution. The input value of the processing limit is ignored.

3 Quotation/String Identifier for Each RDB

The methods to describe schema names, table names, column names, and strings differ depending on RDB type.

Schema names, table names, and column names should be enclosed in quotation identifiers, and other strings should be enclosed in string identifiers.

RDB	Quoted identifier	String identifier
Except MySQL and MariaDB	Double quotation ("abc")	Single quotation ('abc')
MySQL, MariaDB	Back quotation (`abc`)	

For example, to specify the table name `test_table` with the schema name `test_schema`, use the following

- For PostgreSQL, Microsoft SQL Server, and Oracle Database:
`"test_schema"."test_table"`
(For Microsoft SQL Server, use `[test_schema]. [test_table]` is also acceptable)
- For MySQL / MariaDB:
``test_schema`.`test_table``

Reference When using Microsoft SQL Server, brackets ([]) may be used instead of double quotation marks as quotation identifiers.

Note that in a ladder program, double quotation marks (") signify the beginning and end of a string. To append double quotation marks as characters, use two double quotation marks followed by ("") in the string.

If quoted identifiers are not appended to table or column names, they may not be recognized correctly as the RDB automatically converts all letters to lower or upper case. In this case, an error will occur stating that the specified table or column that does not exist.

4 SQL Command Description

This section describes the SQL commands that can be used with the RDB connection function. For details of each command, refer to the specifications of each RDB and SQL command.

INSERT

INSERT is a command to register a new record in the RDB.

(Example) SQL statement that registers 123 in column1 and 456 in column2 of the table called Sales.Product.

```
INSERT INTO "Sales"."Product" ("column1", "column2") VALUES (123, 456);
```

SELECT

SELECT is a command to retrieve records in the RDB.

(Example) SQL statement that sorts records with column1 value 123 from the table Sales.Product in ascending order of column2 and gets the values of column1 and column2.

```
SELECT "column1", "column2" FROM "Sales"."Product" WHERE "column1"=123
ORDER BY "column2";
```

UPDATE

UPDATE is a command to update records in the RDB.

(Example) SQL statement that updates column1 to 456 and column2 to 123 in a record whose column1 value is 123.

```
UPDATE "Sales"."Product" SET "column1"=456, "column2"=123 WHERE "column1"=123;
```

DELETE

DELETE is a command to delete a record in the RDB.

(Example) SQL statement to delete a record with a column1 value of 123 from the table Sales.Product

```
DELETE FROM "Sales"."Product" WHERE "column1"=123;
```

TRUNCATE

TRUNCATE is a command that deletes all records in a specified table. It tends to be faster than DELETE.

(Example) SQL statement to be generated that deletes all the tables called Sales.Product

```
TRUNCATE TABLE "Sales"."Product"
```

WHERE Clause

This command is used to narrow conditions when searching for, updating, or deleting records.

Specifies the "WHERE" and following clauses with strWhere. Table and column names specified in the WHERE clause should be enclosed in quoted identifiers, and strings should be enclosed in string constants.

□ "3 Quotation/String Identifier for Each RDB" (page A-6)

Double quotation marks should be used as characters "". Comparison operators (=, <, >, <=, >=, <>) and logical operators (AND, OR and LIKE operators (used for string comparison and partial match detection)) can be used. The following are typical examples of operators that can be used with <KV-XCM02>. Other operators supported by RDB can also be used.

Operator	Notation example	Description
=	"data" = 123	"data" equals 123
>	"data" > 123	data" is greater than 123
<	"data" < 123	data" is less than 123
>=	"data" >= 123	data" is greater than 123
<=	"data" <= 123	data" is less than 123
!=	"data" != 123	data" is not 123
<>	"data" <> 123	data" is not 123
AND	("data" >= 12) AND ("data" <= 34)	data" is greater than or equal to 12 and less than or equal to 34
OR	("data" < 12) OR ("data" > 34)	data" is less than 12 or greater than 34
NOT	NOT ("data" = 12)	"data" = not 12
BETWEEN a AND b	"data" BETWEEN 12 AND 34	data" is greater than or equal to 12 and less than or equal to 34
IN	"data" IN (12, 34, 56)	data" is 12, 34 or 56
LIKE	serial_no LIKE '%AB'	serial_no ends with the string AB
	serial_no LIKE 'AB%'	serial_no begins with the string AB
	serial_no LIKE '%AB%'	serial_no consists of the string AB
is NULL	"ColumnA" is NULL	ColumnA" is NULL
is not NULL	"ColumnA" is not NULL	"ColumnA" is not NULL

The use of the operator is explained below.

● The value (number) of a specific column is greater than or equal to a specified number

Condition: The value of "ColumnA" (16-bit signed integer) is 1234 or more.

```
strWhere = ""ColumnA"" >= 1234"
```

WHERE clause to be generated:

```
"ColumnA" >= 1234
```

Condition: The value of "ColumnA" (16-bit signed integer) is greater than or equal to wFilter

```
wFilter = 1234
```

```
strWhere = ""ColumnA"" >= " + STR(wFilter)
```

*1 wFilter is a variable of type INT. STR is a function that converts a number to a string.

WHERE clause to be generated:

```
"ColumnA" >= 1234
```

● The value (string) of a specific column matches the specified string

Condition: The value of "ColumnB" (string) matches 'ABC'.

```
strWhere = ""ColumnB"" = 'ABC'"
```

WHERE clause to be generated:

```
"ColumnB" = 'ABC'
```

Condition: The value of "ColumnB" (string) matches the string stored in strFilter.

```
strFilter = "ABC"
```

```
strWhere = ""ColumnB"" = " + strFilter + ""
```

*1 strFilter is a variable of type STRING.

WHERE clause to be generated:

```
"ColumnB" = 'ABC'
```

● The value (string) of a particular column begins with the specified string

Condition: The value of "ColumnB" (string) starts with 'ABC'.
 strWhere = ""ColumnB"" LIKE 'ABC%'

WHERE clause to be generated:
 "ColumnB" LIKE 'ABC%'



Wildcards that can be compared when using the LIKE operator:

- %(any string of any number of characters)
 Example: Specify records where ColumnB consists of "computer"
 strWhere = ""ColumnB"" LIKE '%computer%' "
- _(any one character)
 Example: Specify a record where ColumnB consists of "comput-erX" (X is an arbitrary character).
 strWhere = ""ColumnB"" LIKE 'computer_ ' "

● The value (timestamp) of a particular column is later than the specified timestamp

Condition: The value of "ColumnA" (timestamp) is after the timestamp specified by arrTimeStamp[0]-arrTimeStamp[7]

arrTimeStamp[0] = 2021 (year)
 arrTimeStamp[1] = 4 (month)
 arrTimeStamp[2] = 10 (day)
 arrTimeStamp[3] = 8 (hour)
 arrTimeStamp[4] = 31 (minutes)
 arrTimeStamp[5] = 10 (seconds)
 arrTimeStamp[6] = 123 (milliseconds)
 arrTimeStamp[7] = 0 (Reserved)

To generate a WHERE clause from the timestamp information, convert the timestamp to a string.

To convert to a string, use the following script functions

DASC: A function to convert a numeric value to a string

RIGHT: A function to cut out any number of characters from the right end of a string

If arrTimeStamp[0] = 2021, the number 2021 is converted to a string and two characters are cut out from the right end, resulting in the string "21".

RIGHT(DASC(arrTimeStamp[0], 2), 2)

Conversion Parameters

- 0: Zero suppress OFF, sign omitted OFF
- 1: Zero suppression ON, sign omitted OFF
- 2: Zero suppression OFF, sign omitted ON
- 3: Zero suppression ON, sign omitted ON

Cut out 2 characters from the right end of the string.

Change the timestamps stored in arrTimeStamp[0]-arrTimeStamp[7] to character strings.

```
strWhere=""ColumnA"">""+RIGHT(DASC(arrTimeStamp[0],2,4)+"-"+
RIGHT(DASC(arrTimeStamp[1],2,2)+"-"+
RIGHT(DASC(arrTimeStamp[2],2,2)+"-"+
RIGHT(DASC(arrTimeStamp[3],2,2)+"-"+
RIGHT(DASC(arrTimeStamp[4],2,2)+"-"+
RIGHT(DASC(arrTimeStamp[5],2,2)+"-"+
RIGHT(DASC(arrTimeStamp[6],2,3)+"-"
```



Timestamps must be enclosed in single quotes (') (treated like strings).

● Specific column is NULL

Condition: "ColumnA" is NULL.
 strWhere = ""ColumnA"" is NULL"

WHERE clause to be generated:
 "ColumnA" is NULL

● Specific column is not NULL

Condition: "ColumnA" is not NULL.
 strWhere = ""ColumnA"" is not NULL"

WHERE clause to be generated:
 "ColumnA" is not NULL

● Combine multiple conditions

Condition: "ColumnA is not NULL" and "ColumnB (16-bit signed integer) is greater than or equal to 1234 or "ColumnC" (string) begins with 'ABC'
 strWhere = ""ColumnA"" is not NULL AND ("ColumnB" >= 1234 OR ""ColumnC"" LIKE 'ABC%')"

WHERE clause to be generated:
 "ColumnA" is not NULL AND ("ColumnB" >= 1234 OR "ColumnC" LIKE 'ABC%')

● Subqueries

Complex conditions can be specified during SQL execution by using a SELECT statement within the WHERE clause.

● Update the value of the record with the largest value for a particular column (e.g., to specify the most recent record)

Update the value of "ColumnA" (16-bit signed integer) to the value of the record with the largest value.

strWhere = ""ColumnA"" = (SELECT MAX(""ColumnA"" FROM ""Sales"."Product"")"

SQL statement to be generated:
 UPDATE "Sales"."Product" SET "ColumnA" =<value>, "ColumnB" =<value>
 WHERE "ColumnA" = (SELECT MAX("ColumnA") FROM "Sales"."Product"))

ORDER BY Clause

This command specifies the order in which records are searched, updated, and deleted.

The clause below "ORDER BY" is specified by strOrder. The column name specified by strOrder must be enclosed in a quoted identifier.

For details, refer to the quotation/string identifier for each RDB.

To use double quotation marks as characters, use "".

If no LIMIT clause is added, assign 0 to uLimCls.

For ascending order: column name ASC

For descending order: column name DESC

. To specify more than one, separate them by commas.

● Sort id columns in ascending order

strOrder = ""id"" ASC"

ORDER BY clause to be generated:
 ORDER BY "id" ASC

● Sort id column in ascending order and name column in descending order for duplicates

strOrder = ""id" ASC, "name" DESC"

ORDER BY clause to be generated:
 ORDER BY "id" ASC, "name" DESC

LIMIT Clause

This command specifies the maximum number of records to be processed when retrieving, updating, or deleting records. The name of the clause may differ depending on the type of RDB, but in this FB, the LIMIT clause is unified.

● Retrieves 5 id and name columns of records where the id is greater than 10 in descending order

strWhere = ""id"" > 10"
 strOrder = ""id"" DESC"
 uLimCls = 5

SQL statement to be generated:
 SELECT TOP (5) "id", "name" FROM "Sales"."Product" WHERE "id" > 10
 ORDER BY "id" DESC

OFFSET Clause

This command is used to specify how many of the first records in the order specified in the ORDER BY clause should be skipped without processing when searching for records.

- **Sort the id and name columns of records where the id is greater than 10 in descending order by id column, and retrieve 5 records starting from the 20th record.**

```
strWhere = ""id" > 10"
```

```
strOrder = ""id" DESC"
```

```
uLimCls = 5
```

```
duOffsetCls = 19
```

SQL statement to be generated:

```
SELECT "id", "name" FROM "Sales"."Product" WHERE "id" > 10 ORDER BY  
"id" DESC OFFSET 19 ROWS FETCH NEXT 5 ROWS ONLY
```

5 List of Devices Used

The following is a list of devices used in the sample project and how to batch replace devices.

List of Devices Used

The following devices are used in the sample project. Please replace the device range before importing so that the device range does not overlap with the project being imported.



Point

The following devices are not used in the variable version of the sample project.

- Devices with contents that are "reserved"
- Device with variable name

MR list

Device number	Argument name	Description	
MR100000	gRDB_bWholeErrClr	Clear all errors	
MR100001	gRDB_stUnit.bTrigReadCfgFile	Mapfile reading trigger	
MR100002	gRDB_stUnit.bReadCfgErrClr	Clear Mapfile reading error	
MR100003	gRDB_stUnit.bReadCfgFile	Read Mapfile	
MR100004	gRDB_stUnit.bCompReadCfgFile	Mapfile read complete	
MR100005	gRDB_stUnit.bReadCfgErr	Mapfile reading error	
MR100100	gRDB_astCfg[0].stxCtrl.bReqLogin	Connection 0	Login request
MR100101	gRDB_astCfg[0].stxCtrl.bReqLogout		Logout request
MR100102	gRDB_astCfg[0].stxCtrl.bBufferSendEnbl		Enable buffer resend
MR100103	gRDB_astCfg[0].stxCtrl.bBufferClr		Manual buffer clear request
MR100104 to MR100112	gRDB_astCfg[0].stxCtrl.Member1		Reservation
MR100113	gRDB_astCfg[0].stxCtrl.bTcpErrClr		Clear communication error
MR100114	gRDB_astCfg[0].stxCtrl.bTcpErrClr		Clear connection error
MR100115			Reservation
MR100200	gRDB_astCfg[0].stxSts.bCompLogin		Login complete
MR100201	gRDB_astCfg[0].stxSts.bCompLogout		Logout complete
MR100202	gRDB_astCfg[0].stxSts.bConnBusy		Online
MR100203	gRDB_astCfg[0].stxSts.bUpgBusy		UPG running
MR100204	gRDB_astCfg[0].stxSts.bSysErr		System error
MR100205	gRDB_astCfg[0].stxSts.bTcpErr		Communication error
MR100206	gRDB_astCfg[0].stxSts.bCnxErr		Connection error
MR100207	gRDB_astCfg[0].stxSts.bReserved1		Reservation
MR100208	gRDB_astCfg[0].stxSts.bBufferExist		Buffer existence
MR100209	gRDB_astCfg[0].stxSts.bReserved2		Reservation
MR100210	gRDB_astCfg[0].stxSts.bBufferClrEnbl		Enable manual buffer clear
MR100211	gRDB_astCfg[0].stxSts.bBufferClr		Manual buffer clear cmplt.
MR100212 to MR100215	gRDB_astCfg[0].stxCtrl.abReserved3		Reservation
MR100300	gRDB_astCfg[0].bReqManualLogin	Manual login	
MR100301	gRDB_astCfg[0].bReqManualBufferClr	Clear buffer manually	
MR100308	gRDB_astCfg[0].bErr	Error	
MR100400 to MR100608	Connection 1 (variable should read "1" for the element number of gRDB_astCfg[0])		
MR100700 to MR100908	Connection 2 (variable should read "2" for the element number of gRDB_astCfg[0])		
MR101000 to MR101208	Connection 3 (variable should read "3" for the element number of gRDB_astCfg[0])		
MR101400	gRDB_astSel[0].bReq	SELECT Request	
MR101401	gRDB_astSel[0].bBusy	SELECT Running	
MR101402	gRDB_astSel[0].bComp	SELECT Complete	
MR101403	gRDB_astSel[0].bErr	SELECT Error	
MR101404	gRDB_astSel[0].bAbt	SELECT Interruption	
MR101405	gRDB_astSel[0].bErrClr	SELECT Clear error	
MR101406	gRDB_astSel[0].bDistinct	SELECT Enable DISTINCT	
MR101500	gRDB_astIns[0].bReq	INSERT Request	
MR101501	gRDB_astIns[0].bBusy	INSERT Running	
MR101502	gRDB_astIns[0].bComp	INSERT Complete	
MR101503	gRDB_astIns[0].bErr	INSERT Error	
MR101504	gRDB_astIns[0].bAbt	INSERT Interruption	

Device number	Argument name	Description
MR101505	gRDB_astIns[0].bErrClr	INSERT Clear error
MR101600	gRDB_astUpd[0].bReq	UPDATE Request
MR101601	gRDB_astUpd[0].bBusy	UPDATE Running
MR101602	gRDB_astUpd[0].bComp	UPDATE Complete
MR101603	gRDB_astUpd[0].bErr	UPDATE Error
MR101604	gRDB_astUpd[0].bAbt	UPDATE Interruption
MR101605	gRDB_astUpd[0].bErrClr	UPDATE Clear error
MR101700	gRDB_astDel[0].bReq	DELETE Request
MR101701	gRDB_astDel[0].bBusy	DELETE Running
MR101702	gRDB_astDel[0].bComp	DELETE Complete
MR101703	gRDB_astDel[0].bErr	DELETE Error
MR101704	gRDB_astDel[0].bAbt	DELETE Interruption
MR101705	gRDB_astDel[0].bErrClr	DELETE Clear error
MR101800	gRDB_astTrunc[0].bReq	TRUNCATE Request
MR101801	gRDB_astTrunc[0].bBusy	TRUNCATE Running
MR101802	gRDB_astTrunc[0].bComp	TRUNCATE Complete
MR101803	gRDB_astTrunc[0].bErr	TRUNCATE Error
MR101804	gRDB_astTrunc[0].bAbt	TRUNCATE Interruption
MR101805	gRDB_astTrunc[0].bErrClr	TRUNCATE Clear error
MR101900	gRDB_astStored[0].bReq	STORED PROC Request
MR101901	gRDB_astStored[0].bBusy	STORED PROC Running
MR101902	gRDB_astStored[0].bComp	STORED PROC Complete
MR101903	gRDB_astStored[0].bErr	STORED PROC Error
MR101904	gRDB_astStored[0].bAbt	STORED PROC Interruption
MR101905	gRDB_astStored[0].bErrClr	STORED PROC Clear error

ZF list

Device number	Variable name	Description		
ZF100000	gauTimeStamp[0]	Timestamp (year)		
ZF100001	gauTimeStamp[1]	Timestamp (month)		
ZF100002	gauTimeStamp[2]	Timestamp (day)		
ZF100003	gauTimeStamp[3]	Timestamp (hour)		
ZF100004	gauTimeStamp[4]	Timestamp (minute)		
ZF100005	gauTimeStamp[5]	Timestamp (second)		
ZF100006	gauTimeStamp[6]	Timestamp (millisecond)		
ZF100007	gauTimeStamp[7]	Reservation		
ZF100010	gRDB_stUnit.UnitNo	Unit No.		
ZF100011	gRDB_stUnit.UpgNo	Unit program No.		
ZF100012	gRDB_stUnit.uOptimize	Optimization options		
ZF100015	gRDB_stUnit.nReadFile	Number of files to read		
ZF100016	gRDB_stUnit.nReadCol	Number of columns to read		
ZF100017	gRDB_stUnit.wMapErrCD	Reading file error code		
ZF100020 to ZF100039	gRDB_stUnit.strPlcPath	Root folder path		
ZF100100	gRDB_astCfg[0].uRDB_Type	Connection 0	RDBMS type	
ZF100110	gRDB_astCfg[0].arrCfgCnx[0]		Self port number	
ZF100111	gRDB_astCfg[0].arrCfgCnx[1]		Conn. dest. IP address 1	
ZF100112	gRDB_astCfg[0].arrCfgCnx[2]		Conn. dest. IP address 2	
ZF100113	gRDB_astCfg[0].arrCfgCnx[3]		Conn. dest. IP address 3	
ZF100114	gRDB_astCfg[0].arrCfgCnx[4]		Conn. dest. IP address 4	
ZF100115	gRDB_astCfg[0].arrCfgCnx[5]		Conn. dest. port number	
ZF100116	gRDB_astCfg[0].arrCfgCnx[6]		Used Ethernet port desig.	
ZF100117	gRDB_astCfg[0].arrCfgCnx[7]		Reservation	
ZF100118	gRDB_astCfg[0].arrCfgCnx[8]		Reservation	
ZF100119	gRDB_astCfg[0].arrCfgCnx[9]		Reservation	
ZF100120	gRDB_astCfg[0].arrCfgCnx[10]		Connection timeout time[ms]	
ZF100121	gRDB_astCfg[0].arrCfgCnx[11]		Reservation	
ZF100122	gRDB_astCfg[0].arrCfgCnx[12]		Enable buffer resend fnct.	
ZF100123	gRDB_astCfg[0].arrCfgCnx[13]		Reservation	
ZF100124	gRDB_astCfg[0].arrCfgCnx[14]		Reservation	
ZF100125	gRDB_astCfg[0].arrCfgCnx[15]		Reservation	
ZF100126	gRDB_astCfg[0].arrCfgCnx[16]		Reservation	
ZF100127	gRDB_astCfg[0].arrCfgCnx[17]		Char. code of snd str. data	
ZF100128	gRDB_astCfg[0].arrCfgCnx[18]		Char. code of rcv str. data	
ZF100129	gRDB_astCfg[0].arrCfgCnx[19]		System log usage	
ZF100130	gRDB_astCfg[0].arrCfgCnx[20]		System log name options	
ZF100131	gRDB_astCfg[0].arrCfgCnx[21]		GMT offset	
ZF100132	gRDB_astCfg[0].arrCfgCnx[22]		Reservation	
ZF100133	gRDB_astCfg[0].arrCfgCnx[23]		Reservation	
ZF100140	gRDB_astCfg[0].iSysErrCD		System error code	
ZF100141	gRDB_astCfg[0].iTcpErrCD		TCP error code	
ZF100142 to ZF100143	gRDB_astCfg[0].duCnxErrCD		Connection error code	
ZF100144 to ZF100146	gRDB_astCfg[0].strSQLSTATE		SQLSTATE	
ZF100150 to ZF100349	gRDB_astCfg[0].utf16_ErrMsg		ErrMsg	
ZF100360 to ZF100377	gRDB_astSqlInfo[0].arrData[0] to gRDB- B_astSqlInfo[0].arrData[17]		Reservation	
ZF100378 to ZF100379	gRDB_astSqlInfo[0].arrData[0] to gRDB- B_astSqlInfo[0].arrData[19]		Buffer accumulation SQL No.	
ZF100380 to ZF100381	gRDB_astSqlInfo[0].arrData[0] to gRDB- B_astSqlInfo[0].arrData[21]		Buffer file reading offset	
ZF100382	gRDB_astSqlInfo[0].arrData[22]		Buffer usage rate [0.01%]	
ZF100383	gRDB_astSqlInfo[0].arrData[23]		Reservation	
ZF100400 to ZF100683	Connection 1 (variable should read "1" for the element number of gRDB_astSqlInfo[0])			
ZF100700 to ZF100983	Connection 2 (variable should read "2" for the element number of gRDB_astSqlInfo[0])			
ZF101000 to ZF101283	Connection 3 (variable should read "3" for the element number of gRDB_astSqlInfo[0])			
ZF101300 to ZF101301	gRDB_astSel[0].dwErrCD	SELECT Error code		
ZF101302 to ZF101303	gRDB_astSel[0].nRecord	SELECT Search storage number		

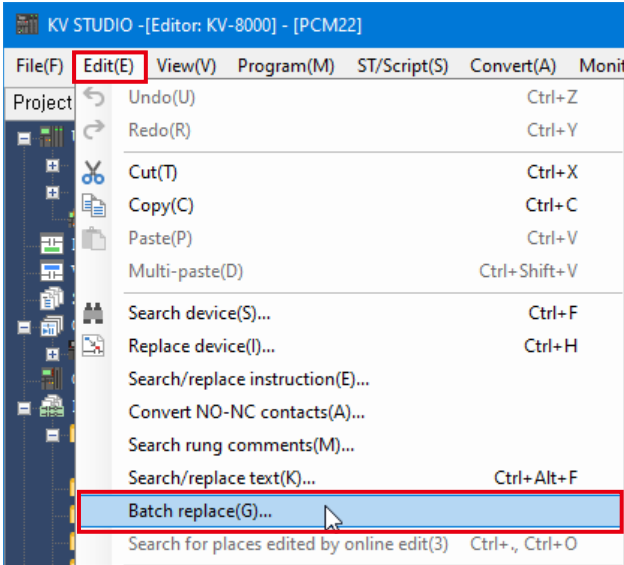
Device number	Variable name	Description
ZF101304 to ZF101305	gRDB_astSel[0].nSelected	SELECT Number of search matches
ZF101306	gRDB_astSel[0].uLimCls	SELECT Max. number of searches
ZF101307	gRDB_astSel[0].uLimCls	SELECT Reservation
ZF101308 to ZF101309	gRDB_astSel[0].duOffsetCls	SELECT Offset number
ZF101310	gRDB_astSel[0].uCfgNo	SELECT Mapfile number
ZF101311 to ZF101312	gRDB_astSel[0].duTimeOut	SELECT Timeout [ms]
ZF101350 to ZF101399	gRDB_astSel[0].strTblName	SELECT Table name
ZF101400 to ZF101401	gRDB_astIns[0].dwErrCD	INSERT Error code
ZF101402 to ZF101403		INSERT Reservation
ZF101404 to ZF101405	gRDB_astIns[0].nRecord	INSERT Registration success No.
ZF101406	gRDB_astIns[0].nRequest	INSERT Registration record No.
ZF101407 to ZF101409		INSERT Reservation
ZF101410	gRDB_astIns[0].uCfgNo	INSERT Mapfile number
ZF101411 to ZF101412	gRDB_astIns[0].duTimeOut	INSERT Timeout [ms]
ZF101450 to ZF101499	gRDB_astIns[0].strTblName	INSERT Table name
ZF101500 to ZF101501	gRDB_astUpd[0].dwErrCD	UPDATE Error code
ZF101502 to ZF101503		UPDATE Reservation
ZF101504 to ZF101505	gRDB_astUpd[0].ndiRecord	UPDATE No. of completed updates
ZF101506	gRDB_astSel[0].uLimCls	UPDATE Max. number of updates
ZF101507 to ZF101509		UPDATE Reservation
ZF101510	gRDB_astUpd[0].uCfgNo	UPDATE Mapfile number
ZF101511 to ZF101512	gRDB_astUpd[0].duTimeOut	UPDATE Timeout [ms]
ZF101550 to ZF101599	gRDB_astUpd[0].strTblName	UPDATE Table name
ZF101600 to ZF101601	gRDB_astDel[0].dwErrCD	DELETE Error code
ZF101602 to ZF101603		DELETE Reservation
ZF101604 to ZF101605	gRDB_astDel[0].ndiDeleted	DELETE No. of cmplt'd. deletions
ZF101606	gRDB_astDel[0].uLimCls	DELETE Max. number of deletions
ZF101607 to ZF101610		DELETE Reservation
ZF101611 to ZF101612	gRDB_astDel[0].duTimeOut	DELETE Timeout [ms]
ZF101650 to ZF101699	gRDB_astDel[0].strTblName	DELETE Table name
ZF101700 to ZF101701	gRDB_astTrunc[0].dwErrCD	TRUNCATE Error code
ZF101702 to ZF101710		TRUNCATE Reservation
ZF101711 to ZF101712	gRDB_astTrunc[0].duTimeOut	TRUNCATE Timeout [ms]
ZF101750 to ZF101799	gRDB_astTrunc[0].strTblName	TRUNCATE Table name
ZF101800 to ZF101801	gRDB_astStored[0].dwErrCD	STORED PROC Error code
ZF101802 to ZF101803	gRDB_astStored[0].nRecord	STORED PROC Result set strg. No.
ZF101804 to ZF101805	gRDB_astStored[0].nSelected	STORED PROC Tot. No. result sets
ZF101806		STORED PROC Reservation
ZF101807	gRDB_astStored[0].aiCfgNo[0]	STORED PROC Mapfile number
ZF101808	gRDB_astStored[0].aiCfgNo[1]	STORED PROC Mapfile number
ZF101809	gRDB_astStored[0].aiCfgNo[2]	STORED PROC Mapfile number
ZF101810	gRDB_astStored[0].aiCfgNo[3]	STORED PROC Mapfile number
ZF101811 to ZF101812	gRDB_astStored[0].duTimeOut	STORED PROC Timeout [ms]
ZF101850 to ZF101899	gRDB_astStored[0].strStoredName	STORED PROC Name
ZF101950 to ZF101999	gRDB_astSel[0].strOrder	SELECT ORDER BY clause
ZF102000 to ZF102999	gRDB_astSel[0].strWhere	SELECT WHERE clause
ZF103000 to ZF103999	gRDB_astUpd[0].strWhere	UPDATE WHERE clause
ZF104000 to ZF104999	gRDB_astDel[0].strWhere	DELETE WHERE clause

Batch Replacement Devices

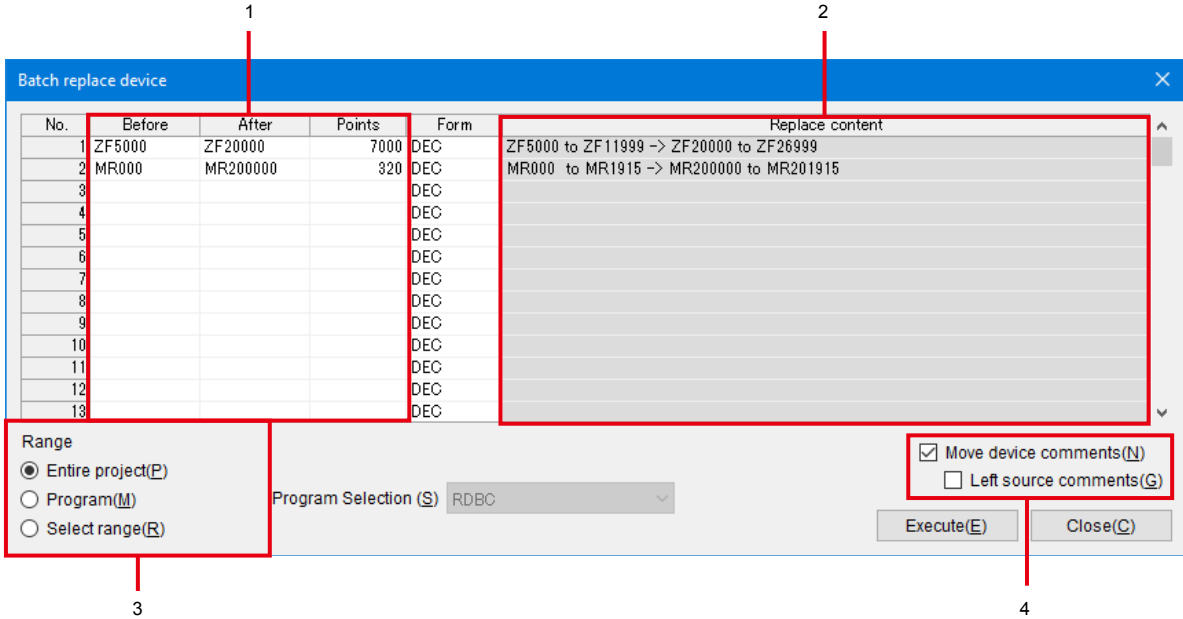
This section explains how to replace devices.

1 Select [Edit (E)] - [Replace All (G)] from the KV STUDIO menu.

The [Replace All Devices] dialog box will appear.



2 Perform batch replacement from the [Replace All Devices] dialog box.



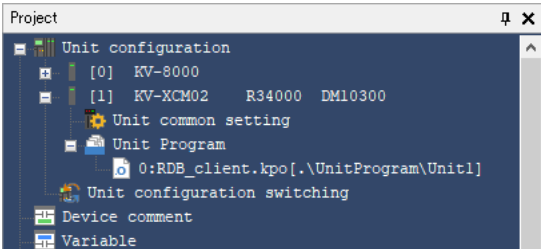
No.	Description
1	Specify the device number before and after replacement and the number of device points to be replaced.
2	The replacement range based on the settings will be displayed.
3	Specify the range of programs to be replaced. Check the "Entire Project" to replace all modules.
4	If "Move Device Comments Also" is checked, device comments can also be copied to the designated device. If there is a check in the "Keep comments from source" checkbox, device comments will remain on the device before replacement.

3 Click [Execute].

The device will be replaced according to the settings.

6 Troubleshooting

Relay Does Not Turn ON During UPG Execution

Cause	Measures
No SD card is inserted	If no SD card is inserted, the unit program cannot be launched, and the relay does not turn ON during UPG execution. Please insert an SD card in the CPU Unit.
File has not been created.	Confirm if the necessary files are stored in the "cfg" folder directly under the root folder specified in the initialization module or if it can be created.
An unsupported SD card is inserted.	An SD card that is not supported by the CPU unit has been inserted. The KV-8000 series and KV-7000 series can only use SD cards and SDHC cards.
Duplicate connection number	The same FB is being executed for the same connection number. When using multiple connections, make sure that the connection numbers are within the range of 0 to 3 and they do not overlap.
Unit program does not exist.	The RDB connection function uses the KV-XCM02 unit program to create and issue SQL. 

Relay Does Not Turn ON During Online

Cause	Measures
Not connected to the PC which the RDB is installed	There is a possibility that the link is not working properly. From KV STUDIO's Communication Test dialog, issue a ping command to the PC which the RDB is installed to see if the link is working.
Incorrect username, password, or database name.	Confirm that the username, password, and database name settings in RDBC_accountCfg match the RDB settings.
Blocked by security software	If the ping command passes in the communication test, it may be blocked by security software. Contact system administrator.
Port is not open	Open the port from the firewall setting. Contact system administrator.
Connection settings on the RDB side are incorrect.	Check the connection settings on the RDB side.
The RDB to be connected is Oracle Database.	When connecting to Oracle Database, the relay does not turn ON during login. Since the login complete relay turns ON for one scan, please create a ladder program that SET any bit device at the rising edge of the login complete relay.

Revision History

[illegible]

WARRANTIES AND DISCLAIMERS

- (1) KEYENCE warrants the Products to be free of defects in materials and workmanship for a period of one (1) year from the date of shipment. If any models or samples were shown to Buyer, such models or samples were used merely to illustrate the general type and quality of the Products and not to represent that the Products would necessarily conform to said models or samples. Any Products found to be defective must be shipped to KEYENCE with all shipping costs paid by Buyer or offered to KEYENCE for inspection and examination. Upon examination by KEYENCE, KEYENCE, at its sole option, will refund the purchase price of, or repair or replace at no charge any Products found to be defective. This warranty does not apply to any defects resulting from any action of Buyer, including but not limited to improper installation, improper interfacing, improper repair, unauthorized modification, misapplication and mishandling, such as exposure to excessive current, heat, coldness, moisture, vibration or outdoors air. Components which wear are not warranted.
- (2) KEYENCE is pleased to offer suggestions on the use of its various Products. They are only suggestions, and it is Buyer's responsibility to ascertain the fitness of the Products for Buyer's intended use. KEYENCE will not be responsible for any damages that may result from the use of the Products.
- (3) The Products and any samples ("Products/Samples") supplied to Buyer are not to be used internally in humans, for human transportation, as safety devices or fail-safe systems, unless their written specifications state otherwise. Should any Products/Samples be used in such a manner or misused in any way, KEYENCE assumes no responsibility, and additionally Buyer will indemnify KEYENCE and hold KEYENCE harmless from any liability or damage whatsoever arising out of any misuse of the Products/Samples.
- (4) **OTHER THAN AS STATED HEREIN, THE PRODUCTS/SAMPLES ARE PROVIDED WITH NO OTHER WARRANTIES WHATSOEVER. ALL EXPRESS, IMPLIED, AND STATUTORY WARRANTIES, INCLUDING, WITHOUT LIMITATION, THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF PROPRIETARY RIGHTS, ARE EXPRESSLY DISCLAIMED. IN NO EVENT SHALL KEYENCE AND ITS AFFILIATED ENTITIES BE LIABLE TO ANY PERSON OR ENTITY FOR ANY DIRECT, INDIRECT, INCIDENTAL, PUNITIVE, SPECIAL OR CONSEQUENTIAL DAMAGES (INCLUDING, WITHOUT LIMITATION, ANY DAMAGES RESULTING FROM LOSS OF USE, BUSINESS INTERRUPTION, LOSS OF INFORMATION, LOSS OR INACCURACY OF DATA, LOSS OF PROFITS, LOSS OF SAVINGS, THE COST OF PROCUREMENT OF SUBSTITUTED GOODS, SERVICES OR TECHNOLOGIES, OR FOR ANY MATTER ARISING OUT OF OR IN CONNECTION WITH THE USE OR INABILITY TO USE THE PRODUCTS, EVEN IF KEYENCE OR ONE OF ITS AFFILIATED ENTITIES WAS ADVISED OF A POSSIBLE THIRD PARTY'S CLAIM FOR DAMAGES OR ANY OTHER CLAIM AGAINST BUYER.** In some jurisdictions, some of the foregoing warranty disclaimers or damage limitations may not apply.

BUYER'S TRANSFER OBLIGATIONS:

If the Products/Samples purchased by Buyer are to be resold or delivered to a third party, Buyer must provide such third party with a copy of this document, all specifications, manuals, catalogs, leaflets and written information provided to Buyer pertaining to the Products/Samples.

Specifications are subject to change without notice.

KEYENCE CORPORATION

1-3-14, Higashi-Nakajima, Higashi-Yodogawa-ku, Osaka, 533-8555, Japan PHONE: +81-6-6379-2211

www.keyence.com/glb

AUSTRIA

Phone: +43 (0)2236 378266 0

BELGIUM

Phone: +32 (0)15 281 222

BRAZIL

Phone: +55-11-3045-4011

CANADA

Phone: +1-905-366-7655

CHINA

Phone: +86-21-3357-1001

CZECH REPUBLIC

Phone: +420 220 184 700

FRANCE

Phone: +33 1 56 37 78 00

GERMANY

Phone: +49-6102-3689-0

HONG KONG

Phone: +852-3104-1010

HUNGARY

Phone: +36 1 802 7360

INDIA

Phone: +91-44-4963-0900

INDONESIA

Phone: +62-21-2966-0120

ITALY

Phone: +39-02-6688220

KOREA

Phone: +82-31-789-4300

MALAYSIA

Phone: +60-3-7883-2211

MEXICO

Phone: +52-55-8850-0100

NETHERLANDS

Phone: +31 (0)40 206 6100

PHILIPPINES

Phone: +63-(0)2-8981-5000

POLAND

Phone: +48 71 368 61 60

ROMANIA

Phone: +40 (0)269 232 808

SINGAPORE

Phone: +65-6392-1011

SLOVAKIA

Phone: +421 (0)2 5939 6461

SLOVENIA

Phone: +386 (0)1 4701 666

SWITZERLAND

Phone: +41 (0)43 455 77 30

TAIWAN

Phone: +886-2-2721-8080

THAILAND

Phone: +66-2-078-1090

UK & IRELAND

Phone: +44 (0)1908-696-900

USA

Phone: +1-201-930-0100

VIETNAM

Phone: +84-24-3772-5555

